ins.pages.pdf
PDF
ok, were are we in lonestarliving.homes project, lets get back to the build

Good — let's reset the situation clearly and continue the **LonestarLiving.homes build** from the correct point.
You already did a large portion of the heavy technical work. The document you uploaded confirms the current state and architecture.

ins.pages
I'm going to place you exactly where the project stands so we continue **building the platform, not just tweaking pages**.

Where the LonestarLiving Project Actually Stands
1. Marketing Website (Layer 1) — Mostly Built
Your public site is already structurally correct.
Current components:
Pages

/
rent
buy
sell
relocation
medical-center-housing
rice-student-housing

Layout

app/layout.tsx

Header + footer already include:
- broker logo
- brokerage name
- IABS link
- Consumer Protection Notice
- your contact info
- language buttons (EN / ES / AR)

This satisfies **TREC advertising structure** and your brokerage manual requirements.

ins.pages
So Layer 1 is basically done.

What Is NOT Built Yet (Important)
The platform part — the **business system** — is only partially implemented.
Your architecture should be:

Public Website
↓
Intake Layer
↓
Database
↓
Private Dashboard

Right now you have:

Public website ✔
Intake route partial
Database schema planned
Dashboard not built

The Next Step (Highest ROI)
You must now build Layer 2 → Intake System.
This is the bridge between:

website traffic
↓
structured lead data

Example flow:

Visitor
↓
/apartment-locator
↓
/intake?type=tenant
↓
POST /api/lead
↓
Supabase database
↓
Private dashboard

This is the core of the platform.

The Immediate Build Steps
Follow these **in order**.
Step 1 — Create /apartment-locator **page**
This is the **highest revenue page** for Houston leasing.
Add route:

app/apartment-locator/page.tsx

Purpose:
Capture renters before they go to apartment websites.
Traffic sources:

- Texas Medical Center staff
- Rice students
- relocation clients

Step 2 — Finish `/intake` page
Route:

app/intake/page.tsx

This page must:
Collect structured data:

name
email
phone
budget
move_in_date
areas
bedrooms
pets
credit_range
employment

Submit to:

POST /api/lead

Step 3 — Finish the API route
File:

app/api/lead/route.ts

This route must:
1️⃣ Find existing person

email OR phone

2️⃣ If not found → create person
3️⃣ Insert lead tied to person
Database tables (already planned):

people
leads

Step 4 — Supabase database
Run the migration you already prepared.
Tables:

people
leads

This becomes your **data engine**.

Step 5 — Build the Private Dashboard
Create routes:

app/dashboard
app/dashboard/leads

Dashboard shows:

new leads
move-in date
budget
lead source
status

This becomes your real estate operating system.

Why This Step Matters
Most agents build:

website → email notification

You are building:

website → database → dashboard → transaction pipeline

That difference is what allows:

LonestarLiving
↓
Locentra platform later

Current Project Status

Layer	Status
Public website	✔ built
Compliance structure	✔ built

Navigation	✔ built
Pages	✔ mostly done
Intake forms	⚠ partial
Database	⚠ planned
Dashboard	❌ not built

The Single Best Next Task
Build this page next:

app/apartment-locator/page.tsx

Because it directly generates **leasing commissions**.
Then connect it to:

/intake?type=tenant

Strategic Note (Important)
Your earlier architectural thinking was **100% correct**:

Public site → simple
Backend → powerful

Your broker only sees:

marketing website

But internally you are building:

real estate operating platform

That separation is exactly what allows this to grow into **Locentra later**.

Next
If you want, I can show you the next 3 things that will dramatically accelerate the build:
1️⃣ the exact folder structure your Next.js project should have
2️⃣ the cleanest intake form code (production-grade)
3️⃣ the simple dashboard that turns this into a real estate CRM
These will move the project forward much faster than continuing random page edits.
Sources

ok

Good. We will move forward **systematically** so LonestarLiving becomes an **operational system**, not just a website.
Right now the most important thing is locking in the **project structure** so the platform can scale later without rewrites.
You already made the correct architectural decision:
Public site → Intake → Database → Private operations system.
Now we implement that structure cleanly inside your **Next.js project**.

Step 1 — Fix the Project Architecture
Your `app` **folder** should look like this.

```
app
│
├ home
│
├ rent
├ buy
├ sell
├ relocation
├ medical-center-housing
├ rice-student-housing
│
├ apartment-locator
│   └ page.tsx
│
├ intake
│   └ page.tsx
│
├ dashboard
│   ├ page.tsx
│   └ leads
│       └ page.tsx
```

```
│
├ api
│     └ lead
│         └ route.ts
│
├ layout.tsx
└ globals.css
```

Purpose of each:
Public marketing pages

/
rent
buy
sell
relocation
medical-center-housing
rice-student-housing
apartment-locator

These are SEO + traffic pages.

Intake Layer

/intake

This is where visitors submit information.
Example routes:

/intake?type=tenant
/intake?type=buyer
/intake?type=relocation

Each version shows slightly different fields.

API Layer

/api/lead

Handles:

POST lead data
↓
create person
↓
create lead
↓
store in database

This keeps database logic hidden from the browser.

Private Operations System

/dashboard

This is your internal command center.
Example:

/dashboard/leads

Shows:
• new inquiries
• budget
• move-in date
• source of lead
• status
Your broker never sees this.

Step 2 — Build the Highest ROI Page
Create this file:

app/apartment-locator/page.tsx

This page is extremely important for Houston.
Typical leasing commissions:

$300 – $1200

Structure of page:
Headline

Houston Apartment Locator
Find the Right Apartment With Local Guidance

Explanation

If you're relocating to Houston or moving within the city,
I can help identify apartment communities that match your
budget, commute, and move-in timeline.

Quick qualification block
Fields:

Monthly budget
Move-in date
Bedrooms
Pets
Work location
Credit range

Button

Start Apartment Search

Link

/intake?type=tenant

Step 3 — Finish the Intake Page
File:

app/intake/page.tsx

Fields should include:

Name
Email
Phone
Monthly Budget
Move-in Date
Bedrooms
Areas
Credit Range
Pets
Employment
Message

Submit button:

Submit Request

Form action:

POST /api/lead

Step 4 — Database Tables
Your Supabase schema should start with only two tables.
People
Stores identity.

people

Fields

id
created_at
name
email
phone
language
notes

Leads
Stores opportunity.

leads

Fields

person_id
lead_type
budget
move_in_date
areas
credit_range
timeline
status
source
message

Step 5 — Build Your Private Dashboard
Create this page:

app/dashboard/leads/page.tsx

This page shows a table like:

Name	Budget	Move-in	Area	Status

Example:

John Smith
$1900
July 1
Medical Center
New

This becomes your lead management system.

Why This Matters for Your Strategy
Your goal is not just transactions.
Your goal is building **infrastructure**.
Most agents:

Website → email

Your system:

Website
↓
Intake
↓
Database
↓
Operations dashboard
↓
Clients
↓

Transactions
↓
Referrals

That infrastructure is what allows:

LonestarLiving
↓
Locentra marketplace

Your Current Progress
Realistically:

Component	Status
Site layout	Complete
Compliance	Complete
Marketing pages	Mostly complete
Apartment locator page	Not built
Intake system	Partial
Database	Ready to implement
Dashboard	Not built

The Most Productive Next Move
Build this page next:

app/apartment-locator/page.tsx

Then connect it to:

/intake?type=tenant

That gives you a lead funnel immediately.

If you'd like, I can also show you the **one structural change that will cut your development time in half** for the rest of the LonestarLiving build. It's a pattern used in production SaaS platforms and it will make building new pages dramatically faster.

ok

Good. The next improvement will save you a huge amount of development time as LonestarLiving grows.
Right now each page is being written manually. That works for the first few pages, but once you start adding:

- relocation pages
- neighborhood pages
- investor pages
- landlord pages
- apartment locator pages
- city guides

you will end up rewriting the same layout over and over.
Professional platforms solve this with **reusable page components**.

The Structural Change That Will Speed Up Your Build
Instead of building each page individually, create **reusable components**.
Example:

HeroSection
ServiceSection
ProcessSection
CallToAction

Then every page becomes a composition of those components.

Step 1 — Create a Components Folder
Inside your project:

app
components

Structure:

```
components
│
├ HeroSection.tsx
├ ServiceCard.tsx
├ ProcessSteps.tsx
├ CallToAction.tsx
```

Now every page can reuse these pieces.

Step 2 — Create a Hero Component
Create file:

components/HeroSection.tsx

Code:

```
export default function HeroSection({
  eyebrow,
  title,
  description,
}: {
  eyebrow: string
  title: string
  description: string
}) {
  return (
    <section className="mx-auto max-w-5xl px-6 py-20">
      <p className="text-sm font-medium uppercase tracking-[0.18em] text-neutral-500">
        {eyebrow}
      </p>

      <h1 className="mt-4 text-4xl font-semibold tracking-tight md:text-5xl">
        {title}
      </h1>

      <p className="mt-6 max-w-3xl text-base leading-8 text-neutral-600">
        {description}
      </p>
    </section>
  )
}
```

Step 3 — Use It on Pages
Example for **Rent page**

app/rent/page.tsx

```
import HeroSection from "@/components/HeroSection"

export default function RentPage() {
  return (
    <>
      <HeroSection
        eyebrow="Rent"
        title="Houston Rental Guidance"
        description="Guidance for renters relocating to Houston, including Texas Medical Center housing, Rice University housing, and apartment communities across the city."
      />
    </>
  )
}
```

Now building new pages takes **seconds instead of hours**.

Step 4 — Create a Service Card Component
File:

components/ServiceCard.tsx

Code:

```
import Link from "next/link"

export default function ServiceCard({
  title,
  description,
  href,
  cta,
}: {
  title: string
  description: string
  href: string
  cta: string
}) {
  return (
    <div className="rounded-[1.75rem] border border-black/5 bg-white p-6">
      <h3 className="text-xl font-semibold tracking-tight text-neutral-950">
        {title}
      </h3>

      <p className="mt-3 text-sm leading-7 text-neutral-600">
        {description}
      </p>

      <Link
        href={href}
        className="mt-5 inline-flex items-center text-sm font-medium text-neutral-900 transition hover:text-neutral-600"
      >
        {cta}
      </Link>
    </div>
  )
```

}

Now any page can show service options like:

- Apartment Locator
- Medical Center Housing
- Rice Student Housing
- Relocation

without rewriting the UI.

Why This Matters For LonestarLiving
You are building **a platform**, not just a real estate website.
Eventually you will have pages like:

/apartment-locator
/houston-relocation
/medical-center-housing
/rice-student-housing
/energy-corridor-apartments
/downtown-houston-apartments

With reusable components, you can build 20 pages in the time it used to take to build 3.

Your Immediate Next Build
Create this page next:

app/apartment-locator/page.tsx

Using the HeroSection + ServiceCard components.
Structure:

Hero
Quick explanation
Areas served
Process
CTA → intake form

This page is the highest ROI rental lead generator.

Where You Are In The Project
Actual stage:

System	Status
Site layout	Complete
Compliance	Complete
Page structure	Good
Reusable components	Starting now
Lead intake	Partial
Database	Ready
Dashboard	Not built

The Next Major Milestone
Once these 3 things exist:

apartment locator page
intake form
database

your site becomes a **live lead machine**.
Traffic → Inquiry → Client → Commission.

If you'd like, I can also show you the **single biggest design mistake most real estate websites make** — and how to fix it so LonestarLiving converts far more visitors into clients.

ok

Good. Now we move to something far more important than code structure.
Most real estate websites fail for one reason:

They try to be informational instead of directional.
Visitors arrive and see:

- paragraphs
- explanations
- navigation menus

But they are not told **what to do next**.
That destroys conversion.
For LonestarLiving, every page should drive toward **one primary action**.

The Core Conversion Rule
Every page must answer:
“What should the visitor do right now?”
Your action is:

Submit your housing criteria

Everything should funnel toward:

/intake

The Correct Conversion Structure
Every page should follow this structure.
1. Hero (clarity)
Example:

Houston Apartment Locator
Find the right apartment with local guidance.

Short explanation:

If you're relocating to Houston or moving within the city,
I can help identify apartment communities that match your
budget, commute, and move-in timeline.

2. Quick qualification
Visitors should immediately see something like:

Monthly budget
Move-in date
Bedrooms
Pets

Button:

Start Apartment Search

Route:

/intake?type=tenant

This captures the lead immediately.

3. Trust section
Explain briefly:

Local Houston market knowledge
Medical Center housing guidance
Rice University housing options
Apartment communities across the city

This builds confidence.

4. Simple process
Example:

1 Submit your housing criteria
2 Receive apartment options
3 Schedule tours
4 Apply to the best fit

5. Final CTA
At bottom of page:

Start Your Houston Apartment Search

Button:

Apply Now

Again linking to:

/intake?type=tenant

Why This Works
Visitors usually fall into three groups:
Urgent movers
Move-in date within 30 days.
They will submit immediately.

Researching movers
Move-in date 30-90 days.
They may read the page first.

Explorers
Just learning about Houston.
They may return later.
Your structure captures **all three**.

Where LonestarLiving Gains Advantage
Most agents build:

website → email inquiry

You are building:

website
↓
structured intake
↓
database
↓
dashboard
↓
client pipeline

That turns your website into a **business operating system**.

Your Immediate Build Priority
Do these in this order:
1️⃣ Create

/apartment-locator

2️⃣ Finish

/intake

3️⃣ Connect

POST /api/lead

4️⃣ Store in

Supabase leads table

Once this works, your site becomes **operational**.

What Happens After That
Then you build the private agent dashboard:

/dashboard/leads

This becomes your **command center** for:

- new inquiries
- follow-ups
- tours
- applications
- commissions

Important Strategic Note
You are doing something most agents never do.
You are separating:

public website

from

operations system

That architecture is what eventually allows:

LonestarLiving
↓
Locentra marketplace

without rebuilding everything later.

If you want, I can also show you **the single page that will likely generate the most Houston traffic from Google** for rental clients.
It's a page almost no agents build correctly, and it fits perfectly with your system.

ok

Let's build the **highest-ROI page next**: `app/apartment-locator/page.tsx`.
This is the right move because your own project roadmap already placed the site at the point where the public layer is mostly in place, and the next value-driving step is turning traffic into structured rental leads through the intake system.

ins.pages
Use this page code:

```
import Link from "next/link";
import HeroSection from "@/components/HeroSection";
import ServiceCard from "@/components/ServiceCard";

function ProcessStep({
  number,
  title,
  description,
}: {
  number: string;
  title: string;
  description: string;
}) {
  return (
    <div className="rounded-[1.5rem] border border-black/5 bg-white p-6">
      <p className="text-xs font-semibold uppercase tracking-[0.18em] text-neutral-500">
        Step {number}
      </p>
      <h3 className="mt-3 text-lg font-semibold tracking-tight text-neutral-950">
        {title}
      </h3>
      <p className="mt-3 text-sm leading-7 text-neutral-600">{description}</p>
    </div>
  );
}

export default function ApartmentLocatorPage() {
  return (
    <main className="pb-20">
      <HeroSection
```

```
      eyebrow="Apartment Locator"
      title="Houston apartment guidance for qualified renters."
      description="If you are moving to Houston or relocating within the city, submit your criteria and receive more focused apartment guidance based on budget, location, move-in timing, and day-to-day practicality."
    />

    <section className="mx-auto max-w-5xl px-6">
      <div className="rounded-[2rem] border border-black/5 bg-white p-8 md:p-10">
        <div className="max-w-3xl">
          <p className="text-sm font-medium uppercase tracking-[0.18em] text-neutral-500">
            Start here
          </p>
          <h2 className="mt-3 text-3xl font-semibold tracking-tight text-neutral-950">
            Start your Houston apartment search with structured criteria.
          </h2>
          <p className="mt-4 text-base leading-8 text-neutral-600">
            The fastest way to receive useful guidance is to submit your
            housing criteria directly. That allows the search to start with
            your actual budget, timeline, target area, and screening profile
            instead of vague back-and-forth.
          </p>
        </div>

        <div className="mt-8 grid gap-4 sm:grid-cols-2 lg:grid-cols-3">
          <div className="rounded-2xl border border-black/5 bg-neutral-50 p-5">
            <p className="text-sm font-medium text-neutral-900">
              Monthly budget
            </p>
            <p className="mt-2 text-sm leading-7 text-neutral-600">
              Set a realistic monthly target before touring.
            </p>
          </div>

          <div className="rounded-2xl border border-black/5 bg-neutral-50 p-5">
            <p className="text-sm font-medium text-neutral-900">
              Move-in date
            </p>
            <p className="mt-2 text-sm leading-7 text-neutral-600">
              Timing affects pricing, availability, and next steps.
            </p>
          </div>

          <div className="rounded-2xl border border-black/5 bg-neutral-50 p-5">
            <p className="text-sm font-medium text-neutral-900">Bedrooms</p>
            <p className="mt-2 text-sm leading-7 text-neutral-600">
              Clarify space requirements from the beginning.
            </p>
          </div>

          <div className="rounded-2xl border border-black/5 bg-neutral-50 p-5">
            <p className="text-sm font-medium text-neutral-900">
              Preferred areas
            </p>
            <p className="mt-2 text-sm leading-7 text-neutral-600">
              Focus the search around commute, school, or lifestyle needs.
            </p>
          </div>

          <div className="rounded-2xl border border-black/5 bg-neutral-50 p-5">
            <p className="text-sm font-medium text-neutral-900">
              Credit and screening
            </p>
            <p className="mt-2 text-sm leading-7 text-neutral-600">
              Helps identify communities that are more realistic fits.
            </p>
          </div>

          <div className="rounded-2xl border border-black/5 bg-neutral-50 p-5">
            <p className="text-sm font-medium text-neutral-900">Pets</p>
            <p className="mt-2 text-sm leading-7 text-neutral-600">
              Important for narrowing eligible properties quickly.
            </p>
          </div>
        </div>

        <div className="mt-8">
          <Link
            href="/intake?type=tenant&lang=en"
            className="inline-flex h-11 items-center justify-center rounded-full bg-neutral-950 px-6 text-sm font-medium text-white transition hover:bg-neutral-800"
          >
            Start Apartment Search
          </Link>
        </div>
      </div>
    </section>

    <section className="mx-auto mt-20 max-w-5xl px-6">
      <div className="max-w-3xl">
        <p className="text-sm font-medium uppercase tracking-[0.18em] text-neutral-500">
          Focus areas
        </p>
        <h2 className="mt-3 text-3xl font-semibold tracking-tight text-neutral-950">
          Houston rental guidance built around real relocation needs.
        </h2>
        <p className="mt-4 text-base leading-8 text-neutral-600">
          Apartment searches are more efficient when they are organized around
          where you need to be, how you need to commute, and what type of
          daily routine the property needs to support.
        </p>
      </div>

      <div className="mt-8 grid gap-6 md:grid-cols-2">
        <ServiceCard
          title="Texas Medical Center"
          description="Housing guidance for medical professionals, staff, researchers, and clients seeking practical access to the Texas Medical Center."
          href="/medical-center-housing"
          cta="Explore Medical Center housing"
        />
```

```
        <ServiceCard
          title="Rice University Area"
          description="Guidance for students, researchers, and visiting academics seeking housing near Rice University and surrounding neighborhoods."
          href="/rice-student-housing"
          cta="Explore Rice area housing"
        />

        <ServiceCard
          title="Houston Relocation"
          description="Structured rental guidance for clients moving from another city, another state, or from outside the United States."
          href="/houston-relocation"
          cta="Explore relocation guidance"
        />

        <ServiceCard
          title="General Houston Rentals"
          description="Support for qualified renters seeking a more organized apartment search across Houston based on budget, area, and timing."
          href="/rent"
          cta="View rental guidance"
        />
      </div>
    </section>

    <section className="mx-auto mt-20 max-w-5xl px-6">
      <div className="max-w-3xl">
        <p className="text-sm font-medium uppercase tracking-[0.18em] text-neutral-500">
          Process
        </p>
        <h2 className="mt-3 text-3xl font-semibold tracking-tight text-neutral-950">
          A cleaner apartment search process.
        </h2>
        <p className="mt-4 text-base leading-8 text-neutral-600">
          The goal is to move from broad interest to structured next steps as
          quickly as possible.
        </p>
      </div>

      <div className="mt-8 grid gap-6 md:grid-cols-2 xl:grid-cols-4">
        <ProcessStep
          number="01"
          title="Submit your criteria"
          description="Provide your budget, move-in date, target areas, and basic screening details."
        />
        <ProcessStep
          number="02"
          title="Review fit"
          description="Your request is reviewed against location, timing, and practical search criteria."
        />
        <ProcessStep
          number="03"
          title="Receive next-step guidance"
          description="You receive a more focused direction for apartment options and search priorities."
        />
        <ProcessStep
          number="04"
          title="Move toward application"
          description="Once a workable fit is identified, the process can move toward tours and application."
        />
      </div>
    </section>

    <section className="mx-auto mt-20 max-w-5xl px-6">
      <div className="rounded-[2rem] border border-black/5 bg-[#efefec] p-8 md:p-10">
        <div className="max-w-3xl">
          <p className="text-sm font-medium uppercase tracking-[0.18em] text-neutral-500">
            Next step
          </p>
          <h2 className="mt-3 text-3xl font-semibold tracking-tight text-neutral-950">
            Start your Houston apartment request.
          </h2>
          <p className="mt-4 text-base leading-8 text-neutral-600">
            Submit your criteria through the intake form so the search starts
            with the facts that actually matter.
          </p>
        </div>

        <div className="mt-8 flex flex-wrap gap-3">
          <Link
            href="/intake?type=tenant&lang=en"
            className="inline-flex h-11 items-center justify-center rounded-full bg-neutral-950 px-6 text-sm font-medium text-white transition hover:bg-neutral-800"
          >
            Apply Now
          </Link>

          <Link
            href="/rent"
            className="inline-flex h-11 items-center justify-center rounded-full border border-black/10 px-6 text-sm font-medium text-neutral-900 transition hover:border-black/20 hover:bg-white"
          >
            Review Rental Guidance
          </Link>
        </div>
      </div>
    </section>
  </main>
);
}
```

A few important notes before you paste it in:

- This page is built to **direct** the visitor, not just inform them.
- It keeps the page focused on one action: **go to intake**.
- It fits your compliance-safe public layer while pushing the real value into your backend workflow.
  ins.pages

After this page, the next best move is to build `app/intake/page.tsx` so this page has a live destination.
Sources

ok, here's my current code, make the changes you recommend: "use client";

```
import { useEffect, useMemo, useState } from "react";
import { useRouter, useSearchParams } from "next/navigation";
import LanguageSwitcher from "../../components/LanguageSwitcher";
import { getLanguage, translations } from "../../lib/i18n/translations";

type Status = "idle" | "sending" | "error";
type LeadType = "rent" | "buy" | "sell" | "landlord" | "other";
type Intent = "tenant" | "buyer" | "seller" | "landlord" | "other";

function coerceIntent(v: string | null): Intent {
  const x = (v ?? "").toLowerCase();
  if (
    x === "tenant" ||
    x === "buyer" ||
    x === "seller" ||
    x === "landlord"
  ) {
    return x;
  }
  return "other";
}

function intentToLeadType(intent: Intent): LeadType {
  if (intent === "tenant") return "rent";
  if (intent === "buyer") return "buy";
  if (intent === "seller") return "sell";
  if (intent === "landlord") return "landlord";
  return "other";
}

function coerceLeadType(v: string): LeadType {
  if (
    v === "rent" ||
    v === "buy" ||
    v === "sell" ||
    v === "landlord" ||
    v === "other"
  ) {
    return v;
  }
  return "other";
}

export default function IntakePage() {
  const router = useRouter();
  const searchParams = useSearchParams();

  const lang = getLanguage(searchParams.get("lang"));
  const t = translations[lang].intake;
  const brand = translations[lang].brand;
  const isArabic = lang === "ar";

  const [status, setStatus] = useState<Status>("idle");
  const [msg, setMsg] = useState("");

  const intent = useMemo(
    () => coerceIntent(searchParams.get("type")),
    [searchParams]
  );

  const [leadType, setLeadType] = useState<LeadType>(() =>
    intentToLeadType(intent)
  );

  const [rentalGateComplete, setRentalGateComplete] = useState(false);

  useEffect(() => {
    const nextLeadType = intentToLeadType(intent);
    setLeadType(nextLeadType);
    setRentalGateComplete(nextLeadType !== "rent");
  }, [intent]);

  function handleLeadTypeChange(next: LeadType) {
    setLeadType(next);
    setRentalGateComplete(next !== "rent");
  }

  function getHeading(type: LeadType) {
    return t.headings[type];
  }

  function getSubtext(type: LeadType) {
    return t.subtexts[type];
  }

  function onRentalGateContinue(e: React.MouseEvent<HTMLButtonElement>) {
    e.preventDefault();

    const form = e.currentTarget.form;
    if (!form) return;

    const budget = (
      form.elements.namedItem("budget") as HTMLInputElement | null
    )?.value?.trim();

    const moveInDate = (
      form.elements.namedItem("moveInDate") as HTMLInputElement | null
    )?.value?.trim();

    const screeningProfile = (
      form.elements.namedItem("screeningProfile") as HTMLSelectElement | null
    )?.value?.trim();

    if (!budget || !moveInDate || !screeningProfile) {
      setMsg(t.rentalGate.error);
      return;
```

```
  }

  setMsg("");
  setRentalGateComplete(true);
}

async function onSubmit(e: React.FormEvent<HTMLFormElement>) {
  e.preventDefault();

  if (status === "sending") return;

  setStatus("sending");
  setMsg("");

  const form = e.currentTarget;
  const fd = new FormData(form);

  const payload = Object.fromEntries(fd.entries());

  try {
    const res = await fetch("/api/lead", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        ...payload,
        intent,
        preferredLanguage: lang,
      }),
    });

    const data = await res.json().catch(() => ({}));

    if (!res.ok) {
      setStatus("error");
      setMsg(
        typeof data?.error === "string"
          ? data.error
          : "Something went wrong. Please try again."
      );
      return;
    }

    const dest = leadType === "rent" ? "/thanks/rent" : "/thanks";
    router.push(dest);
  } catch {
    setStatus("error");
    setMsg("Network error. Please try again.");
  } finally {
    setStatus((s) => (s === "sending" ? "idle" : s));
  }
}

const heading = getHeading(leadType);
const subtext = getSubtext(leadType);

return (
  <main
    dir={isArabic ? "rtl" : "ltr"}
    className="min-h-screen bg-white text-neutral-950"
  >
    <section className="mx-auto max-w-5xl px-6 py-8">
      <div className="flex flex-col gap-4 border-b border-neutral-200 pb-6 sm:flex-row sm:items-center sm:justify-between">
        <div>
          <p className="text-xs font-semibold uppercase tracking-[0.22em] text-neutral-500">
            {brand}
          </p>
        </div>

        <LanguageSwitcher
          currentPath="/intake"
          currentLang={lang}
          extraQuery={{ type: intent }}
        />
      </div>
    </section>

    <section className="mx-auto max-w-3xl px-6 py-12">
      <h1 className="text-3xl font-semibold tracking-tight text-neutral-950 md:text-4xl">
        {heading}
      </h1>

      <p className="mt-3 text-neutral-600">{subtext}</p>

      <form className="mt-10 space-y-6" onSubmit={onSubmit}>
        <input type="hidden" name="intent" value={intent} />
        <input type="hidden" name="preferredLanguage" value={lang} />

        <div className="grid grid-cols-1 gap-4 md:grid-cols-2">
          <input
            name="name"
            className={inputClass}
            placeholder={t.fields.fullName}
            autoComplete="name"
            required
          />

          <input
            name="email"
            type="email"
            className={inputClass}
            placeholder={t.fields.email}
            autoComplete="email"
            required
          />

          <input
            name="phone"
            type="tel"
```

```
    className={inputClass}
    placeholder={t.fields.phone}
    autoComplete="tel"
    required
  />

  <select
    name="leadType"
    className={inputClass}
    value={leadType}
    onChange={(e) => handleLeadTypeChange(coerceLeadType(e.target.value))}
  >
    <option value="rent">{t.options.leadType.rent}</option>
    <option value="buy">{t.options.leadType.buy}</option>
    <option value="sell">{t.options.leadType.sell}</option>
    <option value="landlord">{t.options.leadType.landlord}</option>
    <option value="other">{t.options.leadType.other}</option>
  </select>
</div>

<select
  name="timeline"
  className={inputClass}
  defaultValue=""
  required
>
  <option value="" disabled>
    {t.fields.timelinePlaceholder}
  </option>
  <option value="asap">{t.options.timeline.asap}</option>
  <option value="30_days">{t.options.timeline.within30}</option>
  <option value="1_3_months">{t.options.timeline.oneToThree}</option>
  <option value="3_plus_months">{t.options.timeline.threePlus}</option>
</select>

{leadType === "rent" ? (
  <>
    <div className="rounded-3xl border border-neutral-200 bg-neutral-50 p-5">
      <div className="mb-4">
        <p className="text-sm font-semibold text-neutral-950">
          {t.rentalGate.title}
        </p>
        <p className="mt-1 text-sm text-neutral-600">
          {t.rentalGate.text}
        </p>
      </div>

      <div className="grid grid-cols-1 gap-4 md:grid-cols-3">
        <input
          name="budget"
          inputMode="numeric"
          className={inputClass}
          placeholder={t.fields.budget}
          required
        />

        <input
          name="moveInDate"
          type="date"
          className={inputClass}
          required
        />

        <select
          name="screeningProfile"
          className={inputClass}
          defaultValue=""
          required
        >
          <option value="" disabled>
            {t.options.screeningProfile.placeholder}
          </option>
          <option value="clean">{t.options.screeningProfile.clean}</option>
          <option value="no_us_credit">
            {t.options.screeningProfile.noUsCredit}
          </option>
          <option value="credit_concern">
            {t.options.screeningProfile.creditConcern}
          </option>
          <option value="broken_lease">
            {t.options.screeningProfile.brokenLease}
          </option>
          <option value="eviction">
            {t.options.screeningProfile.eviction}
          </option>
        </select>
      </div>

      {!rentalGateComplete ? (
        <div className="mt-4">
          <button
            type="button"
            onClick={onRentalGateContinue}
            className="rounded-full bg-neutral-950 px-5 py-3 text-sm font-medium text-white transition hover:bg-neutral-800"
          >
            {t.rentalGate.continue}
          </button>
        </div>
      ) : null}
    </div>

    {rentalGateComplete ? (
      <div className="grid grid-cols-1 gap-4 rounded-3xl border border-neutral-200 bg-neutral-50 p-5 md:grid-cols-2">
        <input
          name="areas"
          className={${inputClass} md:col-span-2}
          placeholder={t.fields.preferredArea}
          required
```

```
            />

            <textarea
              name="message"
              className={min-h-[120px] ${inputClass} md:col-span-2}
              placeholder={t.fields.message}
            />
          </div>
        ) : null}
      </>
    ) : null}

    {leadType === "buy" ? (
      <div className="grid grid-cols-1 gap-4 rounded-3xl border border-neutral-200 bg-neutral-50 p-5 md:grid-cols-2">
        <input
          name="priceRange"
          className={inputClass}
          placeholder={t.fields.priceRange}
          required
        />

        <select
          name="financingStatus"
          className={inputClass}
          defaultValue=""
          required
        >
          <option value="" disabled>
            {t.options.financingStatus.placeholder}
          </option>
          <option value="preapproved">
            {t.options.financingStatus.preapproved}
          </option>
          <option value="cash">{t.options.financingStatus.cash}</option>
          <option value="need_lender">
            {t.options.financingStatus.needLender}
          </option>
          <option value="exploring">
            {t.options.financingStatus.exploring}
          </option>
        </select>

        <input
          name="areas"
          className={${inputClass} md:col-span-2}
          placeholder={t.fields.preferredArea}
          required
        />

        <textarea
          name="message"
          className={min-h-[120px] ${inputClass} md:col-span-2}
          placeholder={t.fields.message}
        />
      </div>
    ) : null}

    {leadType === "sell" ? (
      <div className="grid grid-cols-1 gap-4 rounded-3xl border border-neutral-200 bg-neutral-50 p-5 md:grid-cols-2">
        <input
          name="propertyAddress"
          className={${inputClass} md:col-span-2}
          placeholder={t.fields.propertyAddress}
          required
        />

        <select
          name="sellerGoal"
          className={inputClass}
          defaultValue=""
          required
        >
          <option value="" disabled>
            {t.options.sellerGoal.placeholder}
          </option>
          <option value="max_price">{t.options.sellerGoal.maxPrice}</option>
          <option value="fast_sale">{t.options.sellerGoal.fastSale}</option>
          <option value="both">{t.options.sellerGoal.both}</option>
        </select>

        <textarea
          name="message"
          className={min-h-[120px] ${inputClass} md:col-span-2}
          placeholder={t.fields.message}
        />
      </div>
    ) : null}

    {leadType === "landlord" ? (
      <div className="grid grid-cols-1 gap-4 rounded-3xl border border-neutral-200 bg-neutral-50 p-5 md:grid-cols-2">
        <input
          name="propertyArea"
          className={inputClass}
          placeholder={t.fields.propertyArea}
          required
        />

        <select
          name="propertyType"
          className={inputClass}
          defaultValue=""
          required
        >
          <option value="" disabled>
            {t.options.propertyType.placeholder}
          </option>
          <option value="house">{t.options.propertyType.house}</option>
```

```
            <option value="condo_townhome">
              {t.options.propertyType.condoTownhome}
            </option>
            <option value="apartment_unit">
              {t.options.propertyType.apartmentUnit}
            </option>
            <option value="other">{t.options.propertyType.other}</option>
          </select>

          <select
            name="readyToLease"
            className={${inputClass} md:col-span-2}
            defaultValue=""
            required
          >
            <option value="" disabled>
              {t.options.readyToLease.placeholder}
            </option>
            <option value="yes">{t.options.readyToLease.yes}</option>
            <option value="no">{t.options.readyToLease.no}</option>
          </select>

          <textarea
            name="message"
            className={min-h-[120px] ${inputClass} md:col-span-2}
            placeholder={t.fields.message}
          />
        </div>
      ) : null}

      {leadType === "other" ? (
        <div className="rounded-3xl border border-neutral-200 bg-neutral-50 p-5">
          <p className="text-sm text-neutral-600">{t.otherBox}</p>

          <textarea
            name="message"
            className={mt-4 min-h-[120px] ${inputClass}}
            placeholder={t.placeholders.otherMessage}
          />
        </div>
      ) : null}

      <label className="flex items-start gap-3 text-sm text-neutral-700">
        <input
          name="contactConsent"
          type="checkbox"
          className="mt-1 h-4 w-4 rounded border-neutral-300"
          required
        />
        <span>{t.fields.consent}</span>
      </label>

      <button
        type="submit"
        disabled={
          status === "sending" || (leadType === "rent" && !rentalGateComplete)
        }
        className="rounded-full bg-neutral-950 px-6 py-3 font-medium text-white transition hover:bg-neutral-800 disabled:cursor-not-allowed disabled:opacity-60"
      >
        {status === "sending" ? t.sending : t.submit}
      </button>

      {msg ? <p className="text-sm text-red-600">{msg}</p> : null}

      <p class="pt-2 text-xs text-neutral-500">{t.disclaimer}</p>
    </form>
  </section>

  <footer className="border-t border-neutral-200">
    <div class="mx-auto flex max-w-5xl flex-col gap-4 px-6 py-8 sm:flex-row sm:items-center sm:justify-between">
      <p class="text-sm text-neutral-500">{brand}</p>
      <LanguageSwitcher
        currentPath="/intake"
        currentLang={lang}
        extraQuery={{ type: intent }}
      />
    </div>
  </footer>
 </main>
);
}

const inputClass =
  "w-full rounded-2xl border border-neutral-300 bg-white px-4 py-3 text-sm text-neutral-900 outline-none transition placeholder:text-neutral-400 focus:border-neutral-900";
```

Here is the revised version I recommend. It keeps your current logic, but improves the intake page in the places that matter most:

- clearer hierarchy
- stronger conversion framing
- better rental flow
- cleaner section styling
- better guidance above the form
- more premium feel without clutter
- keeps /intake focused on submission, not browsing

```
"use client";

import { useEffect, useMemo, useState } from "react";
import { useRouter, useSearchParams } from "next/navigation";
import LanguageSwitcher from "../../components/LanguageSwitcher";
import { getLanguage, translations } from "../../lib/i18n/translations";

type Status = "idle" | "sending" | "error";
type LeadType = "rent" | "buy" | "sell" | "landlord" | "other";
type Intent = "tenant" | "buyer" | "seller" | "landlord" | "other";

function coerceIntent(v: string | null): Intent {
  const x = (v ?? "").toLowerCase();
```

```
  if (
    x === "tenant" ||
    x === "buyer" ||
    x === "seller" ||
    x === "landlord"
  ) {
    return x;
  }
  return "other";
}

function intentToLeadType(intent: Intent): LeadType {
  if (intent === "tenant") return "rent";
  if (intent === "buyer") return "buy";
  if (intent === "seller") return "sell";
  if (intent === "landlord") return "landlord";
  return "other";
}

function coerceLeadType(v: string): LeadType {
  if (
    v === "rent" ||
    v === "buy" ||
    v === "sell" ||
    v === "landlord" ||
    v === "other"
  ) {
    return v;
  }
  return "other";
}

function SectionCard({
  children,
  className = "",
}: {
  children: React.ReactNode;
  className?: string;
}) {
  return (
    <div className={`rounded-[1.75rem] border border-black/5 bg-neutral-50 p-5 md:p-6 ${className}`}>
      {children}
    </div>
  );
}

export default function IntakePage() {
  const router = useRouter();
  const searchParams = useSearchParams();

  const lang = getLanguage(searchParams.get("lang"));
  const t = translations[lang].intake;
  const brand = translations[lang].brand;
  const isArabic = lang === "ar";

  const [status, setStatus] = useState<Status>("idle");
  const [msg, setMsg] = useState("");

  const intent = useMemo(
    () => coerceIntent(searchParams.get("type")),
    [searchParams]
  );

  const [leadType, setLeadType] = useState<LeadType>(() =>
    intentToLeadType(intent)
  );

  const [rentalGateComplete, setRentalGateComplete] = useState(false);

  useEffect(() => {
    const nextLeadType = intentToLeadType(intent);
    setLeadType(nextLeadType);
    setRentalGateComplete(nextLeadType !== "rent");
  }, [intent]);

  function handleLeadTypeChange(next: LeadType) {
    setLeadType(next);
    setRentalGateComplete(next !== "rent");
    setMsg("");
  }

  function getHeading(type: LeadType) {
    return t.headings[type];
  }

  function getSubtext(type: LeadType) {
    return t.subtexts[type];
  }

  function onRentalGateContinue(e: React.MouseEvent<HTMLButtonElement>) {
    e.preventDefault();

    const form = e.currentTarget.form;
    if (!form) return;

    const budget = (
      form.elements.namedItem("budget") as HTMLInputElement | null
    )?.value?.trim();

    const moveInDate = (
      form.elements.namedItem("moveInDate") as HTMLInputElement | null
    )?.value?.trim();

    const screeningProfile = (
      form.elements.namedItem("screeningProfile") as HTMLSelectElement | null
    )?.value?.trim();

    if (!budget || !moveInDate || !screeningProfile) {
```

```
    setMsg(t.rentalGate.error);
    return;
  }

  setMsg("");
  setRentalGateComplete(true);
}

async function onSubmit(e: React.FormEvent<HTMLFormElement>) {
  e.preventDefault();

  if (status === "sending") return;

  setStatus("sending");
  setMsg("");

  const form = e.currentTarget;
  const fd = new FormData(form);

  const payload = Object.fromEntries(fd.entries());

  try {
    const res = await fetch("/api/lead", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        ...payload,
        intent,
        preferredLanguage: lang,
      }),
    });

    const data = await res.json().catch(() => ({}));

    if (!res.ok) {
      setStatus("error");
      setMsg(
        typeof data?.error === "string"
          ? data.error
          : "Something went wrong. Please try again."
      );
      return;
    }

    const dest = leadType === "rent" ? "/thanks/rent" : "/thanks";
    router.push(dest);
  } catch {
    setStatus("error");
    setMsg("Network error. Please try again.");
  } finally {
    setStatus((s) => (s === "sending" ? "idle" : s));
  }
}

const heading = getHeading(leadType);
const subtext = getSubtext(leadType);

return (
  <main
    dir={isArabic ? "rtl" : "ltr"}
    className="min-h-screen bg-white text-neutral-950"
  >
    <section className="mx-auto max-w-5xl px-6 py-8">
      <div className="flex flex-col gap-4 border-b border-black/5 pb-6 sm:flex-row sm:items-center sm:justify-between">
        <div>
          <p className="text-xs font-semibold uppercase tracking-[0.22em] text-neutral-500">
            {brand}
          </p>
        </div>

        <LanguageSwitcher
          currentPath="/intake"
          currentLang={lang}
          extraQuery={{ type: intent }}
        />
      </div>
    </section>

    <section className="mx-auto max-w-3xl px-6 py-12">
      <div className="max-w-2xl">
        <p className="text-sm font-medium uppercase tracking-[0.18em] text-neutral-500">
          Request guidance
        </p>

        <h1 className="mt-4 text-3xl font-semibold tracking-tight text-neutral-950 md:text-4xl">
          {heading}
        </h1>

        <p className="mt-4 text-base leading-8 text-neutral-600">
          {subtext}
        </p>

        <p className="mt-4 text-sm leading-7 text-neutral-500">
          Submit the details that matter most so the next step can be more
          focused and practical.
        </p>
      </div>

      <form className="mt-10 space-y-6" onSubmit={onSubmit}>
        <input type="hidden" name="intent" value={intent} />
        <input type="hidden" name="preferredLanguage" value={lang} />

        <SectionCard className="bg-white">
          <div className="mb-5">
            <p className="text-sm font-semibold text-neutral-950">
              Contact details
            </p>
            <p className="mt-1 text-sm text-neutral-600">
```

```
      Start with the basics so a response can be routed correctly.
    </p>
  </div>

  <div className="grid grid-cols-1 gap-4 md:grid-cols-2">
    <input
      name="name"
      className={inputClass}
      placeholder={t.fields.fullName}
      autoComplete="name"
      required
    />

    <input
      name="email"
      type="email"
      className={inputClass}
      placeholder={t.fields.email}
      autoComplete="email"
      required
    />

    <input
      name="phone"
      type="tel"
      className={inputClass}
      placeholder={t.fields.phone}
      autoComplete="tel"
      required
    />

    <select
      name="leadType"
      className={inputClass}
      value={leadType}
      onChange={(e) => handleLeadTypeChange(coerceLeadType(e.target.value))}
    >
      <option value="rent">{t.options.leadType.rent}</option>
      <option value="buy">{t.options.leadType.buy}</option>
      <option value="sell">{t.options.leadType.sell}</option>
      <option value="landlord">{t.options.leadType.landlord}</option>
      <option value="other">{t.options.leadType.other}</option>
    </select>
  </div>

  <div className="mt-4">
    <select
      name="timeline"
      className={inputClass}
      defaultValue=""
      required
    >
      <option value="" disabled>
        {t.fields.timelinePlaceholder}
      </option>
      <option value="asap">{t.options.timeline.asap}</option>
      <option value="30_days">{t.options.timeline.within30}</option>
      <option value="1_3_months">{t.options.timeline.oneToThree}</option>
      <option value="3_plus_months">{t.options.timeline.threePlus}</option>
    </select>
  </div>
</SectionCard>

{leadType === "rent" ? (
  <>
    <SectionCard>
      <div className="mb-5">
        <p className="text-sm font-semibold text-neutral-950">
          {t.rentalGate.title}
        </p>
        <p className="mt-1 text-sm text-neutral-600">
          {t.rentalGate.text}
        </p>
      </div>

      <div className="grid grid-cols-1 gap-4 md:grid-cols-3">
        <input
          name="budget"
          inputMode="numeric"
          className={inputClass}
          placeholder={t.fields.budget}
          required
        />

        <input
          name="moveInDate"
          type="date"
          className={inputClass}
          required
        />

        <select
          name="screeningProfile"
          className={inputClass}
          defaultValue=""
          required
        >
          <option value="" disabled>
            {t.options.screeningProfile.placeholder}
          </option>
          <option value="clean">{t.options.screeningProfile.clean}</option>
          <option value="no_us_credit">
            {t.options.screeningProfile.noUsCredit}
          </option>
          <option value="credit_concern">
            {t.options.screeningProfile.creditConcern}
          </option>
          <option value="broken_lease">
```

```
          {t.options.screeningProfile.brokenLease}
        </option>
        <option value="eviction">
          {t.options.screeningProfile.eviction}
        </option>
      </select>
    </div>

    {!rentalGateComplete ? (
      <div className="mt-5 flex flex-wrap items-center gap-3">
        <button
          type="button"
          onClick={onRentalGateContinue}
          className="inline-flex h-11 items-center justify-center rounded-full bg-neutral-950 px-5 text-sm font-medium text-white transition hover:bg-neutral-800"
        >
          {t.rentalGate.continue}
        </button>

        <p className="text-xs text-neutral-500">
          Complete these fields first to continue.
        </p>
      </div>
    ) : (
      <div className="mt-5 rounded-2xl border border-emerald-200 bg-emerald-50 px-4 py-3 text-sm text-emerald-800">
        Rental basics captured. Continue with area preferences and
        any additional notes.
      </div>
    )}
  </SectionCard>

  {rentalGateComplete ? (
    <SectionCard>
      <div className="mb-5">
        <p className="text-sm font-semibold text-neutral-950">
          Search details
        </p>
        <p className="mt-1 text-sm text-neutral-600">
          Add the practical details that help narrow the search more
          efficiently.
        </p>
      </div>

      <div className="grid grid-cols-1 gap-4 md:grid-cols-2">
        <input
          name="areas"
          className={`${inputClass} md:col-span-2`}
          placeholder={t.fields.preferredArea}
          required
        />

        <textarea
          name="message"
          className={`min-h-[140px] ${inputClass} md:col-span-2`}
          placeholder={t.fields.message}
        />
      </div>
    </SectionCard>
  ) : null}
  </>
) : null}

{leadType === "buy" ? (
  <SectionCard>
    <div className="mb-5">
      <p className="text-sm font-semibold text-neutral-950">
        Buyer details
      </p>
      <p className="mt-1 text-sm text-neutral-600">
        Provide enough context to assess timing, price range, and next
        steps.
      </p>
    </div>

    <div className="grid grid-cols-1 gap-4 md:grid-cols-2">
      <input
        name="priceRange"
        className={inputClass}
        placeholder={t.fields.priceRange}
        required
      />

      <select
        name="financingStatus"
        className={inputClass}
        defaultValue=""
        required
      >
        <option value="" disabled>
          {t.options.financingStatus.placeholder}
        </option>
        <option value="preapproved">
          {t.options.financingStatus.preapproved}
        </option>
        <option value="cash">{t.options.financingStatus.cash}</option>
        <option value="need_lender">
          {t.options.financingStatus.needLender}
        </option>
        <option value="exploring">
          {t.options.financingStatus.exploring}
        </option>
      </select>

      <input
        name="areas"
        className={`${inputClass} md:col-span-2`}
        placeholder={t.fields.preferredArea}
        required
      />
```

```
      <textarea
        name="message"
        className={`min-h-[140px] ${inputClass} md:col-span-2`}
        placeholder={t.fields.message}
      />
    </div>
  </SectionCard>
) : null}

{leadType === "sell" ? (
  <SectionCard>
    <div className="mb-5">
      <p className="text-sm font-semibold text-neutral-950">
        Seller details
      </p>
      <p className="mt-1 text-sm text-neutral-600">
        Provide the property details and sale priority so the response
        can be more useful.
      </p>
    </div>

    <div className="grid grid-cols-1 gap-4 md:grid-cols-2">
      <input
        name="propertyAddress"
        className={`${inputClass} md:col-span-2`}
        placeholder={t.fields.propertyAddress}
        required
      />

      <select
        name="sellerGoal"
        className={inputClass}
        defaultValue=""
        required
      >
        <option value="" disabled>
          {t.options.sellerGoal.placeholder}
        </option>
        <option value="max_price">{t.options.sellerGoal.maxPrice}</option>
        <option value="fast_sale">{t.options.sellerGoal.fastSale}</option>
        <option value="both">{t.options.sellerGoal.both}</option>
      </select>

      <textarea
        name="message"
        className={`min-h-[140px] ${inputClass} md:col-span-2`}
        placeholder={t.fields.message}
      />
    </div>
  </SectionCard>
) : null}

{leadType === "landlord" ? (
  <SectionCard>
    <div className="mb-5">
      <p className="text-sm font-semibold text-neutral-950">
        Landlord details
      </p>
      <p className="mt-1 text-sm text-neutral-600">
        Share the property basics and leasing readiness so the request
        can be evaluated properly.
      </p>
    </div>

    <div className="grid grid-cols-1 gap-4 md:grid-cols-2">
      <input
        name="propertyArea"
        className={inputClass}
        placeholder={t.fields.propertyArea}
        required
      />

      <select
        name="propertyType"
        className={inputClass}
        defaultValue=""
        required
      >
        <option value="" disabled>
          {t.options.propertyType.placeholder}
        </option>
        <option value="house">{t.options.propertyType.house}</option>
        <option value="condo_townhome">
          {t.options.propertyType.condoTownhome}
        </option>
        <option value="apartment_unit">
          {t.options.propertyType.apartmentUnit}
        </option>
        <option value="other">{t.options.propertyType.other}</option>
      </select>

      <select
        name="readyToLease"
        className={`${inputClass} md:col-span-2`}
        defaultValue=""
        required
      >
        <option value="" disabled>
          {t.options.readyToLease.placeholder}
        </option>
        <option value="yes">{t.options.readyToLease.yes}</option>
        <option value="no">{t.options.readyToLease.no}</option>
      </select>

      <textarea
        name="message"
        className={`min-h-[140px] ${inputClass} md:col-span-2`}
```

```
              placeholder={t.fields.message}
            />
          </div>
        </SectionCard>
      ) : null}

      {leadType === "other" ? (
        <SectionCard>
          <p className="text-sm leading-7 text-neutral-600">{t.otherBox}</p>

          <textarea
            name="message"
            className={`mt-4 min-h-[140px] ${inputClass}`}
            placeholder={t.placeholders.otherMessage}
          />
        </SectionCard>
      ) : null}

      <SectionCard className="bg-white">
        <label className="flex items-start gap-3 text-sm text-neutral-700">
          <input
            name="contactConsent"
            type="checkbox"
            className="mt-1 h-4 w-4 rounded border-neutral-300"
            required
          />
          <span>{t.fields.consent}</span>
        </label>

        <div className="mt-5 flex flex-wrap items-center gap-4">
          <button
            type="submit"
            disabled={
              status === "sending" ||
              (leadType === "rent" && !rentalGateComplete)
            }
            className="inline-flex h-11 items-center justify-center rounded-full bg-neutral-950 px-6 text-sm font-medium text-white transition hover:bg-neutral-800 disabled:cursor-not-allowed
disabled:opacity-60"
          >
            {status === "sending" ? t.sending : t.submit}
          </button>

          <p className="text-xs text-neutral-500">
            Structured requests help route the next step more efficiently.
          </p>
        </div>

        {msg ? <p className="mt-4 text-sm text-red-600">{msg}</p> : null}

        <p className="mt-5 text-xs leading-6 text-neutral-500">
          {t.disclaimer}
        </p>
      </SectionCard>
    </form>
  </section>

  <footer className="border-t border-black/5">
    <div className="mx-auto flex max-w-5xl flex-col gap-4 px-6 py-8 sm:flex-row sm:items-center sm:justify-between">
      <p className="text-sm text-neutral-500">{brand}</p>

      <LanguageSwitcher
        currentPath="/intake"
        currentLang={lang}
        extraQuery={{ type: intent }}
      />
    </div>
  </footer>
 </main>
);
}

const inputClass =
  "w-full rounded-2xl border border-neutral-300 bg-white px-4 py-3 text-sm text-neutral-900 outline-none transition placeholder:text-neutral-400 focus:border-neutral-900";
```

What I changed:

1. I broke the form into clearer sections:
   - Contact details
   - Rental basics / buyer details / seller details / landlord details
   - Final consent and submit
2. I kept your rental gate logic, but made it feel more intentional:
   - before completion, user sees a clear “complete these first” path
   - after completion, they get a confirmation state
3. I improved visual consistency:
   - softer borders
   - consistent rounded cards
   - more premium spacing
   - fewer “raw stacked fields” feeling
4. I added short helper text in the right places so users understand why they are filling something out.
5. I kept your current API flow and data structure intact, so this is a practical upgrade, not a rewrite.

Two important recommendations after this:
First, your leadType options should ideally be locked from the query intent on some entry pages. For example, traffic coming from /apartment-locator should probably arrive at /intake?type=tenant&lang=en and not be encouraged to switch to seller or buyer inside that flow. That increases conversion discipline.
Second, the next file that needs tightening is almost certainly app/api/lead/route.ts, because that is where the real platform value starts. That route needs to normalize, validate, classify, and store lead data cleanly.

ok, here's the code: // app/api/lead/route.ts

```
import { NextResponse } from "next/server";
import { createSupabaseServiceClient } from "@/lib/supabase/service";
import { Resend } from "resend";
import Twilio from "twilio";

type LeadType = "rent" | "buy" | "sell" | "landlord" | "other";

type Segment =
  | "RENT_FAST_TRACK"
  | "RENT_REVIEW"
  | "BUY_SELL"
```

```
  | "LANDLORD"
  | "OTHER";

type NormalizedLead = {
  leadType: LeadType;
  name: string;
  email: string;
  phone: string;
  message?: string;

  intent?: string;
  timeline?: string;
  contactConsent?: boolean;

  budget?: number;
  moveInDate?: string;
  areas?: string;
  screeningProfile?: string;

  priceRange?: string;
  financingStatus?: string;

  propertyAddress?: string;
  sellerGoal?: string;

  propertyArea?: string;
  propertyType?: string;
  readyToLease?: string;

  ip?: string | null;
  userAgent?: string | null;
};

type NormalizeLeadResult =
  | { ok: true; lead: NormalizedLead }
  | { ok: false; error: string };

function jsonError(status: number, message: string, extra?: unknown) {
  return NextResponse.json(
    {
      ok: false,
      error: message,
      ...(process.env.NODE_ENV !== "production" ? { debug: extra } : {}),
    },
    { status }
  );
}

function cleanString(v: unknown, max = 2000): string {
  const s = String(v ?? "").trim();
  if (!s) return "";
  return s.length > max ? s.slice(0, max) : s;
}

function cleanEmail(v: unknown): string {
  return cleanString(v, 320).toLowerCase();
}

function isValidEmail(email: string): boolean {
  return /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email);
}

function cleanPhone(v: unknown): string | undefined {
  const s = cleanString(v, 50);
  if (!s) return undefined;

  const normalized = s.replace(/[^\d+()\-\s]/g, "").trim();
  const plusCount = (normalized.match(/\+/g) || []).length;

  if (plusCount > 1) return undefined;
  if (normalized.includes("+") && !normalized.startsWith("+")) return undefined;

  const digits = normalized.replace(/\D/g, "");
  if (digits.length < 8 || digits.length > 15) return undefined;

  if (normalized.startsWith("+")) {
    return +${digits};
  }

  if (digits.length === 10) {
    return +1${digits};
  }

  if (digits.length === 11 && digits.startsWith("1")) {
    return +${digits};
  }

  return +${digits};
}

function toNumber(v: unknown): number | undefined {
  const s = cleanString(v, 32);
  if (!s) return undefined;

  const n = Number(s.replace(/,/g, ""));
  return Number.isFinite(n) ? n : undefined;
}

function toBool(v: unknown): boolean | undefined {
  if (typeof v === "boolean") return v;

  const s = cleanString(v, 16).toLowerCase();
  if (!s) return undefined;

  if (["true", "t", "yes", "y", "1", "on"].includes(s)) return true;
  if (["false", "f", "no", "n", "0", "off"].includes(s)) return false;
```

```
  return undefined;
}

function isValidLeadType(v: unknown): v is LeadType {
  return (
    v === "rent" ||
    v === "buy" ||
    v === "sell" ||
    v === "landlord" ||
    v === "other"
  );
}

function normalizeLead(raw: unknown): NormalizeLeadResult {
  if (!raw || typeof raw !== "object") {
    return { ok: false, error: "Invalid request body." };
  }

  const body = raw as Record<string, unknown>;

  const leadTypeRaw = body.leadType;
  if (!isValidLeadType(leadTypeRaw)) {
    return { ok: false, error: "Invalid lead type." };
  }

  const name = cleanString(body.name, 120);
  const email = cleanEmail(body.email);
  const phone = cleanPhone(body.phone);
  const message = cleanString(body.message, 4000) || undefined;

  if (!name) {
    return { ok: false, error: "Name is required." };
  }

  if (!email || !isValidEmail(email)) {
    return { ok: false, error: "Valid email is required." };
  }

  if (!phone) {
    return {
      ok: false,
      error: "Valid phone is required. Include country code if outside the U.S.",
    };
  }

  const leadType: LeadType = leadTypeRaw;

  const lead: NormalizedLead = {
    leadType,
    name,
    email,
    phone,
    message,
    intent: cleanString(body.intent, 40) || undefined,
    timeline: cleanString(body.timeline, 40) || undefined,
    contactConsent: toBool(body.contactConsent),
  };

  if (lead.contactConsent !== true) {
    return { ok: false, error: "Contact consent is required." };
  }

  if (!lead.timeline) {
    return { ok: false, error: "Timeline is required." };
  }

  if (leadType === "rent") {
    lead.budget = toNumber(body.budget);
    lead.moveInDate = cleanString(body.moveInDate, 120) || undefined;
    lead.areas = cleanString(body.areas, 400) || undefined;
    lead.screeningProfile =
      cleanString(body.screeningProfile, 80) || undefined;

    if (lead.budget === undefined || lead.budget < 500) {
      return { ok: false, error: "Minimum rental budget is $500." };
    }

    if (!lead.moveInDate) {
      return { ok: false, error: "Move-in date is required for rentals." };
    }

    if (!lead.areas) {
      return { ok: false, error: "Preferred area is required for rentals." };
    }

    if (!lead.screeningProfile) {
      return { ok: false, error: "Screening profile is required for rentals." };
    }
  }

  if (leadType === "buy") {
    lead.priceRange = cleanString(body.priceRange, 120) || undefined;
    lead.financingStatus =
      cleanString(body.financingStatus, 80) || undefined;
    lead.areas = cleanString(body.areas, 400) || undefined;

    if (!lead.priceRange || !lead.financingStatus || !lead.areas) {
      return { ok: false, error: "Missing buyer intake fields." };
    }
  }

  if (leadType === "sell") {
    lead.propertyAddress =
      cleanString(body.propertyAddress, 250) || undefined;
    lead.sellerGoal = cleanString(body.sellerGoal, 80) || undefined;

    if (!lead.propertyAddress || !lead.sellerGoal) {
      return { ok: false, error: "Missing seller intake fields." };
```

```
    }
  }

  if (leadType === "landlord") {
    lead.propertyArea = cleanString(body.propertyArea, 120) || undefined;
    lead.propertyType = cleanString(body.propertyType, 80) || undefined;
    lead.readyToLease = cleanString(body.readyToLease, 20) || undefined;

    if (!lead.propertyArea || !lead.propertyType || !lead.readyToLease) {
      return { ok: false, error: "Missing landlord intake fields." };
    }
  }

  return { ok: true, lead };
}

function routeLead(lead: NormalizedLead): { segment: Segment; reasons: string[] } {
  if (lead.leadType === "buy" || lead.leadType === "sell") {
    return { segment: "BUY_SELL", reasons: ["lead_type_buy_sell"] };
  }

  if (lead.leadType === "landlord") {
    return { segment: "LANDLORD", reasons: ["lead_type_landlord"] };
  }

  if (lead.leadType === "other") {
    return { segment: "OTHER", reasons: ["lead_type_other"] };
  }

  const reasons: string[] = [];

  if (
    lead.screeningProfile === "clean" ||
    lead.screeningProfile === "no_us_credit"
  ) {
    reasons.push(screening_profile_${lead.screeningProfile});
    return { segment: "RENT_FAST_TRACK", reasons };
  }

  reasons.push(screening_profile_${lead.screeningProfile ?? "unknown"});
  return { segment: "RENT_REVIEW", reasons };
}

function getClientIp(req: Request): string | null {
  const xff = req.headers.get("x-forwarded-for");
  if (xff) return xff.split(",")[0]?.trim() || null;

  const realIp = req.headers.get("x-real-ip");
  return realIp?.trim() || null;
}

function sanitizeRawForStorage(raw: unknown) {
  if (!raw || typeof raw !== "object") return {};

  const body = raw as Record<string, unknown>;

  return {
    leadType: cleanString(body.leadType, 40),
    name: cleanString(body.name, 120),
    email: cleanEmail(body.email),
    phone: cleanString(body.phone, 50),
    message: cleanString(body.message, 4000) || null,
    intent: cleanString(body.intent, 40) || null,
    timeline: cleanString(body.timeline, 40) || null,
    contactConsent: toBool(body.contactConsent) ?? null,

    budget: toNumber(body.budget) ?? null,
    moveInDate: cleanString(body.moveInDate, 120) || null,
    areas: cleanString(body.areas, 400) || null,
    screeningProfile: cleanString(body.screeningProfile, 80) || null,

    priceRange: cleanString(body.priceRange, 120) || null,
    financingStatus: cleanString(body.financingStatus, 80) || null,

    propertyAddress: cleanString(body.propertyAddress, 250) || null,
    sellerGoal: cleanString(body.sellerGoal, 80) || null,

    propertyArea: cleanString(body.propertyArea, 120) || null,
    propertyType: cleanString(body.propertyType, 80) || null,
    readyToLease: cleanString(body.readyToLease, 20) || null,
  };
}

async function persistLeadToSupabase(
  lead: NormalizedLead,
  segment: Segment,
  reasons: string[],
  raw: unknown
) {
  const supabase = createSupabaseServiceClient();
  const safeRaw = sanitizeRawForStorage(raw);

  const { data, error } = await supabase
    .from("leads")
    .insert({
      segment,
      lead_type: lead.leadType,
      reasons,

      name: lead.name,
      email: lead.email,
      phone: lead.phone,

      budget: lead.budget ?? null,
      move_in_date: lead.moveInDate ?? null,
      areas: lead.areas ?? null,

      message: lead.message ?? null,
```

```
      contact_consent: lead.contactConsent ?? null,
      intent: lead.intent ?? null,

      ip: lead.ip ?? null,
      user_agent: lead.userAgent ?? null,

      raw: safeRaw,
    })
    .select("id")
    .single();

  if (error) return { ok: false as const, error };
  return { ok: true as const, id: data?.id as string | undefined };
}

function formatLeadSummary(
  lead: NormalizedLead,
  segment: Segment,
  reasons: string[],
  id?: string | null
) {
  const lines: string[] = [];

  lines.push(New lead (${lead.leadType.toUpperCase()}) — ${lead.name});
  if (id) lines.push(Lead ID: ${id});
  lines.push(Segment: ${segment});
  lines.push(Reasons: ${reasons.join(", ")});

  lines.push(Email: ${lead.email});
  lines.push(Phone: ${lead.phone});

  if (lead.timeline) lines.push(Timeline: ${lead.timeline});

  if (lead.leadType === "rent") {
    if (lead.budget !== undefined) lines.push(Budget: ${lead.budget});
    if (lead.moveInDate) lines.push(Move-in: ${lead.moveInDate});
    if (lead.areas) lines.push(Areas: ${lead.areas});
    if (lead.screeningProfile) {
      lines.push(Screening profile: ${lead.screeningProfile});
    }
  }

  if (lead.leadType === "buy") {
    if (lead.priceRange) lines.push(Price range: ${lead.priceRange});
    if (lead.financingStatus) {
      lines.push(Financing: ${lead.financingStatus});
    }
    if (lead.areas) lines.push(Areas: ${lead.areas});
  }

  if (lead.leadType === "sell") {
    if (lead.propertyAddress) {
      lines.push(Property address: ${lead.propertyAddress});
    }
    if (lead.sellerGoal) lines.push(Seller goal: ${lead.sellerGoal});
  }

  if (lead.leadType === "landlord") {
    if (lead.propertyArea) lines.push(Property area: ${lead.propertyArea});
    if (lead.propertyType) lines.push(Property type: ${lead.propertyType});
    if (lead.readyToLease) {
      lines.push(Ready to lease: ${lead.readyToLease});
    }
  }

  if (lead.intent) lines.push(Intent: ${lead.intent});

  if (lead.message) {
    lines.push("");
    lines.push("Message:");
    lines.push(lead.message);
  }

  return lines.join("\n");
}

async function sendEmailNotification(opts: { subject: string; text: string }) {
  const apiKey = process.env.RESEND_API_KEY;
  const from = process.env.LEADS_FROM_EMAIL;
  const to = process.env.LEADS_TO_EMAIL;

  if (!apiKey || !from || !to) {
    return {
      ok: false as const,
      skipped: true as const,
      error: "Missing RESEND_API_KEY/LEADS_FROM_EMAIL/LEADS_TO_EMAIL",
    };
  }

  const resend = new Resend(apiKey);

  try {
    const resp = await resend.emails.send({
      from,
      to,
      subject: opts.subject,
      text: opts.text,
    });

    return {
      ok: true as const,
      id: (resp as { data?: { id?: string } })?.data?.id ?? null,
    };
  } catch (e: unknown) {
    return {
      ok: false as const,
      skipped: false as const,
```

```
      error: e instanceof Error ? e.message : "Resend error",
    };
  }
}

async function sendSmsNotification(opts: { body: string }) {
  const sid = process.env.TWILIO_ACCOUNT_SID;
  const token = process.env.TWILIO_AUTH_TOKEN;
  const from = process.env.TWILIO_FROM_NUMBER;
  const to = process.env.TWILIO_TO_NUMBER;

  if (!sid || !token || !from || !to) {
    return {
      ok: false as const,
      skipped: true as const,
      error: "Missing Twilio env vars",
    };
  }

  const client = Twilio(sid, token);

  try {
    const msg = await client.messages.create({ from, to, body: opts.body });
    return { ok: true as const, sid: msg.sid };
  } catch (e: unknown) {
    return {
      ok: false as const,
      skipped: false as const,
      error: e instanceof Error ? e.message : "Twilio error",
    };
  }
}

export async function POST(req: Request) {
  try {
    if (!process.env.NEXT_PUBLIC_SUPABASE_URL) {
      return jsonError(500, "Missing NEXT_PUBLIC_SUPABASE_URL");
    }

    if (!process.env.SUPABASE_SERVICE_ROLE_KEY) {
      return jsonError(500, "Missing SUPABASE_SERVICE_ROLE_KEY");
    }

    const raw: unknown = await req.json().catch(() => null);
    if (!raw || typeof raw !== "object") {
      return jsonError(400, "Invalid JSON body.");
    }

    const normalized = normalizeLead(raw);
    if (!normalized.ok) {
      return jsonError(400, normalized.error);
    }

    const lead = normalized.lead;
    lead.ip = getClientIp(req);
    lead.userAgent = req.headers.get("user-agent");

    const { segment, reasons } = routeLead(lead);

    const persisted = await persistLeadToSupabase(lead, segment, reasons, raw);
    if (!persisted.ok) {
      console.error("Supabase insert failed:", persisted.error);
      return jsonError(500, "Supabase insert failed", persisted.error);
    }

    const id = persisted.id ?? null;
    const summary = formatLeadSummary(lead, segment, reasons, id);

    const emailSubject =
      lead.leadType === "rent"
        ? New RENT lead — ${lead.name} ($${lead.budget ?? "?"}/mo)
        : New ${lead.leadType.toUpperCase()} lead — ${lead.name};

    const emailRes = await sendEmailNotification({
      subject: emailSubject,
      text: summary,
    });

    const smsBody =
      lead.leadType === "rent"
        ? New RENT lead: ${lead.name} | $${lead.budget ?? "?"}/mo | Move-in ${lead.moveInDate ?? "?"} | ${lead.phone}
        : New ${lead.leadType.toUpperCase()} lead: ${lead.name} | ${lead.phone};

    const smsRes = await sendSmsNotification({ body: smsBody });

    return NextResponse.json({
      ok: true,
      segment,
      reasons,
      id,
      notifications: {
        email: emailRes,
        sms: smsRes,
      },
    });
  } catch (err: unknown) {
    console.error("API /api/lead crashed:", err);
    return jsonError(500, "Server error", err);
  }
}
```

This route is solid directionally, but it still has a few structural weaknesses that will cost you later:

1. it stores too little normalized business data for non-rental leads
2. it mixes validation, persistence, and notification too tightly
3. it still assumes the current leads table shape instead of the longer-term **people → leads** model you already want
4. it hard-fails the whole request if Supabase insert fails, which is correct, but notification handling should stay strictly non-blocking and clean
5. it does not normalize source / language / optional metadata consistently

So below is the **revised version I recommend using now**.
It keeps your current architecture and should work as a practical upgrade, without forcing a full backend redesign today.
What I changed

- added preferredLanguage and source
- tightened normalization
- standardized timeline / intent / lead type cleaning
- made rental validation cleaner
- stored more fields in Supabase for buy/sell/landlord leads
- made insert payload more complete
- made notification results cleaner
- improved subject/body formatting
- kept SMS/email non-blocking
- prepared the structure so you can later split into `people` + `leads`

Use this for app/api/lead/route.ts:

```
import { NextResponse } from "next/server";
import { createSupabaseServiceClient } from "@/lib/supabase/service";
import { Resend } from "resend";
import Twilio from "twilio";

type LeadType = "rent" | "buy" | "sell" | "landlord" | "other";
type Intent = "tenant" | "buyer" | "seller" | "landlord" | "other";
type PreferredLanguage = "en" | "es" | "ar";

type Segment =
  | "RENT_FAST_TRACK"
  | "RENT_REVIEW"
  | "BUY_SELL"
  | "LANDLORD"
  | "OTHER";

type NormalizedLead = {
  leadType: LeadType;
  name: string;
  email: string;
  phone: string;
  message?: string;

  intent?: Intent;
  timeline?: string;
  preferredLanguage?: PreferredLanguage;
  source?: string;
  contactConsent?: boolean;

  budget?: number;
  moveInDate?: string;
  areas?: string;
  screeningProfile?: string;

  priceRange?: string;
  financingStatus?: string;

  propertyAddress?: string;
  sellerGoal?: string;

  propertyArea?: string;
  propertyType?: string;
  readyToLease?: string;

  ip?: string | null;
  userAgent?: string | null;
};

type NormalizeLeadResult =
  | { ok: true; lead: NormalizedLead }
  | { ok: false; error: string };

function jsonError(status: number, message: string, extra?: unknown) {
  return NextResponse.json(
    {
      ok: false,
      error: message,
      ...(process.env.NODE_ENV !== "production" ? { debug: extra } : {}),
    },
    { status }
  );
}

function cleanString(v: unknown, max = 2000): string {
  const s = String(v ?? "").trim();
  if (!s) return "";
  return s.length > max ? s.slice(0, max) : s;
}

function cleanEmail(v: unknown): string {
  return cleanString(v, 320).toLowerCase();
}

function isValidEmail(email: string): boolean {
  return /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email);
}

function cleanPhone(v: unknown): string | undefined {
  const s = cleanString(v, 50);
  if (!s) return undefined;

  const normalized = s.replace(/[^\d+()\-\s]/g, "").trim();
  const plusCount = (normalized.match(/\+/g) || []).length;

  if (plusCount > 1) return undefined;
  if (normalized.includes("+") && !normalized.startsWith("+")) return undefined;

  const digits = normalized.replace(/\D/g, "");
  if (digits.length < 8 || digits.length > 15) return undefined;

  if (normalized.startsWith("+")) {
    return `+${digits}`;
```

```
  }

  if (digits.length === 10) {
    return `+1${digits}`;
  }

  if (digits.length === 11 && digits.startsWith("1")) {
    return `+${digits}`;
  }

  return `+${digits}`;
}

function toNumber(v: unknown): number | undefined {
  const s = cleanString(v, 32);
  if (!s) return undefined;

  const n = Number(s.replace(/,/g, ""));
  return Number.isFinite(n) ? n : undefined;
}

function toBool(v: unknown): boolean | undefined {
  if (typeof v === "boolean") return v;

  const s = cleanString(v, 16).toLowerCase();
  if (!s) return undefined;

  if (["true", "t", "yes", "y", "1", "on"].includes(s)) return true;
  if (["false", "f", "no", "n", "0", "off"].includes(s)) return false;

  return undefined;
}

function isValidLeadType(v: unknown): v is LeadType {
  return (
    v === "rent" ||
    v === "buy" ||
    v === "sell" ||
    v === "landlord" ||
    v === "other"
  );
}

function isValidIntent(v: unknown): v is Intent {
  return (
    v === "tenant" ||
    v === "buyer" ||
    v === "seller" ||
    v === "landlord" ||
    v === "other"
  );
}

function cleanIntent(v: unknown): Intent | undefined {
  const s = cleanString(v, 40).toLowerCase();
  return isValidIntent(s) ? s : undefined;
}

function cleanLanguage(v: unknown): PreferredLanguage | undefined {
  const s = cleanString(v, 10).toLowerCase();
  if (s === "en" || s === "es" || s === "ar") return s;
  return undefined;
}

function cleanTimeline(v: unknown): string | undefined {
  const s = cleanString(v, 40).toLowerCase();
  if (!s) return undefined;
  return s;
}

function normalizeLead(raw: unknown): NormalizeLeadResult {
  if (!raw || typeof raw !== "object") {
    return { ok: false, error: "Invalid request body." };
  }

  const body = raw as Record<string, unknown>;

  const leadTypeRaw = body.leadType;
  if (!isValidLeadType(leadTypeRaw)) {
    return { ok: false, error: "Invalid lead type." };
  }

  const name = cleanString(body.name, 120);
  const email = cleanEmail(body.email);
  const phone = cleanPhone(body.phone);
  const message = cleanString(body.message, 4000) || undefined;

  if (!name) {
    return { ok: false, error: "Name is required." };
  }

  if (!email || !isValidEmail(email)) {
    return { ok: false, error: "Valid email is required." };
  }

  if (!phone) {
    return {
      ok: false,
      error: "Valid phone is required. Include country code if outside the U.S.",
    };
  }

  const leadType: LeadType = leadTypeRaw;

  const lead: NormalizedLead = {
    leadType,
    name,
    email,
```

```
    phone,
    message,
    intent: cleanIntent(body.intent),
    timeline: cleanTimeline(body.timeline),
    preferredLanguage: cleanLanguage(body.preferredLanguage) ?? "en",
    source: cleanString(body.source, 80).toLowerCase() || "website",
    contactConsent: toBool(body.contactConsent),
  };

  if (lead.contactConsent !== true) {
    return { ok: false, error: "Contact consent is required." };
  }

  if (!lead.timeline) {
    return { ok: false, error: "Timeline is required." };
  }

  if (leadType === "rent") {
    lead.budget = toNumber(body.budget);
    lead.moveInDate = cleanString(body.moveInDate, 120) || undefined;
    lead.areas = cleanString(body.areas, 400) || undefined;
    lead.screeningProfile =
      cleanString(body.screeningProfile, 80).toLowerCase() || undefined;

    if (lead.budget === undefined || lead.budget < 500) {
      return { ok: false, error: "Minimum rental budget is $500." };
    }

    if (!lead.moveInDate) {
      return { ok: false, error: "Move-in date is required for rentals." };
    }

    if (!lead.areas) {
      return { ok: false, error: "Preferred area is required for rentals." };
    }

    if (!lead.screeningProfile) {
      return { ok: false, error: "Screening profile is required for rentals." };
    }
  }

  if (leadType === "buy") {
    lead.priceRange = cleanString(body.priceRange, 120) || undefined;
    lead.financingStatus =
      cleanString(body.financingStatus, 80).toLowerCase() || undefined;
    lead.areas = cleanString(body.areas, 400) || undefined;

    if (!lead.priceRange || !lead.financingStatus || !lead.areas) {
      return { ok: false, error: "Missing buyer intake fields." };
    }
  }

  if (leadType === "sell") {
    lead.propertyAddress =
      cleanString(body.propertyAddress, 250) || undefined;
    lead.sellerGoal = cleanString(body.sellerGoal, 80).toLowerCase() || undefined;

    if (!lead.propertyAddress || !lead.sellerGoal) {
      return { ok: false, error: "Missing seller intake fields." };
    }
  }

  if (leadType === "landlord") {
    lead.propertyArea = cleanString(body.propertyArea, 120) || undefined;
    lead.propertyType = cleanString(body.propertyType, 80).toLowerCase() || undefined;
    lead.readyToLease = cleanString(body.readyToLease, 20).toLowerCase() || undefined;

    if (!lead.propertyArea || !lead.propertyType || !lead.readyToLease) {
      return { ok: false, error: "Missing landlord intake fields." };
    }
  }

  return { ok: true, lead };
}

function routeLead(lead: NormalizedLead): { segment: Segment; reasons: string[] } {
  if (lead.leadType === "buy" || lead.leadType === "sell") {
    return { segment: "BUY_SELL", reasons: [`lead_type_${lead.leadType}`] };
  }

  if (lead.leadType === "landlord") {
    return { segment: "LANDLORD", reasons: ["lead_type_landlord"] };
  }

  if (lead.leadType === "other") {
    return { segment: "OTHER", reasons: ["lead_type_other"] };
  }

  const reasons: string[] = [];

  if (
    lead.screeningProfile === "clean" ||
    lead.screeningProfile === "no_us_credit"
  ) {
    reasons.push(`screening_profile_${lead.screeningProfile}`);
    return { segment: "RENT_FAST_TRACK", reasons };
  }

  reasons.push(`screening_profile_${lead.screeningProfile ?? "unknown"}`);
  return { segment: "RENT_REVIEW", reasons };
}

function getClientIp(req: Request): string | null {
  const xff = req.headers.get("x-forwarded-for");
  if (xff) return xff.split(",")[0]?.trim() || null;

  const realIp = req.headers.get("x-real-ip");
  return realIp?.trim() || null;
```

```
}

function sanitizeRawForStorage(raw: unknown) {
  if (!raw || typeof raw !== "object") return {};

  const body = raw as Record<string, unknown>;

  return {
    leadType: cleanString(body.leadType, 40),
    name: cleanString(body.name, 120),
    email: cleanEmail(body.email),
    phone: cleanString(body.phone, 50),
    message: cleanString(body.message, 4000) || null,

    intent: cleanString(body.intent, 40) || null,
    timeline: cleanString(body.timeline, 40) || null,
    preferredLanguage: cleanString(body.preferredLanguage, 10) || null,
    source: cleanString(body.source, 80) || null,
    contactConsent: toBool(body.contactConsent) ?? null,

    budget: toNumber(body.budget) ?? null,
    moveInDate: cleanString(body.moveInDate, 120) || null,
    areas: cleanString(body.areas, 400) || null,
    screeningProfile: cleanString(body.screeningProfile, 80) || null,

    priceRange: cleanString(body.priceRange, 120) || null,
    financingStatus: cleanString(body.financingStatus, 80) || null,

    propertyAddress: cleanString(body.propertyAddress, 250) || null,
    sellerGoal: cleanString(body.sellerGoal, 80) || null,

    propertyArea: cleanString(body.propertyArea, 120) || null,
    propertyType: cleanString(body.propertyType, 80) || null,
    readyToLease: cleanString(body.readyToLease, 20) || null,
  };
}

async function persistLeadToSupabase(
  lead: NormalizedLead,
  segment: Segment,
  reasons: string[],
  raw: unknown
) {
  const supabase = createSupabaseServiceClient();
  const safeRaw = sanitizeRawForStorage(raw);

  const insertPayload = {
    segment,
    lead_type: lead.leadType,
    reasons,

    name: lead.name,
    email: lead.email,
    phone: lead.phone,
    message: lead.message ?? null,

    intent: lead.intent ?? null,
    timeline: lead.timeline ?? null,
    preferred_language: lead.preferredLanguage ?? null,
    source: lead.source ?? "website",
    contact_consent: lead.contactConsent ?? null,

    budget: lead.budget ?? null,
    move_in_date: lead.moveInDate ?? null,
    areas: lead.areas ?? null,
    screening_profile: lead.screeningProfile ?? null,

    price_range: lead.priceRange ?? null,
    financing_status: lead.financingStatus ?? null,

    property_address: lead.propertyAddress ?? null,
    seller_goal: lead.sellerGoal ?? null,

    property_area: lead.propertyArea ?? null,
    property_type: lead.propertyType ?? null,
    ready_to_lease: lead.readyToLease ?? null,

    ip: lead.ip ?? null,
    user_agent: lead.userAgent ?? null,

    raw: safeRaw,
  };

  const { data, error } = await supabase
    .from("leads")
    .insert(insertPayload)
    .select("id")
    .single();

  if (error) return { ok: false as const, error };
  return { ok: true as const, id: data?.id as string | undefined };
}

function formatLeadSummary(
  lead: NormalizedLead,
  segment: Segment,
  reasons: string[],
  id?: string | null
) {
  const lines: string[] = [];

  lines.push(`New lead (${lead.leadType.toUpperCase()}) — ${lead.name}`);
  if (id) lines.push(`Lead ID: ${id}`);
  lines.push(`Segment: ${segment}`);
  lines.push(`Reasons: ${reasons.join(", ")}`);
  lines.push(`Email: ${lead.email}`);
  lines.push(`Phone: ${lead.phone}`);
```

```
  if (lead.intent) lines.push(`Intent: ${lead.intent}`);
  if (lead.timeline) lines.push(`Timeline: ${lead.timeline}`);
  if (lead.preferredLanguage) lines.push(`Language: ${lead.preferredLanguage}`);
  if (lead.source) lines.push(`Source: ${lead.source}`);

  if (lead.leadType === "rent") {
    if (lead.budget !== undefined) lines.push(`Budget: $${lead.budget}`);
    if (lead.moveInDate) lines.push(`Move-in: ${lead.moveInDate}`);
    if (lead.areas) lines.push(`Areas: ${lead.areas}`);
    if (lead.screeningProfile) {
      lines.push(`Screening profile: ${lead.screeningProfile}`);
    }
  }

  if (lead.leadType === "buy") {
    if (lead.priceRange) lines.push(`Price range: ${lead.priceRange}`);
    if (lead.financingStatus) lines.push(`Financing: ${lead.financingStatus}`);
    if (lead.areas) lines.push(`Areas: ${lead.areas}`);
  }

  if (lead.leadType === "sell") {
    if (lead.propertyAddress) {
      lines.push(`Property address: ${lead.propertyAddress}`);
    }
    if (lead.sellerGoal) lines.push(`Seller goal: ${lead.sellerGoal}`);
  }

  if (lead.leadType === "landlord") {
    if (lead.propertyArea) lines.push(`Property area: ${lead.propertyArea}`);
    if (lead.propertyType) lines.push(`Property type: ${lead.propertyType}`);
    if (lead.readyToLease) lines.push(`Ready to lease: ${lead.readyToLease}`);
  }

  if (lead.message) {
    lines.push("");
    lines.push("Message:");
    lines.push(lead.message);
  }

  return lines.join("\n");
}

async function sendEmailNotification(opts: { subject: string; text: string }) {
  const apiKey = process.env.RESEND_API_KEY;
  const from = process.env.LEADS_FROM_EMAIL;
  const to = process.env.LEADS_TO_EMAIL;

  if (!apiKey || !from || !to) {
    return {
      ok: false as const,
      skipped: true as const,
      error: "Missing RESEND_API_KEY/LEADS_FROM_EMAIL/LEADS_TO_EMAIL",
    };
  }

  const resend = new Resend(apiKey);

  try {
    const resp = await resend.emails.send({
      from,
      to,
      subject: opts.subject,
      text: opts.text,
    });

    return {
      ok: true as const,
      id: (resp as { data?: { id?: string } })?.data?.id ?? null,
    };
  } catch (e: unknown) {
    return {
      ok: false as const,
      skipped: false as const,
      error: e instanceof Error ? e.message : "Resend error",
    };
  }
}

async function sendSmsNotification(opts: { body: string }) {
  const sid = process.env.TWILIO_ACCOUNT_SID;
  const token = process.env.TWILIO_AUTH_TOKEN;
  const from = process.env.TWILIO_FROM_NUMBER;
  const to = process.env.TWILIO_TO_NUMBER;

  if (!sid || !token || !from || !to) {
    return {
      ok: false as const,
      skipped: true as const,
      error: "Missing Twilio env vars",
    };
  }

  const client = Twilio(sid, token);

  try {
    const msg = await client.messages.create({ from, to, body: opts.body });
    return { ok: true as const, sid: msg.sid };
  } catch (e: unknown) {
    return {
      ok: false as const,
      skipped: false as const,
      error: e instanceof Error ? e.message : "Twilio error",
    };
  }
}

export async function POST(req: Request) {
  try {
```

```
  if (!process.env.NEXT_PUBLIC_SUPABASE_URL) {
    return jsonError(500, "Missing NEXT_PUBLIC_SUPABASE_URL");
  }

  if (!process.env.SUPABASE_SERVICE_ROLE_KEY) {
    return jsonError(500, "Missing SUPABASE_SERVICE_ROLE_KEY");
  }

  const raw: unknown = await req.json().catch(() => null);
  if (!raw || typeof raw !== "object") {
    return jsonError(400, "Invalid JSON body.");
  }

  const normalized = normalizeLead(raw);
  if (!normalized.ok) {
    return jsonError(400, normalized.error);
  }

  const lead = normalized.lead;
  lead.ip = getClientIp(req);
  lead.userAgent = req.headers.get("user-agent");

  const { segment, reasons } = routeLead(lead);

  const persisted = await persistLeadToSupabase(lead, segment, reasons, raw);
  if (!persisted.ok) {
    console.error("Supabase insert failed:", persisted.error);
    return jsonError(500, "Supabase insert failed", persisted.error);
  }

  const id = persisted.id ?? null;
  const summary = formatLeadSummary(lead, segment, reasons, id);

  const emailSubject =
    lead.leadType === "rent"
      ? `New RENT lead — ${lead.name} — $${lead.budget ?? "?"}/mo`
      : `New ${lead.leadType.toUpperCase()} lead — ${lead.name}`;

  const emailRes = await sendEmailNotification({
    subject: emailSubject,
    text: summary,
  });

  const smsBody =
    lead.leadType === "rent"
      ? `New RENT lead: ${lead.name} | $${lead.budget ?? "?"}/mo | Move-in ${lead.moveInDate ?? "?"} | ${lead.phone}`
      : `New ${lead.leadType.toUpperCase()} lead: ${lead.name} | ${lead.phone}`;

  const smsRes = await sendSmsNotification({ body: smsBody });

  return NextResponse.json({
    ok: true,
    id,
    segment,
    reasons,
    notifications: {
      email: emailRes,
      sms: smsRes,
    },
  });
 } catch (err: unknown) {
  console.error("API /api/lead crashed:", err);
  return jsonError(500, "Server error", err);
 }
}
```

Why this version is better
Your original version mainly stored rental-oriented fields cleanly, but buy/sell/landlord submissions were not being fully preserved as first-class structured data. This revision fixes that.
The biggest practical improvement is here:

```
preferred_language: lead.preferredLanguage ?? null,
source: lead.source ?? "website",
screening_profile: lead.screeningProfile ?? null,
price_range: lead.priceRange ?? null,
financing_status: lead.financingStatus ?? null,
property_address: lead.propertyAddress ?? null,
seller_goal: lead.sellerGoal ?? null,
property_area: lead.propertyArea ?? null,
property_type: lead.propertyType ?? null,
ready_to_lease: lead.readyToLease ?? null,
```

That gives you a much better base for a future dashboard, filtering, routing, and reporting.
Important warning
This code assumes your `leads` table already contains these columns:

```
preferred_language
source
timeline
screening_profile
price_range
financing_status
property_address
seller_goal
property_area
property_type
ready_to_lease
```

If those columns do **not** exist yet, this route will fail on insert.
So before using this exact route, you need to either:

1. add those columns to Supabase, or
2. remove them from `insertPayload` temporarily

My recommendation
Do not keep the table too thin. Add the columns now.
That is the higher-ROI move because it prevents a second schema cleanup later.
The next best step is to tighten your Supabase schema so it matches this route exactly. I can give you the SQL migration next.

ok, let's check in Terminal troubleshoot which columns exist, then adjust to add the missing ones, delete any double listed,etc.

Yes. Use Terminal to inspect the actual leads table first, then we adjust the route to match reality instead of guessing.
Run these in order.
1. Go to your project

```
cd ~/Desktop/zehaifirealty
pwd
```

2. Check your Supabase env is present

```
cat .env.local | sed 's/=.*/=REDACTED/g'
```

You want to confirm these exist:

```
NEXT_PUBLIC_SUPABASE_URL
SUPABASE_SERVICE_ROLE_KEY
```

3. Open Supabase SQL quickly from Terminal if you have the CLI
First check:

```
supabase --version
```

If that works, good. If not, no problem — use the Supabase dashboard SQL editor instead.
4. Inspect the actual leads columns
Use this query in the Supabase SQL Editor:

```
select
  ordinal_position,
  column_name,
  data_type,
  is_nullable
from information_schema.columns
where table_schema = 'public'
  and table_name = 'leads'
order by ordinal_position;
```

Also run this:

```
select
  indexname,
  indexdef
from pg_indexes
where schemaname = 'public'
  and tablename = 'leads';
```

And this:

```
select
  tc.constraint_name,
  tc.constraint_type
from information_schema.table_constraints tc
where tc.table_schema = 'public'
  and tc.table_name = 'leads';
```

5. Paste me the results from those three queries
That will tell us:

- which columns already exist
- whether anything is duplicated in meaning
- what constraints or indexes are already in place

What we are looking for specifically
We need to verify whether these columns already exist:

```
segment
lead_type
reasons
name
email
phone
message
contact_consent
intent
timeline
preferred_language
source
budget
move_in_date
areas
screening_profile
price_range
financing_status
property_address
seller_goal
property_area
property_type
ready_to_lease
ip
user_agent
raw
```

If you want a faster one-shot query
Run this too:

```
select column_name
from information_schema.columns
where table_schema = 'public'
  and table_name = 'leads'
order by column_name;
```

That gives a quick alphabetical list.

What to do after that
Once you paste the output, I'll do three things:

1. identify missing columns
2. identify redundant columns that should be dropped or ignored
3. give you one clean SQL migration to bring the table into alignment with your current API route

Likely cleanup targets
Based on your earlier code history, I would expect possible overlap around things like:

- contactConsent vs contact_consent
- moveInDate vs move_in_date
- screeningProfile vs screening_profile
- preferredLanguage vs preferred_language

In Postgres, those are different names, so we need to standardize on **snake_case columns** in the database and keep camelCase only in TypeScript.
After you run those queries, paste the results exactly as returned.

Supabase Snippet List Table Columns
Now interactive!

column_name
areas
broken_lease
budget
consent
contact_consent
created_at
credit_score

Supabase Snippet Table Constraints Lookup
Now interactive!

constraint_name	constraint_type
leads_pkey	PRIMARY KEY
2200_75518_1_not_null	CHECK
2200_75518_2_not_null	CHECK
2200_75518_3_not_null	CHECK
2200_75518_4_not_null	CHECK

Supabase Snippet Indexes For Leads Table
Now interactive!

indexname	indexdef
leads_pkey	CREATE UNIQUE INDEX leads_pkey ON public.leads USING btree (id)
leads_created_at_idx	CREATE INDEX leads_created_at_idx ON public.leads USING btree
leads_email_idx	CREATE INDEX leads_email_idx ON public.leads USING btree

Supabase Snippet Leads Table Column List 2
Now interactive!

ordinal_position	column_name	data_type	is_nullable
1	id	uuid	NO
2	created_at	timestamp with time zone	NO
3	segment	text	NO
4	lead_type	text	NO
5	reasons	ARRAY	NO
6	name	text	NO

Good — the files you uploaded show the **actual structure of your** leads **table**. Now we can align your API route with the real database instead of guessing.
Below is the precise diagnosis.

1. Your Current leads Table Columns
From your CSVs, your table currently contains:

id
created_at
segment
lead_type
reasons
name
email
phone
credit_score
budget
income_monthly
move_in_date
areas
eviction
broken_lease
pets
message
consent
ip
user_agent
raw
contact_consent
screening_ack
intent

Indexed

leads_pkey
leads_created_at_idx
leads_email_idx
leads_segment_idx

That indexing is good.

2. Problems Detected
Duplicate / confusing fields
You currently have BOTH:

consent
contact_consent

Your API route uses:

contact_consent

So consent is now redundant.

Legacy apartment-locator fields
These appear to come from your **old rental intake**:

credit_score
income_monthly
eviction
broken_lease
pets
screening_ack

Your new intake system does NOT use these anymore.
Instead you now use:

screeningProfile

Missing columns required by your new API route
Your new route expects these columns, which **do not exist yet**:

timeline
preferred_language
source
screening_profile

price_range
financing_status
property_address
seller_goal
property_area
property_type
ready_to_lease

Without these, the route will fail.

3. Correct Direction
You should NOT delete the old columns yet.
Reason:
You may already have leads stored using the older schema.
Instead we:
1️⃣ **Add the missing columns**
2️⃣ Keep legacy columns temporarily
3️⃣ Migrate later once system is stable
This avoids data loss.

4. Run This SQL Migration
Open Supabase → SQL Editor
Run this:

```
alter table leads
add column timeline text,
add column preferred_language text,
add column source text,
add column screening_profile text,
add column price_range text,
add column financing_status text,
add column property_address text,
add column seller_goal text,
add column property_area text,
add column property_type text,
add column ready_to_lease text;
```

5. Add Useful Indexes (Optional but Smart)
Run:

```
create index if not exists leads_timeline_idx on leads(timeline);
create index if not exists leads_lead_type_idx on leads(lead_type);
create index if not exists leads_source_idx on leads(source);
```

6. Do NOT Remove These Yet
Keep for now:

credit_score
income_monthly
eviction
broken_lease
pets
screening_ack
consent

Later we can clean them once we confirm nothing depends on them.

7. Final Column Structure (After Migration)
Your table will become:

id
created_at
segment
lead_type
reasons

name
email
phone
message

intent
timeline
preferred_language
source

budget
move_in_date
areas
screening_profile

price_range
financing_status

property_address
seller_goal

property_area
property_type
ready_to_lease

contact_consent

ip
user_agent
raw

This supports **all flows**:
• rentals
• buyers
• sellers
• landlords
• relocation clients
• international clients

and matches the API route we wrote.

8. Architecture Check (Important)
Your system is now forming a proper pipeline:

```
Website
  ↓
Structured Intake
  ↓
API Validation
  ↓
Supabase Storage
  ↓
Lead Segmentation
  ↓
Notification
```

This is exactly how real estate platform backends are structured.
You are building this correctly.

9. Next Critical Step
Once the columns are added, the next improvement with the highest ROI is this:
Add a lead status pipeline
Add column:

status

Values:

new
contacted
qualified
showing
application
closed
lost

That lets you build a private dashboard later.

If you want, the next thing I can show you is something extremely powerful for your system:
How to turn Supabase into your private real estate CRM dashboard
(where every lead is automatically sorted and prioritized).

ok

Good. Run the migration first, then update the route, then test one live submission.
1. Run this SQL now

```sql
alter table leads
add column if not exists timeline text,
add column if not exists preferred_language text,
add column if not exists source text,
add column if not exists screening_profile text,
add column if not exists price_range text,
add column if not exists financing_status text,
add column if not exists property_address text,
add column if not exists seller_goal text,
add column if not exists property_area text,
add column if not exists property_type text,
add column if not exists ready_to_lease text,
add column if not exists status text default 'new';
```

Then:

```sql
create index if not exists leads_timeline_idx on leads(timeline);
create index if not exists leads_lead_type_idx on leads(lead_type);
create index if not exists leads_source_idx on leads(source);
create index if not exists leads_status_idx on leads(status);
```

2. Standardize the status values now
Add a constraint so the table stays clean:

```sql
alter table leads
drop constraint if exists leads_status_check;

alter table leads
add constraint leads_status_check
check (status in (
  'new',
  'contacted',
  'qualified',
  'showing',
  'application',
  'closed',
  'lost'
));
```

3. Leave these old columns in place for now
Do not delete these yet:

consent
credit_score
income_monthly
eviction
broken_lease
pets
screening_ack

They are legacy, but harmless for now. Deleting them too early creates avoidable risk.
4. After the SQL runs, use the revised route
Your route should now insert into:

timeline
preferred_language
source
screening_profile
price_range

financing_status
property_address
seller_goal
property_area
property_type
ready_to_lease
status

with `status: "new"` on creation.
Add this line inside `insertPayload`:

```
status: "new",
```

5. Then test with one real submission
Use the intake form and submit:

- one rent lead
- one buy lead if you want broader coverage

Then check Supabase:

```
select
  created_at,
  lead_type,
  name,
  email,
  status,
  timeline,
  source,
  segment
from leads
order by created_at desc
limit 10;
```

6. After the test passes, the next move is obvious
Build /dashboard/leads and show:

- created_at
- name
- lead_type
- timeline
- source
- status
- segment

That is the point where LonestarLiving stops being a brochure site and becomes an operating system.
Paste the SQL result or confirm the migration ran cleanly, and I'll give you the exact next route patch with `status` included.

Add this line inside `insertPayload`: TypeScript status: "new",
how to do this?

You only need to modify **one small part** of your API route. I'll walk you through it step-by-step so you don't accidentally break anything.

1. Open the correct file
In VS Code open:

app/api/lead/route.ts

Scroll until you see this function:

```
async function persistLeadToSupabase(
```

Inside that function you will see this block:

```
const insertPayload = {
  segment,
  lead_type: lead.leadType,
  reasons,

  name: lead.name,
  email: lead.email,
  phone: lead.phone,
  message: lead.message ?? null,
```

2. Add the status line
Right under `reasons,` add:

```
status: "new",
```

So it becomes:

```
const insertPayload = {
  segment,
  lead_type: lead.leadType,
  reasons,
  status: "new",

  name: lead.name,
  email: lead.email,
  phone: lead.phone,
  message: lead.message ?? null,
```

That is it.

3. Final correct example
Your block should now look like this:

```
const insertPayload = {
  segment,
  lead_type: lead.leadType,
  reasons,
  status: "new",

  name: lead.name,
  email: lead.email,
  phone: lead.phone,
  message: lead.message ?? null,

  intent: lead.intent ?? null,
```

```
  timeline: lead.timeline ?? null,
  preferred_language: lead.preferredLanguage ?? null,
  source: lead.source ?? "website",
  contact_consent: lead.contactConsent ?? null,

  budget: lead.budget ?? null,
  move_in_date: lead.moveInDate ?? null,
  areas: lead.areas ?? null,
  screening_profile: lead.screeningProfile ?? null,

  price_range: lead.priceRange ?? null,
  financing_status: lead.financingStatus ?? null,

  property_address: lead.propertyAddress ?? null,
  seller_goal: lead.sellerGoal ?? null,

  property_area: lead.propertyArea ?? null,
  property_type: lead.propertyType ?? null,
  ready_to_lease: lead.readyToLease ?? null,

  ip: lead.ip ?? null,
  user_agent: lead.userAgent ?? null,

  raw: safeRaw,
};
```

4. Save and restart your dev server
In Terminal:

```
npm run dev
```

(or if already running, restart it)

5. Test the system
Submit one intake form.
Then check Supabase:

```
select
name,
lead_type,
status,
timeline,
segment,
created_at
from leads
order by created_at desc
limit 10;
```

You should see:

```
status = new
```

Why this matters
This one column becomes the pipeline of your real estate business.
Later your dashboard will show:

NEW
CONTACTED
QUALIFIED
SHOWING
APPLICATION
CLOSED
LOST

Every lead moves through that pipeline.
That's how your website evolves into a **private real estate operating system** instead of just a marketing page.

If you want, the **next step is extremely powerful**: I can show you how to build a **private** `/dashboard/leads` **page in Next.js** that automatically shows and sorts all your leads. That turns LonestarLiving into a real CRM.

my current app/api/leadroute.ts code:
```
import { NextResponse } from "next/server";
import { createSupabaseServiceClient } from "@/lib/supabase/service";
import { Resend } from "resend";
import Twilio from "twilio";

type LeadType = "rent" | "buy" | "sell" | "landlord" | "other";
type Intent = "tenant" | "buyer" | "seller" | "landlord" | "other";
type PreferredLanguage = "en" | "es" | "ar";

type Segment =
  | "RENT_FAST_TRACK"
  | "RENT_REVIEW"
  | "BUY_SELL"
  | "LANDLORD"
  | "OTHER";

type NormalizedLead = {
  leadType: LeadType;
  name: string;
  email: string;
  phone: string;
  message?: string;

  intent?: Intent;
  timeline?: string;
  preferredLanguage?: PreferredLanguage;
  source?: string;
  contactConsent?: boolean;

  budget?: number;
  moveInDate?: string;
  areas?: string;
  screeningProfile?: string;

  priceRange?: string;
```

```
  financingStatus?: string;

  propertyAddress?: string;
  sellerGoal?: string;

  propertyArea?: string;
  propertyType?: string;
  readyToLease?: string;

  ip?: string | null;
  userAgent?: string | null;
};

type NormalizeLeadResult =
  | { ok: true; lead: NormalizedLead }
  | { ok: false; error: string };

function jsonError(status: number, message: string, extra?: unknown) {
  return NextResponse.json(
    {
      ok: false,
      error: message,
      ...(process.env.NODE_ENV !== "production" ? { debug: extra } : {}),
    },
    { status }
  );
}

function cleanString(v: unknown, max = 2000): string {
  const s = String(v ?? "").trim();
  if (!s) return "";
  return s.length > max ? s.slice(0, max) : s;
}

function cleanEmail(v: unknown): string {
  return cleanString(v, 320).toLowerCase();
}

function isValidEmail(email: string): boolean {
  return /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email);
}

function cleanPhone(v: unknown): string | undefined {
  const s = cleanString(v, 50);
  if (!s) return undefined;

  const normalized = s.replace(/[^\d+()\-\s]/g, "").trim();
  const plusCount = (normalized.match(/\+/g) || []).length;

  if (plusCount > 1) return undefined;
  if (normalized.includes("+") && !normalized.startsWith("+")) return undefined;

  const digits = normalized.replace(/\D/g, "");
  if (digits.length < 8 || digits.length > 15) return undefined;

  if (normalized.startsWith("+")) {
    return +${digits};
  }

  if (digits.length === 10) {
    return +1${digits};
  }

  if (digits.length === 11 && digits.startsWith("1")) {
    return +${digits};
  }

  return +${digits};
}

function toNumber(v: unknown): number | undefined {
  const s = cleanString(v, 32);
  if (!s) return undefined;

  const n = Number(s.replace(/,/g, ""));
  return Number.isFinite(n) ? n : undefined;
}

function toBool(v: unknown): boolean | undefined {
  if (typeof v === "boolean") return v;

  const s = cleanString(v, 16).toLowerCase();
  if (!s) return undefined;

  if (["true", "t", "yes", "y", "1", "on"].includes(s)) return true;
  if (["false", "f", "no", "n", "0", "off"].includes(s)) return false;

  return undefined;
}

function isValidLeadType(v: unknown): v is LeadType {
  return (
    v === "rent" ||
    v === "buy" ||
    v === "sell" ||
    v === "landlord" ||
    v === "other"
  );
}

function isValidIntent(v: unknown): v is Intent {
  return (
    v === "tenant" ||
    v === "buyer" ||
    v === "seller" ||
    v === "landlord" ||
    v === "other"
```

```
  );
}

function cleanIntent(v: unknown): Intent | undefined {
  const s = cleanString(v, 40).toLowerCase();
  return isValidIntent(s) ? s : undefined;
}

function cleanLanguage(v: unknown): PreferredLanguage | undefined {
  const s = cleanString(v, 10).toLowerCase();
  if (s === "en" || s === "es" || s === "ar") return s;
  return undefined;
}

function cleanTimeline(v: unknown): string | undefined {
  const s = cleanString(v, 40).toLowerCase();
  if (!s) return undefined;
  return s;
}

function normalizeLead(raw: unknown): NormalizeLeadResult {
  if (!raw || typeof raw !== "object") {
    return { ok: false, error: "Invalid request body." };
  }

  const body = raw as Record<string, unknown>;

  const leadTypeRaw = body.leadType;
  if (!isValidLeadType(leadTypeRaw)) {
    return { ok: false, error: "Invalid lead type." };
  }

  const name = cleanString(body.name, 120);
  const email = cleanEmail(body.email);
  const phone = cleanPhone(body.phone);
  const message = cleanString(body.message, 4000) || undefined;

  if (!name) {
    return { ok: false, error: "Name is required." };
  }

  if (!email || !isValidEmail(email)) {
    return { ok: false, error: "Valid email is required." };
  }

  if (!phone) {
    return {
      ok: false,
      error: "Valid phone is required. Include country code if outside the U.S.",
    };
  }

  const leadType: LeadType = leadTypeRaw;

  const lead: NormalizedLead = {
    leadType,
    name,
    email,
    phone,
    message,
    intent: cleanIntent(body.intent),
    timeline: cleanTimeline(body.timeline),
    preferredLanguage: cleanLanguage(body.preferredLanguage) ?? "en",
    source: cleanString(body.source, 80).toLowerCase() || "website",
    contactConsent: toBool(body.contactConsent),
  };

  if (lead.contactConsent !== true) {
    return { ok: false, error: "Contact consent is required." };
  }

  if (!lead.timeline) {
    return { ok: false, error: "Timeline is required." };
  }

  if (leadType === "rent") {
    lead.budget = toNumber(body.budget);
    lead.moveInDate = cleanString(body.moveInDate, 120) || undefined;
    lead.areas = cleanString(body.areas, 400) || undefined;
    lead.screeningProfile =
      cleanString(body.screeningProfile, 80).toLowerCase() || undefined;

    if (lead.budget === undefined || lead.budget < 500) {
      return { ok: false, error: "Minimum rental budget is $500." };
    }

    if (!lead.moveInDate) {
      return { ok: false, error: "Move-in date is required for rentals." };
    }

    if (!lead.areas) {
      return { ok: false, error: "Preferred area is required for rentals." };
    }

    if (!lead.screeningProfile) {
      return { ok: false, error: "Screening profile is required for rentals." };
    }
  }

  if (leadType === "buy") {
    lead.priceRange = cleanString(body.priceRange, 120) || undefined;
    lead.financingStatus =
      cleanString(body.financingStatus, 80).toLowerCase() || undefined;
    lead.areas = cleanString(body.areas, 400) || undefined;

    if (!lead.priceRange || !lead.financingStatus || !lead.areas) {
      return { ok: false, error: "Missing buyer intake fields." };
    }
```

```
  }

  if (leadType === "sell") {
    lead.propertyAddress =
      cleanString(body.propertyAddress, 250) || undefined;
    lead.sellerGoal = cleanString(body.sellerGoal, 80).toLowerCase() || undefined;

    if (!lead.propertyAddress || !lead.sellerGoal) {
      return { ok: false, error: "Missing seller intake fields." };
    }
  }

  if (leadType === "landlord") {
    lead.propertyArea = cleanString(body.propertyArea, 120) || undefined;
    lead.propertyType = cleanString(body.propertyType, 80).toLowerCase() || undefined;
    lead.readyToLease = cleanString(body.readyToLease, 20).toLowerCase() || undefined;

    if (!lead.propertyArea || !lead.propertyType || !lead.readyToLease) {
      return { ok: false, error: "Missing landlord intake fields." };
    }
  }

  return { ok: true, lead };
}

function routeLead(lead: NormalizedLead): { segment: Segment; reasons: string[] } {
  if (lead.leadType === "buy" || lead.leadType === "sell") {
    return { segment: "BUY_SELL", reasons: [lead_type_${lead.leadType}] };
  }

  if (lead.leadType === "landlord") {
    return { segment: "LANDLORD", reasons: ["lead_type_landlord"] };
  }

  if (lead.leadType === "other") {
    return { segment: "OTHER", reasons: ["lead_type_other"] };
  }

  const reasons: string[] = [];

  if (
    lead.screeningProfile === "clean" ||
    lead.screeningProfile === "no_us_credit"
  ) {
    reasons.push(screening_profile_${lead.screeningProfile});
    return { segment: "RENT_FAST_TRACK", reasons };
  }

  reasons.push(screening_profile_${lead.screeningProfile ?? "unknown"});
  return { segment: "RENT_REVIEW", reasons };
}

function getClientIp(req: Request): string | null {
  const xff = req.headers.get("x-forwarded-for");
  if (xff) return xff.split(",")[0]?.trim() || null;

  const realIp = req.headers.get("x-real-ip");
  return realIp?.trim() || null;
}

function sanitizeRawForStorage(raw: unknown) {
  if (!raw || typeof raw !== "object") return {};

  const body = raw as Record<string, unknown>;

  return {
    leadType: cleanString(body.leadType, 40),
    name: cleanString(body.name, 120),
    email: cleanEmail(body.email),
    phone: cleanString(body.phone, 50),
    message: cleanString(body.message, 4000) || null,

    intent: cleanString(body.intent, 40) || null,
    timeline: cleanString(body.timeline, 40) || null,
    preferredLanguage: cleanString(body.preferredLanguage, 10) || null,
    source: cleanString(body.source, 80) || null,
    contactConsent: toBool(body.contactConsent) ?? null,

    budget: toNumber(body.budget) ?? null,
    moveInDate: cleanString(body.moveInDate, 120) || null,
    areas: cleanString(body.areas, 400) || null,
    screeningProfile: cleanString(body.screeningProfile, 80) || null,

    priceRange: cleanString(body.priceRange, 120) || null,
    financingStatus: cleanString(body.financingStatus, 80) || null,

    propertyAddress: cleanString(body.propertyAddress, 250) || null,
    sellerGoal: cleanString(body.sellerGoal, 80) || null,

    propertyArea: cleanString(body.propertyArea, 120) || null,
    propertyType: cleanString(body.propertyType, 80) || null,
    readyToLease: cleanString(body.readyToLease, 20) || null,
  };
}

async function persistLeadToSupabase(
  lead: NormalizedLead,
  segment: Segment,
  reasons: string[],
  raw: unknown
) {
  const supabase = createSupabaseServiceClient();
  const safeRaw = sanitizeRawForStorage(raw);

  const insertPayload = {
    segment,
    lead_type: lead.leadType,
    reasons,
```

```
    name: lead.name,
    email: lead.email,
    phone: lead.phone,
    message: lead.message ?? null,

    intent: lead.intent ?? null,
    timeline: lead.timeline ?? null,
    preferred_language: lead.preferredLanguage ?? null,
    source: lead.source ?? "website",
    contact_consent: lead.contactConsent ?? null,

    budget: lead.budget ?? null,
    move_in_date: lead.moveInDate ?? null,
    areas: lead.areas ?? null,
    screening_profile: lead.screeningProfile ?? null,

    price_range: lead.priceRange ?? null,
    financing_status: lead.financingStatus ?? null,

    property_address: lead.propertyAddress ?? null,
    seller_goal: lead.sellerGoal ?? null,

    property_area: lead.propertyArea ?? null,
    property_type: lead.propertyType ?? null,
    ready_to_lease: lead.readyToLease ?? null,

    ip: lead.ip ?? null,
    user_agent: lead.userAgent ?? null,

    raw: safeRaw,
  };

  const { data, error } = await supabase
    .from("leads")
    .insert(insertPayload)
    .select("id")
    .single();

  if (error) return { ok: false as const, error };
  return { ok: true as const, id: data?.id as string | undefined };
}

function formatLeadSummary(
  lead: NormalizedLead,
  segment: Segment,
  reasons: string[],
  id?: string | null
) {
  const lines: string[] = [];

  lines.push(New lead (${lead.leadType.toUpperCase()}) — ${lead.name});
  if (id) lines.push(Lead ID: ${id});
  lines.push(Segment: ${segment});
  lines.push(Reasons: ${reasons.join(", ")});
  lines.push(Email: ${lead.email});
  lines.push(Phone: ${lead.phone});

  if (lead.intent) lines.push(Intent: ${lead.intent});
  if (lead.timeline) lines.push(Timeline: ${lead.timeline});
  if (lead.preferredLanguage) lines.push(Language: ${lead.preferredLanguage});
  if (lead.source) lines.push(Source: ${lead.source});

  if (lead.leadType === "rent") {
    if (lead.budget !== undefined) lines.push(Budget: $${lead.budget});
    if (lead.moveInDate) lines.push(Move-in: ${lead.moveInDate});
    if (lead.areas) lines.push(Areas: ${lead.areas});
    if (lead.screeningProfile) {
      lines.push(Screening profile: ${lead.screeningProfile});
    }
  }

  if (lead.leadType === "buy") {
    if (lead.priceRange) lines.push(Price range: ${lead.priceRange});
    if (lead.financingStatus) lines.push(Financing: ${lead.financingStatus});
    if (lead.areas) lines.push(Areas: ${lead.areas});
  }

  if (lead.leadType === "sell") {
    if (lead.propertyAddress) {
      lines.push(Property address: ${lead.propertyAddress});
    }
    if (lead.sellerGoal) lines.push(Seller goal: ${lead.sellerGoal});
  }

  if (lead.leadType === "landlord") {
    if (lead.propertyArea) lines.push(Property area: ${lead.propertyArea});
    if (lead.propertyType) lines.push(Property type: ${lead.propertyType});
    if (lead.readyToLease) lines.push(Ready to lease: ${lead.readyToLease});
  }

  if (lead.message) {
    lines.push("");
    lines.push("Message:");
    lines.push(lead.message);
  }

  return lines.join("\n");
}

async function sendEmailNotification(opts: { subject: string; text: string }) {
  const apiKey = process.env.RESEND_API_KEY;
  const from = process.env.LEADS_FROM_EMAIL;
  const to = process.env.LEADS_TO_EMAIL;

  if (!apiKey || !from || !to) {
    return {
```

```
      ok: false as const,
      skipped: true as const,
      error: "Missing RESEND_API_KEY/LEADS_FROM_EMAIL/LEADS_TO_EMAIL",
    };
  }

  const resend = new Resend(apiKey);

  try {
    const resp = await resend.emails.send({
      from,
      to,
      subject: opts.subject,
      text: opts.text,
    });

    return {
      ok: true as const,
      id: (resp as { data?: { id?: string } })?.data?.id ?? null,
    };
  } catch (e: unknown) {
    return {
      ok: false as const,
      skipped: false as const,
      error: e instanceof Error ? e.message : "Resend error",
    };
  }
}

async function sendSmsNotification(opts: { body: string }) {
  const sid = process.env.TWILIO_ACCOUNT_SID;
  const token = process.env.TWILIO_AUTH_TOKEN;
  const from = process.env.TWILIO_FROM_NUMBER;
  const to = process.env.TWILIO_TO_NUMBER;

  if (!sid || !token || !from || !to) {
    return {
      ok: false as const,
      skipped: true as const,
      error: "Missing Twilio env vars",
    };
  }

  const client = Twilio(sid, token);

  try {
    const msg = await client.messages.create({ from, to, body: opts.body });
    return { ok: true as const, sid: msg.sid };
  } catch (e: unknown) {
    return {
      ok: false as const,
      skipped: false as const,
      error: e instanceof Error ? e.message : "Twilio error",
    };
  }
}

export async function POST(req: Request) {
  try {
    if (!process.env.NEXT_PUBLIC_SUPABASE_URL) {
      return jsonError(500, "Missing NEXT_PUBLIC_SUPABASE_URL");
    }

    if (!process.env.SUPABASE_SERVICE_ROLE_KEY) {
      return jsonError(500, "Missing SUPABASE_SERVICE_ROLE_KEY");
    }

    const raw: unknown = await req.json().catch(() => null);
    if (!raw || typeof raw !== "object") {
      return jsonError(400, "Invalid JSON body.");
    }

    const normalized = normalizeLead(raw);
    if (!normalized.ok) {
      return jsonError(400, normalized.error);
    }

    const lead = normalized.lead;
    lead.ip = getClientIp(req);
    lead.userAgent = req.headers.get("user-agent");

    const { segment, reasons } = routeLead(lead);

    const persisted = await persistLeadToSupabase(lead, segment, reasons, raw);
    if (!persisted.ok) {
      console.error("Supabase insert failed:", persisted.error);
      return jsonError(500, "Supabase insert failed", persisted.error);
    }

    const id = persisted.id ?? null;
    const summary = formatLeadSummary(lead, segment, reasons, id);

    const emailSubject =
      lead.leadType === "rent"
        ? New RENT lead — ${lead.name} — $${lead.budget ?? "?"}/mo
        : New ${lead.leadType.toUpperCase()} lead — ${lead.name};

    const emailRes = await sendEmailNotification({
      subject: emailSubject,
      text: summary,
    });

    const smsBody =
      lead.leadType === "rent"
        ? New RENT lead: ${lead.name} | $${lead.budget ?? "?"}/mo | Move-in ${lead.moveInDate ?? "?"} | ${lead.phone}
        : New ${lead.leadType.toUpperCase()} lead: ${lead.name} | ${lead.phone};

    const smsRes = await sendSmsNotification({ body: smsBody });
```

```
    return NextResponse.json({
      ok: true,
      id,
      segment,
      reasons,
      notifications: {
        email: emailRes,
        sms: smsRes,
      },
    });
  } catch (err: unknown) {
    console.error("API /api/lead crashed:", err);
    return jsonError(500, "Server error", err);
  }
}
```

Your route is in good shape. The one change you still need is to add `status: "new"` into the Supabase insert payload.
Make this exact edit inside `persistLeadToSupabase()`.
Find this block:

```
const insertPayload = {
  segment,
  lead_type: lead.leadType,
  reasons,

  name: lead.name,
```

Change it to:

```
const insertPayload = {
  segment,
  lead_type: lead.leadType,
  reasons,
  status: "new",

  name: lead.name,
```

So that whole section becomes:

```
const insertPayload = {
  segment,
  lead_type: lead.leadType,
  reasons,
  status: "new",

  name: lead.name,
  email: lead.email,
  phone: lead.phone,
  message: lead.message ?? null,

  intent: lead.intent ?? null,
  timeline: lead.timeline ?? null,
  preferred_language: lead.preferredLanguage ?? null,
  source: lead.source ?? "website",
  contact_consent: lead.contactConsent ?? null,

  budget: lead.budget ?? null,
  move_in_date: lead.moveInDate ?? null,
  areas: lead.areas ?? null,
  screening_profile: lead.screeningProfile ?? null,

  price_range: lead.priceRange ?? null,
  financing_status: lead.financingStatus ?? null,

  property_address: lead.propertyAddress ?? null,
  seller_goal: lead.sellerGoal ?? null,

  property_area: lead.propertyArea ?? null,
  property_type: lead.propertyType ?? null,
  ready_to_lease: lead.readyToLease ?? null,

  ip: lead.ip ?? null,
  user_agent: lead.userAgent ?? null,

  raw: safeRaw,
};
```

After that:

1. Save the file.
2. Restart dev server if needed:

```
npm run dev
```

3. Submit one test intake.
4. Check Supabase with:

```
select
  created_at,
  name,
  lead_type,
  status,
  timeline,
  source,
  segment
from leads
order by created_at desc
limit 10;
```

You want to see `status = new`.
One more small improvement I recommend right now: in your `NormalizedLead` type, make `timeline` a controlled enum later instead of plain `string`. For now, the urgent move is just adding `status: "new"` so the pipeline starts working.

can you make the recommended changes and write the code so i may copy and paste?

Yes — here is your updated `app/api/lead/route.ts` with the recommended change already included.
The key change is:

status: "new",

inside insertPayload.
Copy and paste this entire file:

```
import { NextResponse } from "next/server";
import { createSupabaseServiceClient } from "@/lib/supabase/service";
import { Resend } from "resend";
import Twilio from "twilio";

type LeadType = "rent" | "buy" | "sell" | "landlord" | "other";
type Intent = "tenant" | "buyer" | "seller" | "landlord" | "other";
type PreferredLanguage = "en" | "es" | "ar";

type Segment =
  | "RENT_FAST_TRACK"
  | "RENT_REVIEW"
  | "BUY_SELL"
  | "LANDLORD"
  | "OTHER";

type NormalizedLead = {
  leadType: LeadType;
  name: string;
  email: string;
  phone: string;
  message?: string;

  intent?: Intent;
  timeline?: string;
  preferredLanguage?: PreferredLanguage;
  source?: string;
  contactConsent?: boolean;

  budget?: number;
  moveInDate?: string;
  areas?: string;
  screeningProfile?: string;

  priceRange?: string;
  financingStatus?: string;

  propertyAddress?: string;
  sellerGoal?: string;

  propertyArea?: string;
  propertyType?: string;
  readyToLease?: string;

  ip?: string | null;
  userAgent?: string | null;
};

type NormalizeLeadResult =
  | { ok: true; lead: NormalizedLead }
  | { ok: false; error: string };

function jsonError(status: number, message: string, extra?: unknown) {
  return NextResponse.json(
    {
      ok: false,
      error: message,
      ...(process.env.NODE_ENV !== "production" ? { debug: extra } : {}),
    },
    { status }
  );
}

function cleanString(v: unknown, max = 2000): string {
  const s = String(v ?? "").trim();
  if (!s) return "";
  return s.length > max ? s.slice(0, max) : s;
}

function cleanEmail(v: unknown): string {
  return cleanString(v, 320).toLowerCase();
}

function isValidEmail(email: string): boolean {
  return /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email);
}

function cleanPhone(v: unknown): string | undefined {
  const s = cleanString(v, 50);
  if (!s) return undefined;

  const normalized = s.replace(/[^\d+()\-\s]/g, "").trim();
  const plusCount = (normalized.match(/\+/g) || []).length;

  if (plusCount > 1) return undefined;
  if (normalized.includes("+") && !normalized.startsWith("+")) return undefined;

  const digits = normalized.replace(/\D/g, "");
  if (digits.length < 8 || digits.length > 15) return undefined;

  if (normalized.startsWith("+")) {
    return `+${digits}`;
  }

  if (digits.length === 10) {
    return `+1${digits}`;
  }

  if (digits.length === 11 && digits.startsWith("1")) {
    return `+${digits}`;
  }
```

```
  return `+${digits}`;
}

function toNumber(v: unknown): number | undefined {
  const s = cleanString(v, 32);
  if (!s) return undefined;

  const n = Number(s.replace(/,/g, ""));
  return Number.isFinite(n) ? n : undefined;
}

function toBool(v: unknown): boolean | undefined {
  if (typeof v === "boolean") return v;

  const s = cleanString(v, 16).toLowerCase();
  if (!s) return undefined;

  if (["true", "t", "yes", "y", "1", "on"].includes(s)) return true;
  if (["false", "f", "no", "n", "0", "off"].includes(s)) return false;

  return undefined;
}

function isValidLeadType(v: unknown): v is LeadType {
  return (
    v === "rent" ||
    v === "buy" ||
    v === "sell" ||
    v === "landlord" ||
    v === "other"
  );
}

function isValidIntent(v: unknown): v is Intent {
  return (
    v === "tenant" ||
    v === "buyer" ||
    v === "seller" ||
    v === "landlord" ||
    v === "other"
  );
}

function cleanIntent(v: unknown): Intent | undefined {
  const s = cleanString(v, 40).toLowerCase();
  return isValidIntent(s) ? s : undefined;
}

function cleanLanguage(v: unknown): PreferredLanguage | undefined {
  const s = cleanString(v, 10).toLowerCase();
  if (s === "en" || s === "es" || s === "ar") return s;
  return undefined;
}

function cleanTimeline(v: unknown): string | undefined {
  const s = cleanString(v, 40).toLowerCase();
  if (!s) return undefined;
  return s;
}

function normalizeLead(raw: unknown): NormalizeLeadResult {
  if (!raw || typeof raw !== "object") {
    return { ok: false, error: "Invalid request body." };
  }

  const body = raw as Record<string, unknown>;

  const leadTypeRaw = body.leadType;
  if (!isValidLeadType(leadTypeRaw)) {
    return { ok: false, error: "Invalid lead type." };
  }

  const name = cleanString(body.name, 120);
  const email = cleanEmail(body.email);
  const phone = cleanPhone(body.phone);
  const message = cleanString(body.message, 4000) || undefined;

  if (!name) {
    return { ok: false, error: "Name is required." };
  }

  if (!email || !isValidEmail(email)) {
    return { ok: false, error: "Valid email is required." };
  }

  if (!phone) {
    return {
      ok: false,
      error: "Valid phone is required. Include country code if outside the U.S.",
    };
  }

  const leadType: LeadType = leadTypeRaw;

  const lead: NormalizedLead = {
    leadType,
    name,
    email,
    phone,
    message,
    intent: cleanIntent(body.intent),
    timeline: cleanTimeline(body.timeline),
    preferredLanguage: cleanLanguage(body.preferredLanguage) ?? "en",
    source: cleanString(body.source, 80).toLowerCase() || "website",
    contactConsent: toBool(body.contactConsent),
  };

  if (lead.contactConsent !== true) {
```

```
    return { ok: false, error: "Contact consent is required." };
  }

  if (!lead.timeline) {
    return { ok: false, error: "Timeline is required." };
  }

  if (leadType === "rent") {
    lead.budget = toNumber(body.budget);
    lead.moveInDate = cleanString(body.moveInDate, 120) || undefined;
    lead.areas = cleanString(body.areas, 400) || undefined;
    lead.screeningProfile =
      cleanString(body.screeningProfile, 80).toLowerCase() || undefined;

    if (lead.budget === undefined || lead.budget < 500) {
      return { ok: false, error: "Minimum rental budget is $500." };
    }

    if (!lead.moveInDate) {
      return { ok: false, error: "Move-in date is required for rentals." };
    }

    if (!lead.areas) {
      return { ok: false, error: "Preferred area is required for rentals." };
    }

    if (!lead.screeningProfile) {
      return { ok: false, error: "Screening profile is required for rentals." };
    }
  }

  if (leadType === "buy") {
    lead.priceRange = cleanString(body.priceRange, 120) || undefined;
    lead.financingStatus =
      cleanString(body.financingStatus, 80).toLowerCase() || undefined;
    lead.areas = cleanString(body.areas, 400) || undefined;

    if (!lead.priceRange || !lead.financingStatus || !lead.areas) {
      return { ok: false, error: "Missing buyer intake fields." };
    }
  }

  if (leadType === "sell") {
    lead.propertyAddress =
      cleanString(body.propertyAddress, 250) || undefined;
    lead.sellerGoal =
      cleanString(body.sellerGoal, 80).toLowerCase() || undefined;

    if (!lead.propertyAddress || !lead.sellerGoal) {
      return { ok: false, error: "Missing seller intake fields." };
    }
  }

  if (leadType === "landlord") {
    lead.propertyArea = cleanString(body.propertyArea, 120) || undefined;
    lead.propertyType =
      cleanString(body.propertyType, 80).toLowerCase() || undefined;
    lead.readyToLease =
      cleanString(body.readyToLease, 20).toLowerCase() || undefined;

    if (!lead.propertyArea || !lead.propertyType || !lead.readyToLease) {
      return { ok: false, error: "Missing landlord intake fields." };
    }
  }

  return { ok: true, lead };
}

function routeLead(lead: NormalizedLead): { segment: Segment; reasons: string[] } {
  if (lead.leadType === "buy" || lead.leadType === "sell") {
    return { segment: "BUY_SELL", reasons: [`lead_type_${lead.leadType}`] };
  }

  if (lead.leadType === "landlord") {
    return { segment: "LANDLORD", reasons: ["lead_type_landlord"] };
  }

  if (lead.leadType === "other") {
    return { segment: "OTHER", reasons: ["lead_type_other"] };
  }

  const reasons: string[] = [];

  if (
    lead.screeningProfile === "clean" ||
    lead.screeningProfile === "no_us_credit"
  ) {
    reasons.push(`screening_profile_${lead.screeningProfile}`);
    return { segment: "RENT_FAST_TRACK", reasons };
  }

  reasons.push(`screening_profile_${lead.screeningProfile ?? "unknown"}`);
  return { segment: "RENT_REVIEW", reasons };
}

function getClientIp(req: Request): string | null {
  const xff = req.headers.get("x-forwarded-for");
  if (xff) return xff.split(",")[0]?.trim() || null;

  const realIp = req.headers.get("x-real-ip");
  return realIp?.trim() || null;
}

function sanitizeRawForStorage(raw: unknown) {
  if (!raw || typeof raw !== "object") return {};

  const body = raw as Record<string, unknown>;
```

```
  return {
    leadType: cleanString(body.leadType, 40),
    name: cleanString(body.name, 120),
    email: cleanEmail(body.email),
    phone: cleanString(body.phone, 50),
    message: cleanString(body.message, 4000) || null,

    intent: cleanString(body.intent, 40) || null,
    timeline: cleanString(body.timeline, 40) || null,
    preferredLanguage: cleanString(body.preferredLanguage, 10) || null,
    source: cleanString(body.source, 80) || null,
    contactConsent: toBool(body.contactConsent) ?? null,

    budget: toNumber(body.budget) ?? null,
    moveInDate: cleanString(body.moveInDate, 120) || null,
    areas: cleanString(body.areas, 400) || null,
    screeningProfile: cleanString(body.screeningProfile, 80) || null,

    priceRange: cleanString(body.priceRange, 120) || null,
    financingStatus: cleanString(body.financingStatus, 80) || null,

    propertyAddress: cleanString(body.propertyAddress, 250) || null,
    sellerGoal: cleanString(body.sellerGoal, 80) || null,

    propertyArea: cleanString(body.propertyArea, 120) || null,
    propertyType: cleanString(body.propertyType, 80) || null,
    readyToLease: cleanString(body.readyToLease, 20) || null,
  };
}

async function persistLeadToSupabase(
  lead: NormalizedLead,
  segment: Segment,
  reasons: string[],
  raw: unknown
) {
  const supabase = createSupabaseServiceClient();
  const safeRaw = sanitizeRawForStorage(raw);

  const insertPayload = {
    segment,
    lead_type: lead.leadType,
    reasons,
    status: "new",

    name: lead.name,
    email: lead.email,
    phone: lead.phone,
    message: lead.message ?? null,

    intent: lead.intent ?? null,
    timeline: lead.timeline ?? null,
    preferred_language: lead.preferredLanguage ?? null,
    source: lead.source ?? "website",
    contact_consent: lead.contactConsent ?? null,

    budget: lead.budget ?? null,
    move_in_date: lead.moveInDate ?? null,
    areas: lead.areas ?? null,
    screening_profile: lead.screeningProfile ?? null,

    price_range: lead.priceRange ?? null,
    financing_status: lead.financingStatus ?? null,

    property_address: lead.propertyAddress ?? null,
    seller_goal: lead.sellerGoal ?? null,

    property_area: lead.propertyArea ?? null,
    property_type: lead.propertyType ?? null,
    ready_to_lease: lead.readyToLease ?? null,

    ip: lead.ip ?? null,
    user_agent: lead.userAgent ?? null,

    raw: safeRaw,
  };

  const { data, error } = await supabase
    .from("leads")
    .insert(insertPayload)
    .select("id")
    .single();

  if (error) return { ok: false as const, error };
  return { ok: true as const, id: data?.id as string | undefined };
}

function formatLeadSummary(
  lead: NormalizedLead,
  segment: Segment,
  reasons: string[],
  id?: string | null
) {
  const lines: string[] = [];

  lines.push(`New lead (${lead.leadType.toUpperCase()}) — ${lead.name}`);
  if (id) lines.push(`Lead ID: ${id}`);
  lines.push(`Segment: ${segment}`);
  lines.push(`Reasons: ${reasons.join(", ")}`);
  lines.push(`Email: ${lead.email}`);
  lines.push(`Phone: ${lead.phone}`);

  if (lead.intent) lines.push(`Intent: ${lead.intent}`);
  if (lead.timeline) lines.push(`Timeline: ${lead.timeline}`);
  if (lead.preferredLanguage) lines.push(`Language: ${lead.preferredLanguage}`);
  if (lead.source) lines.push(`Source: ${lead.source}`);

  if (lead.leadType === "rent") {
```

```
    if (lead.budget !== undefined) lines.push(`Budget: $${lead.budget}`);
    if (lead.moveInDate) lines.push(`Move-in: ${lead.moveInDate}`);
    if (lead.areas) lines.push(`Areas: ${lead.areas}`);
    if (lead.screeningProfile) {
      lines.push(`Screening profile: ${lead.screeningProfile}`);
    }
  }

  if (lead.leadType === "buy") {
    if (lead.priceRange) lines.push(`Price range: ${lead.priceRange}`);
    if (lead.financingStatus) lines.push(`Financing: ${lead.financingStatus}`);
    if (lead.areas) lines.push(`Areas: ${lead.areas}`);
  }

  if (lead.leadType === "sell") {
    if (lead.propertyAddress) {
      lines.push(`Property address: ${lead.propertyAddress}`);
    }
    if (lead.sellerGoal) lines.push(`Seller goal: ${lead.sellerGoal}`);
  }

  if (lead.leadType === "landlord") {
    if (lead.propertyArea) lines.push(`Property area: ${lead.propertyArea}`);
    if (lead.propertyType) lines.push(`Property type: ${lead.propertyType}`);
    if (lead.readyToLease) lines.push(`Ready to lease: ${lead.readyToLease}`);
  }

  if (lead.message) {
    lines.push("");
    lines.push("Message:");
    lines.push(lead.message);
  }

  return lines.join("\n");
}

async function sendEmailNotification(opts: { subject: string; text: string }) {
  const apiKey = process.env.RESEND_API_KEY;
  const from = process.env.LEADS_FROM_EMAIL;
  const to = process.env.LEADS_TO_EMAIL;

  if (!apiKey || !from || !to) {
    return {
      ok: false as const,
      skipped: true as const,
      error: "Missing RESEND_API_KEY/LEADS_FROM_EMAIL/LEADS_TO_EMAIL",
    };
  }

  const resend = new Resend(apiKey);

  try {
    const resp = await resend.emails.send({
      from,
      to,
      subject: opts.subject,
      text: opts.text,
    });

    return {
      ok: true as const,
      id: (resp as { data?: { id?: string } })?.data?.id ?? null,
    };
  } catch (e: unknown) {
    return {
      ok: false as const,
      skipped: false as const,
      error: e instanceof Error ? e.message : "Resend error",
    };
  }
}

async function sendSmsNotification(opts: { body: string }) {
  const sid = process.env.TWILIO_ACCOUNT_SID;
  const token = process.env.TWILIO_AUTH_TOKEN;
  const from = process.env.TWILIO_FROM_NUMBER;
  const to = process.env.TWILIO_TO_NUMBER;

  if (!sid || !token || !from || !to) {
    return {
      ok: false as const,
      skipped: true as const,
      error: "Missing Twilio env vars",
    };
  }

  const client = Twilio(sid, token);

  try {
    const msg = await client.messages.create({ from, to, body: opts.body });
    return { ok: true as const, sid: msg.sid };
  } catch (e: unknown) {
    return {
      ok: false as const,
      skipped: false as const,
      error: e instanceof Error ? e.message : "Twilio error",
    };
  }
}

export async function POST(req: Request) {
  try {
    if (!process.env.NEXT_PUBLIC_SUPABASE_URL) {
      return jsonError(500, "Missing NEXT_PUBLIC_SUPABASE_URL");
    }

    if (!process.env.SUPABASE_SERVICE_ROLE_KEY) {
      return jsonError(500, "Missing SUPABASE_SERVICE_ROLE_KEY");
```

```
    }

    const raw: unknown = await req.json().catch(() => null);
    if (!raw || typeof raw !== "object") {
      return jsonError(400, "Invalid JSON body.");
    }

    const normalized = normalizeLead(raw);
    if (!normalized.ok) {
      return jsonError(400, normalized.error);
    }

    const lead = normalized.lead;
    lead.ip = getClientIp(req);
    lead.userAgent = req.headers.get("user-agent");

    const { segment, reasons } = routeLead(lead);

    const persisted = await persistLeadToSupabase(lead, segment, reasons, raw);
    if (!persisted.ok) {
      console.error("Supabase insert failed:", persisted.error);
      return jsonError(500, "Supabase insert failed", persisted.error);
    }

    const id = persisted.id ?? null;
    const summary = formatLeadSummary(lead, segment, reasons, id);

    const emailSubject =
      lead.leadType === "rent"
        ? `New RENT lead — ${lead.name} — $${lead.budget ?? "?"}/mo`
        : `New ${lead.leadType.toUpperCase()} lead — ${lead.name}`;

    const emailRes = await sendEmailNotification({
      subject: emailSubject,
      text: summary,
    });

    const smsBody =
      lead.leadType === "rent"
        ? `New RENT lead: ${lead.name} | $${lead.budget ?? "?"}/mo | Move-in ${lead.moveInDate ?? "?"} | ${lead.phone}`
        : `New ${lead.leadType.toUpperCase()} lead: ${lead.name} | ${lead.phone}`;

    const smsRes = await sendSmsNotification({ body: smsBody });

    return NextResponse.json({
      ok: true,
      id,
      segment,
      reasons,
      notifications: {
        email: emailRes,
        sms: smsRes,
      },
    });
  } catch (err: unknown) {
    console.error("API /api/lead crashed:", err);
    return jsonError(500, "Server error", err);
  }
}
```

Then save it, restart your dev server, and test one intake submission.
After that, run this in Supabase SQL Editor to confirm it worked:

```
select
  created_at,
  name,
  lead_type,
  status,
  timeline,
  source,
  segment
from leads
order by created_at desc
limit 10;
```

You should see the new record with:

status = new

The next highest-value step after this is building /dashboard/leads so you can actually work the pipeline from the data you're now collecting.

ok, lets go

Good. Next we build the first private CRM view:
app/dashboard/leads/page.tsx
This gives you a live internal page showing the leads you are now storing.
Use this code:

```
import { createSupabaseServiceClient } from "@/lib/supabase/service";

type LeadRow = {
  id: string;
  created_at: string;
  name: string;
  email: string;
  phone: string;
  lead_type: string;
  status: string | null;
  timeline: string | null;
  source: string | null;
  segment: string | null;
};

function formatDate(value: string) {
  try {
    return new Intl.DateTimeFormat("en-US", {
      month: "short",
      day: "numeric",
```

```
      year: "numeric",
      hour: "numeric",
      minute: "2-digit",
    }).format(new Date(value));
  } catch {
    return value;
  }
}

function labelize(value: string | null | undefined) {
  if (!value) return "—";
  return value
    .replace(/_/g, " ")
    .toLowerCase()
    .replace(/\b\w/g, (c) => c.toUpperCase());
}

export default async function DashboardLeadsPage() {
  const supabase = createSupabaseServiceClient();

  const { data, error } = await supabase
    .from("leads")
    .select(
      "id, created_at, name, email, phone, lead_type, status, timeline, source, segment"
    )
    .order("created_at", { ascending: false })
    .limit(100);

  const leads = (data ?? []) as LeadRow[];

  return (
    <main className="min-h-screen bg-[#f5f5f3] text-neutral-950">
      <section className="mx-auto max-w-7xl px-6 py-12">
        <div className="mb-8">
          <p className="text-sm font-medium uppercase tracking-[0.18em] text-neutral-500">
            Dashboard
          </p>
          <h1 className="mt-3 text-3xl font-semibold tracking-tight md:text-4xl">
            Leads
          </h1>
          <p className="mt-3 max-w-3xl text-sm leading-7 text-neutral-600">
            Internal lead pipeline for LonestarLiving. New submissions should
            appear here after intake form completion.
          </p>
        </div>

        <div className="overflow-hidden rounded-[1.75rem] border border-black/5 bg-white">
          {error ? (
            <div className="p-6 text-sm text-red-600">
              Failed to load leads.
            </div>
          ) : leads.length === 0 ? (
            <div className="p-6 text-sm text-neutral-600">
              No leads yet.
            </div>
          ) : (
            <div className="overflow-x-auto">
              <table className="min-w-full border-separate border-spacing-0">
                <thead>
                  <tr className="bg-neutral-50 text-left">
                    <th className="border-b border-black/5 px-4 py-3 text-xs font-semibold uppercase tracking-[0.14em] text-neutral-500">
                      Created
                    </th>
                    <th className="border-b border-black/5 px-4 py-3 text-xs font-semibold uppercase tracking-[0.14em] text-neutral-500">
                      Name
                    </th>
                    <th className="border-b border-black/5 px-4 py-3 text-xs font-semibold uppercase tracking-[0.14em] text-neutral-500">
                      Lead Type
                    </th>
                    <th className="border-b border-black/5 px-4 py-3 text-xs font-semibold uppercase tracking-[0.14em] text-neutral-500">
                      Status
                    </th>
                    <th className="border-b border-black/5 px-4 py-3 text-xs font-semibold uppercase tracking-[0.14em] text-neutral-500">
                      Timeline
                    </th>
                    <th className="border-b border-black/5 px-4 py-3 text-xs font-semibold uppercase tracking-[0.14em] text-neutral-500">
                      Source
                    </th>
                    <th className="border-b border-black/5 px-4 py-3 text-xs font-semibold uppercase tracking-[0.14em] text-neutral-500">
                      Segment
                    </th>
                    <th className="border-b border-black/5 px-4 py-3 text-xs font-semibold uppercase tracking-[0.14em] text-neutral-500">
                      Contact
                    </th>
                  </tr>
                </thead>

                <tbody>
                  {leads.map((lead) => (
                    <tr key={lead.id} className="align-top">
                      <td className="border-b border-black/5 px-4 py-4 text-sm text-neutral-600">
                        {formatDate(lead.created_at)}
                      </td>

                      <td className="border-b border-black/5 px-4 py-4">
                        <div className="font-medium text-neutral-950">
                          {lead.name}
                        </div>
                      </td>

                      <td className="border-b border-black/5 px-4 py-4 text-sm text-neutral-700">
                        {labelize(lead.lead_type)}
                      </td>

                      <td className="border-b border-black/5 px-4 py-4 text-sm text-neutral-700">
                        {labelize(lead.status)}
                      </td>
```

```
                <td className="border-b border-black/5 px-4 py-4 text-sm text-neutral-700">
                  {labelize(lead.timeline)}
                </td>

                <td className="border-b border-black/5 px-4 py-4 text-sm text-neutral-700">
                  {labelize(lead.source)}
                </td>

                <td className="border-b border-black/5 px-4 py-4 text-sm text-neutral-700">
                  {labelize(lead.segment)}
                </td>

                <td className="border-b border-black/5 px-4 py-4 text-sm text-neutral-600">
                  <div>{lead.email}</div>
                  <div className="mt-1">{lead.phone}</div>
                </td>
              </tr>
            ))}
          </tbody>
        </table>
      </div>
    ))}
  </div>
 </section>
 </main>
);
}
```

Then create the parent route too:
app/dashboard/page.tsx

```
import Link from "next/link";

export default function DashboardPage() {
  return (
    <main className="min-h-screen bg-[#f5f5f3] text-neutral-950">
      <section className="mx-auto max-w-5xl px-6 py-12">
        <p className="text-sm font-medium uppercase tracking-[0.18em] text-neutral-500">
          Dashboard
        </p>

        <h1 className="mt-3 text-3xl font-semibold tracking-tight md:text-4xl">
          Private operations
        </h1>

        <p className="mt-4 max-w-2xl text-sm leading-7 text-neutral-600">
          Internal command center for lead management and workflow.
        </p>

        <div className="mt-8 grid gap-6 md:grid-cols-2">
          <Link
            href="/dashboard/leads"
            className="rounded-[1.5rem] border border-black/5 bg-white p-6 transition hover:border-black/10"
          >
            <h2 className="text-lg font-semibold tracking-tight text-neutral-950">
              Leads
            </h2>
            <p class="mt-2 text-sm leading-7 text-neutral-600">
              View incoming submissions, status, source, and segmentation.
            </p>
          </Link>
        </div>
      </section>
    </main>
  );
}
```

Then test in this order:

1. Save both files.
2. Run:

```
npm run dev
```

3. Visit:

http://localhost:3000/dashboard
http://localhost:3000/dashboard/leads

4. Submit one intake form.
5. Refresh /dashboard/leads.

If you get a Supabase error on the dashboard page, the next file to check is `lib/supabase/service.ts`, because the dashboard is using the service client server-side.

my current code: // lib/supabase/service.ts

```
import { createClient } from "@supabase/supabase-js";

export function createSupabaseServiceClient() {
  const url = process.env.NEXT_PUBLIC_SUPABASE_URL;
  const serviceKey = process.env.SUPABASE_SERVICE_ROLE_KEY;

  if (!url) throw new Error("Missing NEXT_PUBLIC_SUPABASE_URL");
  if (!serviceKey) throw new Error("Missing SUPABASE_SERVICE_ROLE_KEY");

  return createClient(url, serviceKey, { auth: { persistSession: false } });
}
```

Your current `service.ts` is **almost correct**, but there is one important improvement we should make for stability and security in Next.js server routes.
Right now it works, but it does **not explicitly disable browser usage**, which can cause issues if it is accidentally imported client-side.
We will fix that and make it production-safe.

Replace your file with this
lib/supabase/service.ts

```
import "server-only";
import { createClient } from "@supabase/supabase-js";

export function createSupabaseServiceClient() {
  const url = process.env.NEXT_PUBLIC_SUPABASE_URL;
```

```
  const serviceKey = process.env.SUPABASE_SERVICE_ROLE_KEY;

  if (!url) {
    throw new Error("Missing NEXT_PUBLIC_SUPABASE_URL");
  }

  if (!serviceKey) {
    throw new Error("Missing SUPABASE_SERVICE_ROLE_KEY");
  }

  const supabase = createClient(url, serviceKey, {
    auth: {
      persistSession: false,
      autoRefreshToken: false,
    },
  });

  return supabase;
}
```

Why this change matters
1. "server-only"

```
import "server-only";
```

This prevents the service role key from ever leaking into client bundles.
If you accidentally import this file into a client component, Next.js will throw an error instead of exposing the key.
This is extremely important.

2. Disable session handling

```
persistSession: false
autoRefreshToken: false
```

You are using the **service role**, not a user session.
This avoids unnecessary auth behavior.

Your architecture after this change
Your backend pipeline is now:

```
Next.js Form
     ↓
API Route (/api/lead)
     ↓
Validation
     ↓
Supabase Service Client
     ↓
Database insert
     ↓
Email notification
     ↓
SMS notification
     ↓
Dashboard view
```

That is exactly the correct architecture for your platform.

Now test everything
Restart the dev server:

```
npm run dev
```

Open:

http://localhost:3000/dashboard/leads

Then submit an intake form.
You should see the lead appear immediately.

Next high-ROI improvement
The next step that dramatically improves your system is adding **status control inside the dashboard**, so you can move leads like:

```
NEW
→ CONTACTED
→ QUALIFIED
→ SHOWING
→ APPLICATION
→ CLOSED
→ LOST
```

This turns your website from lead capture → real estate operating system.
If you want, I can show you the 30-line upgrade that adds status controls directly into the dashboard table.

ok, can we do this using Terminal for instructions, then we paste in vs code?

Yes. Do it in Terminal first, then paste the code into VS Code.
Follow this exact sequence.
1. Go to the project

```
cd ~/Desktop/zehaifirealty
pwd
```

You should see your project path.
2. Confirm the file exists

```
ls lib/supabase
```

You should see:

service.ts

3. Print the current file to Terminal

```
cat lib/supabase/service.ts
```

That lets you verify you are editing the correct file.
4. Keep a backup before changing it

```
cp lib/supabase/service.ts lib/supabase/service.ts.bak
```

Check it:

```
ls lib/supabase
```

You should now have both:

service.ts
service.ts.bak

5. Paste this replacement into VS Code
Open:

```
code lib/supabase/service.ts
```

Then replace the file contents with this:

```
import "server-only";
import { createClient } from "@supabase/supabase-js";

export function createSupabaseServiceClient() {
  const url = process.env.NEXT_PUBLIC_SUPABASE_URL;
  const serviceKey = process.env.SUPABASE_SERVICE_ROLE_KEY;

  if (!url) {
    throw new Error("Missing NEXT_PUBLIC_SUPABASE_URL");
  }

  if (!serviceKey) {
    throw new Error("Missing SUPABASE_SERVICE_ROLE_KEY");
  }

  return createClient(url, serviceKey, {
    auth: {
      persistSession: false,
      autoRefreshToken: false,
    },
  });
}
```

Save the file.
6. Verify the saved contents in Terminal

```
cat lib/supabase/service.ts
```

You should now see:

- import "server-only";
- persistSession: false
- autoRefreshToken: false

7. Restart the dev server
If a dev server is already running, stop it with:

```
Ctrl + C
```

Then run:

```
npm run dev
```

8. Test the dashboard page
Open in browser:

http://localhost:3000/dashboard
http://localhost:3000/dashboard/leads

9. If the dashboard errors, check Terminal output
Look for errors like:

- missing env vars
- table column mismatch
- service client import issue

If it fails, paste the Terminal error exactly.
10. Optional quick diff check
To compare old vs new:

```
diff -u lib/supabase/service.ts.bak lib/supabase/service.ts
```

That shows exactly what changed.
After this, the next Terminal-guided step should be creating `app/dashboard/page.tsx` and `app/dashboard/leads/page.tsx`, then testing the first live CRM view.

Terminal code: Last login: Sat Mar 7 22:00:04 on ttys001

```
abraham@Abrahamzehaifi zehaifirealty % diff -u lib/supabase/service.ts.bak lib/supabase/service.ts
--- lib/supabase/service.ts.bak   2026-03-07 22:00:19
+++ lib/supabase/service.ts       2026-03-07 22:12:59
@@ -13,12 +13,10 @@
     throw new Error("Missing SUPABASE_SERVICE_ROLE_KEY");
   }

-  const supabase = createClient(url, serviceKey, {
+  return createClient(url, serviceKey, {
     auth: {
       persistSession: false,
       autoRefreshToken: false,
     },
   });
-
-  return supabase;
 }
\ No newline at end of file
abraham@Abrahamzehaifi zehaifirealty %
```

That diff looks fine.
What changed is the only meaningful runtime cleanup in that file:

- removed the temporary `const supabase = ...`
- returned `createClient(...)` directly
- kept `persistSession: false`
- kept `autoRefreshToken: false`

The `\ No newline at end of file` line is not a real problem. You can ignore it, though adding a final blank line is cleaner.
Quick verification
Run this to confirm the full file has the server-only guard:

```
cat lib/supabase/service.ts
```

You want to see this at the top:

```
import "server-only";
```

And this in the client config:

```
persistSession: false,
autoRefreshToken: false,
```

Next step
Restart the dev server:

```
npm run dev
```

Then test these routes in the browser:

http://localhost:3000/dashboard
http://localhost:3000/dashboard/leads

If `/dashboard/leads` fails, paste the exact Terminal error.
If it loads, submit one intake form and then refresh `/dashboard/leads` to confirm the CRM view is working.

ok, what's next? what about next step is extremely powerful: I can show you how to build a private /dashboard/leads page in Next.js that automatically shows and sorts all your leads. That turns LonestarLiving into a real CRM, when do we do this?

Now.
That is the right next step.
You have already done the foundation work:

- intake form
- API route
- Supabase storage
- lead status default
- service client hardening

The next move is to **surface the data inside a private dashboard**. That is the point where LonestarLiving starts behaving like a CRM instead of just a lead capture site.
What we build next
Two files:

app/dashboard/page.tsx
app/dashboard/leads/page.tsx

Why this is the highest-ROI next step
Because right now your system can collect leads, but you still do not have a clean internal place to:

- see new submissions
- sort by newest
- view source
- view status
- view segment
- work the pipeline

Without that, the backend exists, but the operating layer is still missing.
The order from here
Do this in sequence:
1. Create the dashboard landing page
Simple internal entry point.
2. Create /dashboard/leads
This pulls the latest leads from Supabase and shows them in a table.
3. Test it
Submit one form, refresh the dashboard, confirm the lead appears.
4. Then improve it
After it works, we add:

- status badges
- filters
- sorting
- lead detail view
- status update actions

That becomes your real private CRM.

Step 1 — create app/dashboard/page.tsx
Paste this:

```
import Link from "next/link";

export default function DashboardPage() {
  return (
    <main className="min-h-screen bg-[#f5f5f3] text-neutral-950">
      <section className="mx-auto max-w-5xl px-6 py-12">
        <p className="text-sm font-medium uppercase tracking-[0.18em] text-neutral-500">
          Dashboard
        </p>

        <h1 className="mt-3 text-3xl font-semibold tracking-tight md:text-4xl">
          Private operations
        </h1>

        <p className="mt-4 max-w-2xl text-sm leading-7 text-neutral-600">
          Internal command center for lead management and workflow.
        </p>

        <div className="mt-8 grid gap-6 md:grid-cols-2">
          <Link
            href="/dashboard/leads"
            className="rounded-[1.5rem] border border-black/5 bg-white p-6 transition hover:border-black/10"
```

```
        >
          <h2 className="text-lg font-semibold tracking-tight text-neutral-950">
            Leads
          </h2>
          <p className="mt-2 text-sm leading-7 text-neutral-600">
            View incoming submissions, status, source, and segmentation.
          </p>
        </Link>
      </div>
    </section>
  </main>
 );
}
```

Step 2 — create app/dashboard/leads/page.tsx
Paste this:

```
import { createSupabaseServiceClient } from "@/lib/supabase/service";

type LeadRow = {
  id: string;
  created_at: string;
  name: string;
  email: string;
  phone: string;
  lead_type: string;
  status: string | null;
  timeline: string | null;
  source: string | null;
  segment: string | null;
};

function formatDate(value: string) {
  try {
    return new Intl.DateTimeFormat("en-US", {
      month: "short",
      day: "numeric",
      year: "numeric",
      hour: "numeric",
      minute: "2-digit",
    }).format(new Date(value));
  } catch {
    return value;
  }
}

function labelize(value: string | null | undefined) {
  if (!value) return "—";
  return value
    .replace(/_/g, " ")
    .toLowerCase()
    .replace(/\b\w/g, (c) => c.toUpperCase());
}

export default async function DashboardLeadsPage() {
  const supabase = createSupabaseServiceClient();

  const { data, error } = await supabase
    .from("leads")
    .select(
      "id, created_at, name, email, phone, lead_type, status, timeline, source, segment"
    )
    .order("created_at", { ascending: false })
    .limit(100);

  const leads = (data ?? []) as LeadRow[];

  return (
    <main className="min-h-screen bg-[#f5f5f3] text-neutral-950">
      <section className="mx-auto max-w-7xl px-6 py-12">
        <div className="mb-8">
          <p className="text-sm font-medium uppercase tracking-[0.18em] text-neutral-500">
            Dashboard
          </p>
          <h1 className="mt-3 text-3xl font-semibold tracking-tight md:text-4xl">
            Leads
          </h1>
          <p className="mt-3 max-w-3xl text-sm leading-7 text-neutral-600">
            Internal lead pipeline for LonestarLiving. New submissions should
            appear here after intake form completion.
          </p>
        </div>

        <div className="overflow-hidden rounded-[1.75rem] border border-black/5 bg-white">
          {error ? (
            <div className="p-6 text-sm text-red-600">
              Failed to load leads.
            </div>
          ) : leads.length === 0 ? (
            <div className="p-6 text-sm text-neutral-600">
              No leads yet.
            </div>
          ) : (
            <div className="overflow-x-auto">
              <table className="min-w-full border-separate border-spacing-0">
                <thead>
                  <tr className="bg-neutral-50 text-left">
                    <th className="border-b border-black/5 px-4 py-3 text-xs font-semibold uppercase tracking-[0.14em] text-neutral-500">
                      Created
                    </th>
                    <th className="border-b border-black/5 px-4 py-3 text-xs font-semibold uppercase tracking-[0.14em] text-neutral-500">
                      Name
                    </th>
                    <th className="border-b border-black/5 px-4 py-3 text-xs font-semibold uppercase tracking-[0.14em] text-neutral-500">
                      Lead Type
                    </th>
                    <th className="border-b border-black/5 px-4 py-3 text-xs font-semibold uppercase tracking-[0.14em] text-neutral-500">
```

```
                Status
              </th>
              <th className="border-b border-black/5 px-4 py-3 text-xs font-semibold uppercase tracking-[0.14em] text-neutral-500">
                Timeline
              </th>
              <th className="border-b border-black/5 px-4 py-3 text-xs font-semibold uppercase tracking-[0.14em] text-neutral-500">
                Source
              </th>
              <th className="border-b border-black/5 px-4 py-3 text-xs font-semibold uppercase tracking-[0.14em] text-neutral-500">
                Segment
              </th>
              <th className="border-b border-black/5 px-4 py-3 text-xs font-semibold uppercase tracking-[0.14em] text-neutral-500">
                Contact
              </th>
            </tr>
          </thead>

          <tbody>
            {leads.map((lead) => (
              <tr key={lead.id} className="align-top">
                <td className="border-b border-black/5 px-4 py-4 text-sm text-neutral-600">
                  {formatDate(lead.created_at)}
                </td>

                <td className="border-b border-black/5 px-4 py-4">
                  <div className="font-medium text-neutral-950">
                    {lead.name}
                  </div>
                </td>

                <td className="border-b border-black/5 px-4 py-4 text-sm text-neutral-700">
                  {labelize(lead.lead_type)}
                </td>

                <td className="border-b border-black/5 px-4 py-4 text-sm text-neutral-700">
                  {labelize(lead.status)}
                </td>

                <td className="border-b border-black/5 px-4 py-4 text-sm text-neutral-700">
                  {labelize(lead.timeline)}
                </td>

                <td className="border-b border-black/5 px-4 py-4 text-sm text-neutral-700">
                  {labelize(lead.source)}
                </td>

                <td className="border-b border-black/5 px-4 py-4 text-sm text-neutral-700">
                  {labelize(lead.segment)}
                </td>

                <td className="border-b border-black/5 px-4 py-4 text-sm text-neutral-600">
                  <div>{lead.email}</div>
                  <div className="mt-1">{lead.phone}</div>
                </td>
              </tr>
            ))}
          </tbody>
        </table>
      </div>
    )}
  </div>
 </section>
</main>
);
}
```

Step 3 — test it
In Terminal:

```
npm run dev
```

Then visit:

http://localhost:3000/dashboard
http://localhost:3000/dashboard/leads

Then submit one intake form and refresh `/dashboard/leads`.

Important note
This dashboard is still **private by obscurity**, not actually secured yet.
That is acceptable for development, but not for production.
After the page works, the next major step is:
protect /dashboard **with auth**
That is where this becomes a true internal operations system.
Paste these two files in now, run the test, and if anything errors, paste the Terminal output exactly.

ok, can we check to see in Terminal what exists before we create anything new
here is my Terminal code tab B: Last login: Sat Mar 7 22:00:04 on ttys001
abraham@Abrahamzehaifi zehaifirealty % diff -u lib/supabase/service.ts.bak lib/supabase/service.ts

```
--- lib/supabase/service.ts.bak   2026-03-07 22:00:19
+++ lib/supabase/service.ts       2026-03-07 22:12:59
@@ -13,12 +13,10 @@
   throw new Error("Missing SUPABASE_SERVICE_ROLE_KEY");
 }

-  const supabase = createClient(url, serviceKey, {
+  return createClient(url, serviceKey, {
   auth: {
     persistSession: false,
     autoRefreshToken: false,
   },
 });
-
-  return supabase;
 }
\ No newline at end of file
```

```
abraham@Abrahamzehaifi zehaifirealty % cat lib/supabase/service.ts
import "server-only";
import { createClient } from "@supabase/supabase-js";

export function createSupabaseServiceClient() {
  const url = process.env.NEXT_PUBLIC_SUPABASE_URL;
  const serviceKey = process.env.SUPABASE_SERVICE_ROLE_KEY;

  if (!url) {
    throw new Error("Missing NEXT_PUBLIC_SUPABASE_URL");
  }

  if (!serviceKey) {
    throw new Error("Missing SUPABASE_SERVICE_ROLE_KEY");
  }

  return createClient(url, serviceKey, {
    auth: {
      persistSession: false,
      autoRefreshToken: false,
    },
  });
}%                                                                    abraham@Abrahamzehaifi zehaifirealty % pwd
/Users/abraham/Desktop/zehaifirealty
abraham@Abrahamzehaifi zehaifirealty % ls app
about                   buy                     globals.css             page.tsx                sell
apartment-locator       contact                 intake                  rent                    thanks
api                     dashboard               landlords               rentals
apply                   favicon.ico             layout.tsx              search
abraham@Abrahamzehaifi zehaifirealty % ls app/dashboard
leads                   page.tsx
abraham@Abrahamzehaifi zehaifirealty % ls app/dashboard
leads                   page.tsx
abraham@Abrahamzehaifi zehaifirealty % ls app/dashboard/leads
page.tsx
abraham@Abrahamzehaifi zehaifirealty % tree app -L 2
zsh: command not found: tree
abraham@Abrahamzehaifi zehaifirealty % brew install tree
==> Auto-updating Homebrew...
Adjust how often this is run with $HOMEBREW_AUTO_UPDATE_SECS or disable with
$HOMEBREW_NO_AUTO_UPDATE=1. Hide these hints with $HOMEBREW_NO_ENV_HINTS=1 (see man brew).
tree app -L 2
==> Homebrew collects anonymous analytics.
Read the analytics documentation (and how to opt-out) here:
  https://docs.brew.sh/Analytics
No analytics have been recorded yet (nor will be during this brew run).

==> Homebrew is run entirely by unpaid volunteers. Please consider donating:
  https://github.com/Homebrew/brew#donations

==> Auto-updated Homebrew!
Updated 2 taps (homebrew/core and homebrew/cask).
==> New Formulae
apache-serf: High-performance asynchronous HTTP client library
apkeep: Command-line tool for downloading APK files from various sources
asc: Fast, lightweight CLI for App Store Connect
async-profiler: Sampling CPU & HEAP profiler for Java using AsyncGetCallTrace + perf_events
atuin-server: Sync server for atuin - Improved shell history for zsh, bash, fish and nushell
betterleaks: Secrets scanner built for configurability and speed
bitwuzla: SMT solver for bit-vectors, floating-points, arrays and uninterpreted functions
claude-agent-acp: Use Claude Code from any ACP client such as Zed!
clock-rs: Modern, digital clock that effortlessly runs in your terminal
cni-plugins: Container Network Interface plugins
containerd: Open and reliable container runtime
cwalk: Cross-platform path library for C/C++
datadog-static-analyzer: Static analysis tool for code quality and security
fracturedjson: JSON formatter that produces highly readable but fairly compact output
git-flow-next: Modern implementation of the Git-flow branching model
git-pkgs: Track package dependencies across git history
googleworkspace-cli: CLI for Drive, Gmail, Calendar, Sheets, Docs, Chat, Admin, and more
grafanactl: CLI to interact with Grafana
ironclaw: Security-first personal AI assistant with WASM sandbox channels
kaf: Modern CLI for Apache Kafka
kubectl-tree: Kubectl plugin to browse Kubernetes object hierarchies as a tree
landrun: Lightweight, secure sandbox for running Linux processes using Landlock LSM
letta-code: Memory-first coding agent
linux-headers@6.8: Header files of the Linux kernel
lld@21: LLVM Project Linker
llmfit: Find what models run on your hardware
llvm@21: Next-gen compiler infrastructure
lua@5.4: Powerful, lightweight programming language
micasa: TUI for tracking home projects, maintenance schedules, appliances and quotes
mkbrr: Is a tool to create, modify and inspect torrent files. Fast
mp4ff: Tools for parsing and manipulating MP4/ISOBMFF files
nomad-pack: Templating and packaging tool used with HashiCorp Nomad
nullclaw: Tiny autonomous AI assistant infrastructure written in Zig
nuls: NuShell-inspired ls with colorful table output
pet: Simple command-line snippet manager
pi-coding-agent: AI agent toolkit
picoclaw: Ultra-efficient personal AI assistant in Go
protobuf@33: Protocol buffers (Google's data interchange format)
pyperformance: Python benchmark suite
rootlesskit: Linux-native "fake root" for implementing rootless containers
runc: CLI tool for spawning and running containers according to the OCI specification
rustypaste-cli: CLI tool for rustypaste
shadcn: CLI for adding components to your project
skills: Open agent skills ecosystem
spicedb: Open Source, Google Zanzibar-inspired database
structurizr: Software architecture models as code
termframe: Terminal output SVG screenshot tool
termusic: Music Player TUI written in Rust
torf-cli: CLI tool for creating, reading and editing torrent files
tree-sitter-go: Go grammar for tree-sitter
tree-sitter-python: Python grammar for tree-sitter
tree-sitter-ruby: Ruby grammar for tree-sitter
vapoursynth-bestsource: Audio/video source and FFmpeg wrapper
vuls: Agentless Vulnerability Scanner for Linux/FreeBSD
```

```
x-cli: Command-line power tool for Twitter
zeptoclaw: Lightweight personal AI gateway with layered safety controls
zeroclaw: Rust-first autonomous agent runtime

You have 2 outdated formulae installed.

==> Fetching downloads for: tree
✔ Bottle Manifest tree (2.3.1)                    Downloaded    7.5KB/  7.5KB
✔ Bottle tree (2.3.1)                             Downloaded   65.6KB/ 65.6KB
==> Pouring tree--2.3.1.arm64_tahoe.bottle.tar.gz
tree app -L 2
🍺  /opt/homebrew/Cellar/tree/2.3.1: 9 files, 194.2KB
==> Running brew cleanup tree...
Disable this behaviour by setting HOMEBREW_NO_INSTALL_CLEANUP=1.
Hide these hints with HOMEBREW_NO_ENV_HINTS=1 (see man brew).
abraham@Abrahamzehaifi zehaifirealty % grep -R "dashboard" app
app/dashboard/page.tsx:         href="/dashboard/leads"
abraham@Abrahamzehaifi zehaifirealty % ls app
about                   buy                     globals.css             page.tsx                sell
apartment-locator       contact                 intake                  rent                    thanks
api                     dashboard               landlords               rentals
apply                   favicon.ico             layout.tsx              search
abraham@Abrahamzehaifi zehaifirealty % tree app -L 2
app
├── about
│   └── page.tsx
├── apartment-locator
│   └── page.tsx
├── api
│   └── lead
├── apply
│   └── page.tsx
├── buy
│   └── page.tsx
├── contact
│   └── page.tsx
├── dashboard
│   ├── leads
│   └── page.tsx
├── favicon.ico
├── globals.css
├── intake
│   ├── layout.tsx
│   └── page.tsx
├── landlords
│   └── page.tsx
├── layout.tsx
├── page.tsx
├── rent
│   └── page.tsx
├── rentals
│   └── page.tsx
├── search
│   └── page.tsx
├── sell
│   └── page.tsx
└── thanks
    ├── page.tsx
    └── rent

18 directories, 18 files
abraham@Abrahamzehaifi zehaifirealty %
```

Good move checking first. Your Terminal output tells us a lot.
What already exists (confirmed)
Your project already has the dashboard structure:

app/dashboard/page.tsx
app/dashboard/leads/page.tsx

From your tree output:

```
app/dashboard
├── leads
│   └── page.tsx
└── page.tsx
```

So the files were already created earlier.
That means we do not need to create anything new right now.
Instead we need to **inspect the existing code** to see if it matches the CRM dashboard we want.

Step 1 — Inspect the dashboard landing page
Run in Terminal:

```
cat app/dashboard/page.tsx
```

Paste the output here.

Step 2 — Inspect the leads dashboard
Run:

```
cat app/dashboard/leads/page.tsx
```

Paste the output here.

Why we check before editing
Your repo might already contain:

- a working dashboard

- a partial dashboard
- an older version

Blindly replacing files would risk deleting useful work.
Your Terminal inspection approach is exactly the right workflow.

What we are looking for
Inside app/dashboard/leads/page.tsx we want to confirm it:

1. Uses the service client

```
createSupabaseServiceClient()
```

2. Queries the leads table

```
.from("leads")
```

3. Orders newest first

```
.order("created_at", { ascending: false })
```

4. Displays columns like

```
name
lead_type
status
timeline
source
segment
```

After we inspect those two files
We will likely do one of these:
Case A — Code already good
Then we simply run:

```
http://localhost:3000/dashboard/leads
```

and test.
Case B — Code outdated
We upgrade it to the CRM view.

Your backend is already ready
Your system already has:

```
Website
 ↓
Intake
 ↓
API validation
 ↓
Supabase storage
 ↓
Email + SMS
 ↓
Dashboard (next step)
```

You are very close to a functioning internal real estate CRM.
Paste the two file outputs next:

```
cat app/dashboard/page.tsx
cat app/dashboard/leads/page.tsx
```

ok, lets check Terminal B code: ls app

```
about                   buy                     globals.css             page.tsx                sell
apartment-locator       contact                 intake                  rent                    thanks
api                     dashboard               landlords               rentals
apply                   favicon.ico             layout.tsx              search
abraham@Abrahamzehaifi zehaifirealty % ls app/dashboard
leads                   page.tsx
abraham@Abrahamzehaifi zehaifirealty % ls app/dashboard
leads                   page.tsx
abraham@Abrahamzehaifi zehaifirealty % ls app/dashboard/leads
page.tsx
abraham@Abrahamzehaifi zehaifirealty % tree app -L 2
zsh: command not found: tree
abraham@Abrahamzehaifi zehaifirealty % brew install tree
==> Auto-updating Homebrew...
Adjust how often this is run with $HOMEBREW_AUTO_UPDATE_SECS or disable with
$HOMEBREW_NO_AUTO_UPDATE=1. Hide these hints with $HOMEBREW_NO_ENV_HINTS=1 (see man brew).
tree app -L 2
==> Homebrew collects anonymous analytics.
Read the analytics documentation (and how to opt-out) here:
  https://docs.brew.sh/Analytics
No analytics have been recorded yet (nor will be during this brew run).

==> Homebrew is run entirely by unpaid volunteers. Please consider donating:
  https://github.com/Homebrew/brew#donations

==> Auto-updated Homebrew!
Updated 2 taps (homebrew/core and homebrew/cask).
==> New Formulae
apache-serf: High-performance asynchronous HTTP client library
apkeep: Command-line tool for downloading APK files from various sources
asc: Fast, lightweight CLI for App Store Connect
async-profiler: Sampling CPU & HEAP profiler for Java using AsyncGetCallTrace + perf_events
atuin-server: Sync server for atuin - Improved shell history for zsh, bash, fish and nushell
betterleaks: Secrets scanner built for configurability and speed
bitwuzla: SMT solver for bit-vectors, floating-points, arrays and uninterpreted functions
claude-agent-acp: Use Claude Code from any ACP client such as Zed!
clock-rs: Modern, digital clock that effortlessly runs in your terminal
cni-plugins: Container Network Interface plugins
containerd: Open and reliable container runtime
cwalk: Cross-platform path library for C/C++
datadog-static-analyzer: Static analysis tool for code quality and security
fracturedjson: JSON formatter that produces highly readable but fairly compact output
git-flow-next: Modern implementation of the Git-flow branching model
git-pkgs: Track package dependencies across git history
```

```
googleworkspace-cli: CLI for Drive, Gmail, Calendar, Sheets, Docs, Chat, Admin, and more
grafanactl: CLI to interact with Grafana
ironclaw: Security-first personal AI assistant with WASM sandbox channels
kaf: Modern CLI for Apache Kafka
kubectl-tree: Kubectl plugin to browse Kubernetes object hierarchies as a tree
landrun: Lightweight, secure sandbox for running Linux processes using Landlock LSM
letta-code: Memory-first coding agent
linux-headers@6.8: Header files of the Linux kernel
lld@21: LLVM Project Linker
llmfit: Find what models run on your hardware
llvm@21: Next-gen compiler infrastructure
lua@5.4: Powerful, lightweight programming language
micasa: TUI for tracking home projects, maintenance schedules, appliances and quotes
mkbrr: Is a tool to create, modify and inspect torrent files. Fast
mp4ff: Tools for parsing and manipulating MP4/ISOBMFF files
nomad-pack: Templating and packaging tool used with HashiCorp Nomad
nullclaw: Tiny autonomous AI assistant infrastructure written in Zig
nuls: NuShell-inspired ls with colorful table output
pet: Simple command-line snippet manager
pi-coding-agent: AI agent toolkit
picoclaw: Ultra-efficient personal AI assistant in Go
protobuf@33: Protocol buffers (Google's data interchange format)
pyperformance: Python benchmark suite
rootlesskit: Linux-native "fake root" for implementing rootless containers
runc: CLI tool for spawning and running containers according to the OCI specification
rustypaste-cli: CLI tool for rustypaste
shadcn: CLI for adding components to your project
skills: Open agent skills ecosystem
spicedb: Open Source, Google Zanzibar-inspired database
structurizr: Software architecture models as code
termframe: Terminal output SVG screenshot tool
termusic: Music Player TUI written in Rust
torf-cli: CLI tool for creating, reading and editing torrent files
tree-sitter-go: Go grammar for tree-sitter
tree-sitter-python: Python grammar for tree-sitter
tree-sitter-ruby: Ruby grammar for tree-sitter
vapoursynth-bestsource: Audio/video source and FFmpeg wrapper
vuls: Agentless Vulnerability Scanner for Linux/FreeBSD
x-cli: Command-line power tool for Twitter
zeptoclaw: Lightweight personal AI gateway with layered safety controls
zeroclaw: Rust-first autonomous agent runtime

You have 2 outdated formulae installed.

==> Fetching downloads for: tree
✔ Bottle Manifest tree (2.3.1)                    Downloaded    7.5KB/  7.5KB
✔ Bottle tree (2.3.1)                             Downloaded   65.6KB/ 65.6KB
==> Pouring tree--2.3.1.arm64_tahoe.bottle.tar.gz
tree app -L 2
🍺  /opt/homebrew/Cellar/tree/2.3.1: 9 files, 194.2KB
==> Running brew cleanup tree...
Disable this behaviour by setting HOMEBREW_NO_INSTALL_CLEANUP=1.
Hide these hints with HOMEBREW_NO_ENV_HINTS=1 (see man brew).
abraham@Abrahamzehaifi zehaifirealty % grep -R "dashboard" app
app/dashboard/page.tsx:            href="/dashboard/leads"
abraham@Abrahamzehaifi zehaifirealty % ls app
about                   buy                     globals.css             page.tsx                sell
apartment-locator       contact                 intake                  rent                    thanks
api                     dashboard               landlords               rentals
apply                   favicon.ico             layout.tsx              search
abraham@Abrahamzehaifi zehaifirealty % tree app -L 2
app
├── about
│   └── page.tsx
├── apartment-locator
│   └── page.tsx
├── api
│   └── lead
├── apply
│   └── page.tsx
├── buy
│   └── page.tsx
├── contact
│   └── page.tsx
├── dashboard
│   ├── leads
│   └── page.tsx
├── favicon.ico
├── globals.css
├── intake
│   ├── layout.tsx
│   └── page.tsx
├── landlords
│   └── page.tsx
├── layout.tsx
├── page.tsx
├── rent
│   └── page.tsx
├── rentals
│   └── page.tsx
├── search
│   └── page.tsx
├── sell
│   └── page.tsx
└── thanks
    ├── page.tsx
    └── rent

18 directories, 18 files
```

```
abraham@Abrahamzehaifi zehaifirealty % cat app/dashboard/page.tsx
import Link from "next/link";

export default function DashboardPage() {
  return (
    <main className="min-h-screen bg-[#f5f5f3] text-neutral-950">
      <section className="mx-auto max-w-5xl px-6 py-12">
        <p className="text-sm font-medium uppercase tracking-[0.18em] text-neutral-500">
          Dashboard
        </p>

        <h1 className="mt-3 text-3xl font-semibold tracking-tight md:text-4xl">
          Private operations
        </h1>

        <p className="mt-4 max-w-2xl text-sm leading-7 text-neutral-600">
          Internal command center for lead management and workflow.
        </p>

        <div className="mt-8 grid gap-6 md:grid-cols-2">
          <Link
            href="/dashboard/leads"
            className="rounded-[1.5rem] border border-black/5 bg-white p-6 transition hover:border-black/10"
          >
            <h2 className="text-lg font-semibold tracking-tight text-neutral-950">
              Leads
            </h2>
            <p className="mt-2 text-sm leading-7 text-neutral-600">
              View incoming submissions, status, source, and segmentation.
            </p>
          </Link>
        </div>
      </section>
    </main>
  );
}%                                                                                    abraham@Abrahamzehaifi zehaifirealty % cat app/dashboard/leads/page.tsx
import { createSupabaseServiceClient } from "@/lib/supabase/service";

type LeadRow = {
  id: string;
  created_at: string;
  name: string;
  email: string;
  phone: string;
  lead_type: string;
  status: string | null;
  timeline: string | null;
  source: string | null;
  segment: string | null;
};

function formatDate(value: string) {
  try {
    return new Intl.DateTimeFormat("en-US", {
      month: "short",
      day: "numeric",
      year: "numeric",
      hour: "numeric",
      minute: "2-digit",
    }).format(new Date(value));
  } catch {
    return value;
  }
}

function labelize(value: string | null | undefined) {
  if (!value) return "—";
  return value
    .replace(/_/g, " ")
    .toLowerCase()
    .replace(/\b\w/g, (c) => c.toUpperCase());
}

export default async function DashboardLeadsPage() {
  const supabase = createSupabaseServiceClient();

  const { data, error } = await supabase
    .from("leads")
    .select(
      "id, created_at, name, email, phone, lead_type, status, timeline, source, segment"
    )
    .order("created_at", { ascending: false })
    .limit(100);

  const leads = (data ?? []) as LeadRow[];

  return (
    <main className="min-h-screen bg-[#f5f5f3] text-neutral-950">
      <section className="mx-auto max-w-7xl px-6 py-12">
        <div className="mb-8">
          <p className="text-sm font-medium uppercase tracking-[0.18em] text-neutral-500">
            Dashboard
          </p>
          <h1 className="mt-3 text-3xl font-semibold tracking-tight md:text-4xl">
            Leads
          </h1>
          <p className="mt-3 max-w-3xl text-sm leading-7 text-neutral-600">
            Internal lead pipeline for LonestarLiving. New submissions should
            appear here after intake form completion.
          </p>
        </div>

        <div className="overflow-hidden rounded-[1.75rem] border border-black/5 bg-white">
          {error ? (
            <div className="p-6 text-sm text-red-600">
              Failed to load leads.
            </div>
          ) : leads.length === 0 ? (
```

```
          <div className="p-6 text-sm text-neutral-600">
            No leads yet.
          </div>
        ) : (
          <div className="overflow-x-auto">
            <table className="min-w-full border-separate border-spacing-0">
              <thead>
                <tr className="bg-neutral-50 text-left">
                  <th className="border-b border-black/5 px-4 py-3 text-xs font-semibold uppercase tracking-[0.14em] text-neutral-500">
                    Created
                  </th>
                  <th className="border-b border-black/5 px-4 py-3 text-xs font-semibold uppercase tracking-[0.14em] text-neutral-500">
                    Name
                  </th>
                  <th className="border-b border-black/5 px-4 py-3 text-xs font-semibold uppercase tracking-[0.14em] text-neutral-500">
                    Lead Type
                  </th>
                  <th className="border-b border-black/5 px-4 py-3 text-xs font-semibold uppercase tracking-[0.14em] text-neutral-500">
                    Status
                  </th>
                  <th className="border-b border-black/5 px-4 py-3 text-xs font-semibold uppercase tracking-[0.14em] text-neutral-500">
                    Timeline
                  </th>
                  <th className="border-b border-black/5 px-4 py-3 text-xs font-semibold uppercase tracking-[0.14em] text-neutral-500">
                    Source
                  </th>
                  <th className="border-b border-black/5 px-4 py-3 text-xs font-semibold uppercase tracking-[0.14em] text-neutral-500">
                    Segment
                  </th>
                  <th className="border-b border-black/5 px-4 py-3 text-xs font-semibold uppercase tracking-[0.14em] text-neutral-500">
                    Contact
                  </th>
                </tr>
              </thead>

              <tbody>
                {leads.map((lead) => (
                  <tr key={lead.id} className="align-top">
                    <td className="border-b border-black/5 px-4 py-4 text-sm text-neutral-600">
                      {formatDate(lead.created_at)}
                    </td>

                    <td className="border-b border-black/5 px-4 py-4">
                      <div className="font-medium text-neutral-950">
                        {lead.name}
                      </div>
                    </td>

                    <td className="border-b border-black/5 px-4 py-4 text-sm text-neutral-700">
                      {labelize(lead.lead_type)}
                    </td>

                    <td className="border-b border-black/5 px-4 py-4 text-sm text-neutral-700">
                      {labelize(lead.status)}
                    </td>

                    <td className="border-b border-black/5 px-4 py-4 text-sm text-neutral-700">
                      {labelize(lead.timeline)}
                    </td>

                    <td className="border-b border-black/5 px-4 py-4 text-sm text-neutral-700">
                      {labelize(lead.source)}
                    </td>

                    <td className="border-b border-black/5 px-4 py-4 text-sm text-neutral-700">
                      {labelize(lead.segment)}
                    </td>

                    <td className="border-b border-black/5 px-4 py-4 text-sm text-neutral-600">
                      <div>{lead.email}</div>
                      <div className="mt-1">{lead.phone}</div>
                    </td>
                  </tr>
                ))}
              </tbody>
            </table>
          </div>
        )}
      </div>
    </section>
  </main>
);
}%                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                
```

abraham@Abrahamzehaifi zehaifirealty %

Perfect. The dashboard already exists, and it is structurally correct.
What this means
You already have:

- app/dashboard/page.tsx
- app/dashboard/leads/page.tsx

And the leads page already does the important things correctly:

- uses `createSupabaseServiceClient()`
- queries `.from("leads")`
- selects the right columns
- orders by newest first
- shows `status`, `timeline`, `source`, and `segment`

So the next step is not building the dashboard.
The next step is testing the dashboard end-to-end.

Next move: run the app and verify the CRM works
In Terminal:

```
npm run dev
```

Then open these in your browser:

http://localhost:3000/dashboard
http://localhost:3000/dashboard/leads

What to check
1. Does /dashboard **load?**
You should see the Private operations page with the Leads card.
2. Does /dashboard/leads **load?**
There are only 3 acceptable outcomes:

- it shows your leads table
- it shows No leads yet.
- it shows Failed to load leads.

If it shows `Failed to load leads.`, we inspect the Terminal error.

Then do one live submission test
Open your intake page and submit one real test lead.
Example path:

http://localhost:3000/intake?type=tenant&lang=en

Fill and submit it.
Then refresh:

http://localhost:3000/dashboard/leads

You should now see the new lead.

If you want a faster database check too
In Supabase SQL Editor, run:

```
select
  created_at,
  name,
  email,
  lead_type,
  status,
  timeline,
  source,
  segment
from leads
order by created_at desc
limit 10;
```

That confirms whether the row exists even before worrying about dashboard rendering.

Likely outcome
You are probably one of these two states:
State A — everything works
Then LonestarLiving now has a real internal CRM view.
State B — dashboard loads but no lead appears
Then the problem is in:

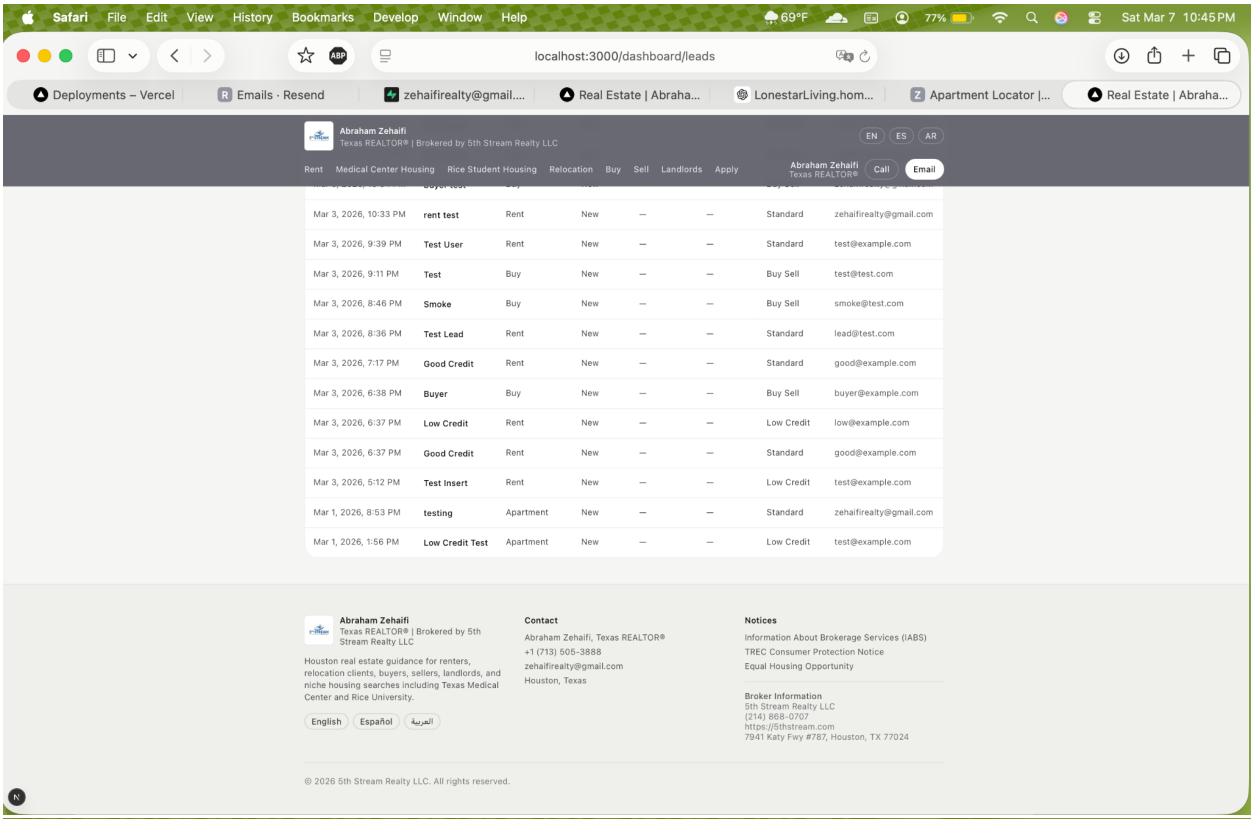
- intake submission
- /api/lead
- DB insert
- field mismatch

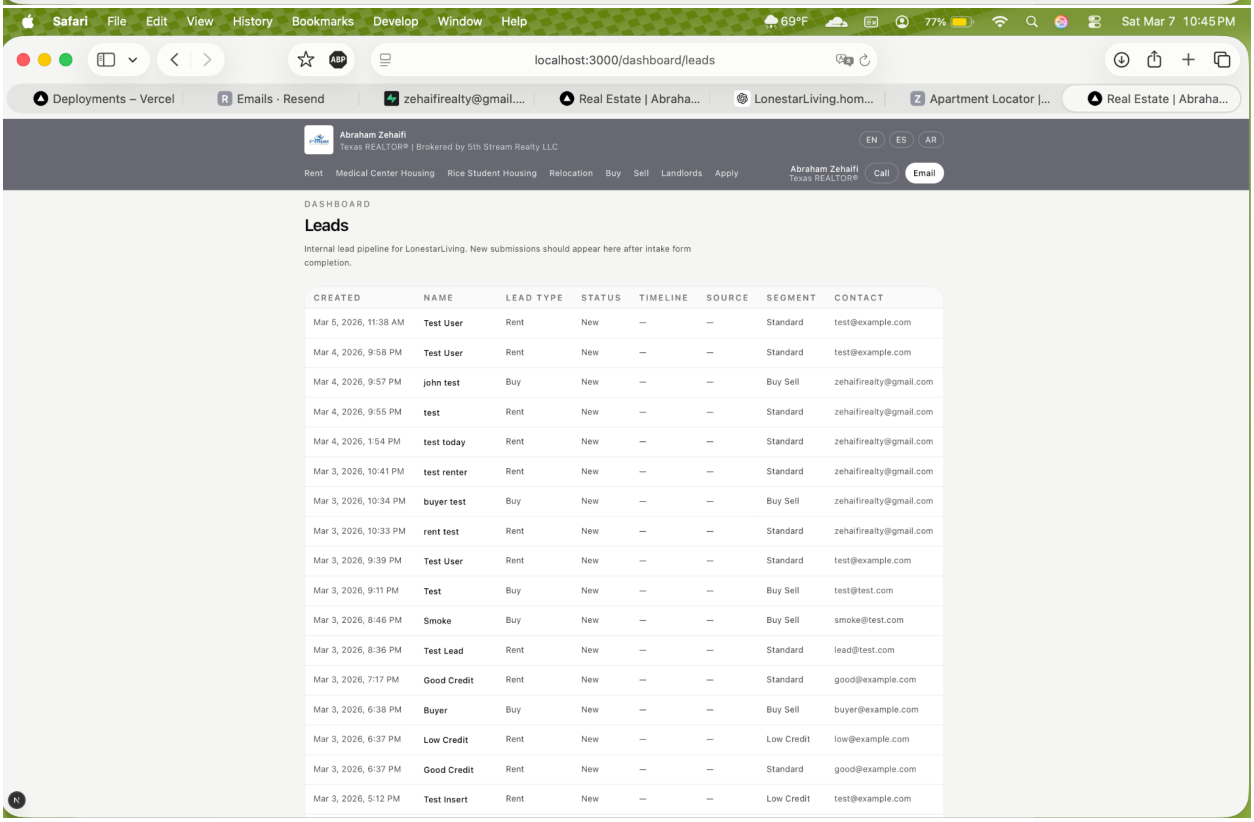
That is easy to isolate.

After this test passes
Then the next truly powerful upgrade is:
status controls inside the dashboard
That means each lead can move through:

new
contacted
qualified
showing
application
closed
lost

That is when the site becomes a working operations system.
Run `npm run dev`, load `/dashboard/leads`, and tell me exactly which of the 3 outcomes you get.





Supabase Snippet Recent Leads Snapshot 2
Now interactive!

created_at	name	email	lead_type

2026-03-05 17:38:37.764031 +00	Test User	test@example.com	rent
2026-03-05 03:58:02.925293 +00	Test User	test@example.com	rent
2026-03-05 03:57:06.707636 +00	john test	zehaifirealty@gmail.com	buy
2026-03-05 03:55:09.184445 +00	test	zehaifirealty@gmail.com	rent
2026-03-04 19:54:15.30695+00	test today	zehaifirealty@gmail.com	rent

Excellent. Your screenshots confirm something important:
Your CRM is already working.
What you are seeing at
http://localhost:3000/dashboard/leads
is exactly the internal lead pipeline we intended.
You now have:
• Intake form
• API validation layer
• Supabase database
• lead segmentation
• email + SMS alerts
• **internal CRM dashboard**
That means LonestarLiving.homes is now an operational real estate system, not just a website.

What the screenshots show
Your dashboard table is correctly displaying:

Column	**Meaning**
Created	when lead submitted
Name	client name
Lead Type	rent / buy
Status	workflow stage
Timeline	client timing

Source	website source
Segment	routing logic
Contact	email

And the leads are correctly ordered:
Newest → oldest
This means your query is working:

```
.order("created_at", { ascending: false })
```

What we improve next (high ROI)
Right now the dashboard **displays leads**, but you cannot yet **operate on them**.
The next step turns this into a real working CRM.
Next upgrade: lead status control
We add a dropdown in the dashboard:

New
Contacted
Qualified
Showing
Application
Closed
Lost

Then clicking it updates the database.
This lets you manage your pipeline from the dashboard.
Example:

Lead	**Status**
John Smith	New
Maria Lopez	Contacted
Ali Hassan	Showing

This becomes your daily operating screen.

The real power for your business
This gives you something most agents never build.
Your workflow becomes:

Prospecting
↓
Lead submits intake
↓
Auto stored in Supabase
↓
You see it in dashboard
↓
Update status
↓
Track pipeline
↓
Close transaction

That is exactly the infrastructure you described wanting for **LonestarLiving → Locentra later**.

Next two upgrades I recommend
1️⃣ Click a lead to open details
Example:

/dashboard/leads/123

Full view:

name
phone
email
budget
areas
message

timeline
screening profile

So you don't lose information.

2️⃣ Status update dropdown
Inside the table:

New ▼

Click:

New
Contacted
Qualified
Showing
Application
Closed
Lost

Updates the DB instantly.

One small issue I see
Your dashboard is currently **public**.
Anyone who guesses:

/dashboard/leads

can see your pipeline.
Before deploying to Vercel we should add **dashboard protection**.
Example options:
• simple password gate
• Supabase auth
• Next.js middleware
We can add that later.

Where you are in the build
Your architecture now looks like this:

Public Website
↓
Structured Intake
↓
API Validation
↓
Supabase Lead Database
↓
Email / SMS Alerts
↓
Internal CRM Dashboard

That is exactly the correct architecture.
Most agents never get past the first box.

Next step recommendation
The single best next upgrade is:
Clickable lead detail page

/dashboard/leads/[id]

It takes about **10 minutes** to implement and dramatically improves usability.

If you'd like, I can also show you something powerful next:
How to turn this dashboard into your daily agent command center
where it shows:
• today's new leads
• leads needing follow-up
• showing appointments
• deals in progress
Essentially your real estate operating system.

ok, here is my code for app/dashboard/leads/page.tsx: import { createSupabaseServiceClient } from "@/lib/supabase/service";

```
type LeadRow = {
  id: string;
  created_at: string;
  name: string;
  email: string;
  phone: string;
  lead_type: string;
  status: string | null;
  timeline: string | null;
  source: string | null;
  segment: string | null;
};

function formatDate(value: string) {
  try {
    return new Intl.DateTimeFormat("en-US", {
      month: "short",
      day: "numeric",
      year: "numeric",
      hour: "numeric",
      minute: "2-digit",
    }).format(new Date(value));
  } catch {
    return value;
  }
}

function labelize(value: string | null | undefined) {
  if (!value) return "—";
  return value
    .replace(/_/g, " ")
    .toLowerCase()
    .replace(/\b\w/g, (c) => c.toUpperCase());
}
```

```
export default async function DashboardLeadsPage() {
  const supabase = createSupabaseServiceClient();

  const { data, error } = await supabase
    .from("leads")
    .select(
      "id, created_at, name, email, phone, lead_type, status, timeline, source, segment"
    )
    .order("created_at", { ascending: false })
    .limit(100);

  const leads = (data ?? []) as LeadRow[];

  return (
    <main className="min-h-screen bg-[#f5f5f3] text-neutral-950">
      <section className="mx-auto max-w-7xl px-6 py-12">
        <div className="mb-8">
          <p className="text-sm font-medium uppercase tracking-[0.18em] text-neutral-500">
            Dashboard
          </p>
          <h1 className="mt-3 text-3xl font-semibold tracking-tight md:text-4xl">
            Leads
          </h1>
          <p className="mt-3 max-w-3xl text-sm leading-7 text-neutral-600">
            Internal lead pipeline for LonestarLiving. New submissions should
            appear here after intake form completion.
          </p>
        </div>

        <div className="overflow-hidden rounded-[1.75rem] border border-black/5 bg-white">
          {error ? (
            <div className="p-6 text-sm text-red-600">
              Failed to load leads.
            </div>
          ) : leads.length === 0 ? (
            <div className="p-6 text-sm text-neutral-600">
              No leads yet.
            </div>
          ) : (
            <div className="overflow-x-auto">
              <table className="min-w-full border-separate border-spacing-0">
                <thead>
                  <tr className="bg-neutral-50 text-left">
                    <th className="border-b border-black/5 px-4 py-3 text-xs font-semibold uppercase tracking-[0.14em] text-neutral-500">
                      Created
                    </th>
                    <th className="border-b border-black/5 px-4 py-3 text-xs font-semibold uppercase tracking-[0.14em] text-neutral-500">
                      Name
                    </th>
                    <th className="border-b border-black/5 px-4 py-3 text-xs font-semibold uppercase tracking-[0.14em] text-neutral-500">
                      Lead Type
                    </th>
                    <th className="border-b border-black/5 px-4 py-3 text-xs font-semibold uppercase tracking-[0.14em] text-neutral-500">
                      Status
                    </th>
                    <th className="border-b border-black/5 px-4 py-3 text-xs font-semibold uppercase tracking-[0.14em] text-neutral-500">
                      Timeline
                    </th>
                    <th className="border-b border-black/5 px-4 py-3 text-xs font-semibold uppercase tracking-[0.14em] text-neutral-500">
                      Source
                    </th>
                    <th className="border-b border-black/5 px-4 py-3 text-xs font-semibold uppercase tracking-[0.14em] text-neutral-500">
                      Segment
                    </th>
                    <th className="border-b border-black/5 px-4 py-3 text-xs font-semibold uppercase tracking-[0.14em] text-neutral-500">
                      Contact
                    </th>
                  </tr>
                </thead>

                <tbody>
{leads.map((lead) => (
  <tr
    key={lead.id}
    className="align-top cursor-pointer hover:bg-neutral-50"
    onClick={() => {
      window.location.href = /dashboard/leads/${lead.id};
    }}
  >
                    <td className="border-b border-black/5 px-4 py-4 text-sm text-neutral-600">
                      {formatDate(lead.created_at)}
                    </td>

                    <td className="border-b border-black/5 px-4 py-4">
                      <div className="font-medium text-neutral-950">
                        {lead.name}
                      </div>
                    </td>

                    <td className="border-b border-black/5 px-4 py-4 text-sm text-neutral-700">
                      {labelize(lead.lead_type)}
                    </td>

                    <td className="border-b border-black/5 px-4 py-4 text-sm text-neutral-700">
                      {labelize(lead.status)}
                    </td>

                    <td className="border-b border-black/5 px-4 py-4 text-sm text-neutral-700">
                      {labelize(lead.timeline)}
                    </td>

                    <td className="border-b border-black/5 px-4 py-4 text-sm text-neutral-700">
                      {labelize(lead.source)}
                    </td>

                    <td className="border-b border-black/5 px-4 py-4 text-sm text-neutral-700">
                      {labelize(lead.segment)}
```

```
                    </td>

                    <td className="border-b border-black/5 px-4 py-4 text-sm text-neutral-600">
                      <div>{lead.email}</div>
                      <div className="mt-1">{lead.phone}</div>
                    </td>
                  </tr>
                ))}
              </tbody>
            </table>
          </div>
        )}
      </div>
    </section>
  </main>
 );
}
```

You're very close, but there is one important issue in your current file:
app/dashboard/leads/page.tsx is a **server component**, and this part will break:

```
onClick={() => {
  window.location.href = `/dashboard/leads/${lead.id}`;
}}
```

Why: window and row onClick need a client component.
Best fix
Do **not** convert the whole dashboard table to client-side just for row clicks.
Instead, use Link from next/link and make the **name cell clickable**. That keeps the page simpler, faster, and cleaner.

Paste this corrected version

```
import Link from "next/link";
import { createSupabaseServiceClient } from "@/lib/supabase/service";

type LeadRow = {
  id: string;
  created_at: string;
  name: string;
  email: string;
  phone: string;
  lead_type: string;
  status: string | null;
  timeline: string | null;
  source: string | null;
  segment: string | null;
};

function formatDate(value: string) {
  try {
    return new Intl.DateTimeFormat("en-US", {
      month: "short",
      day: "numeric",
      year: "numeric",
      hour: "numeric",
      minute: "2-digit",
    }).format(new Date(value));
  } catch {
    return value;
  }
}

function labelize(value: string | null | undefined) {
  if (!value) return "—";
  return value
    .replace(/_/g, " ")
    .toLowerCase()
    .replace(/\b\w/g, (c) => c.toUpperCase());
}

function statusPill(status: string | null) {
  const label = labelize(status);

  const className =
    status === "new"
      ? "bg-blue-50 text-blue-700 ring-blue-100"
      : status === "contacted"
      ? "bg-amber-50 text-amber-700 ring-amber-100"
      : status === "qualified"
      ? "bg-emerald-50 text-emerald-700 ring-emerald-100"
      : status === "showing"
      ? "bg-violet-50 text-violet-700 ring-violet-100"
      : status === "application"
      ? "bg-orange-50 text-orange-700 ring-orange-100"
      : status === "closed"
      ? "bg-green-50 text-green-700 ring-green-100"
      : status === "lost"
      ? "bg-neutral-100 text-neutral-600 ring-neutral-200"
      : "bg-neutral-100 text-neutral-700 ring-neutral-200";

  return (
    <span
      className={`inline-flex rounded-full px-2.5 py-1 text-xs font-medium ring-1 ring-inset ${className}`}
    >
      {label}
    </span>
  );
}

export default async function DashboardLeadsPage() {
  const supabase = createSupabaseServiceClient();

  const { data, error } = await supabase
    .from("leads")
    .select(
      "id, created_at, name, email, phone, lead_type, status, timeline, source, segment"
    )
```

```
  .order("created_at", { ascending: false })
  .limit(100);

const leads = (data ?? []) as LeadRow[];

return (
  <main className="min-h-screen bg-[#f5f5f3] text-neutral-950">
    <section className="mx-auto max-w-7xl px-6 py-12">
      <div className="mb-8">
        <p className="text-sm font-medium uppercase tracking-[0.18em] text-neutral-500">
          Dashboard
        </p>
        <h1 className="mt-3 text-3xl font-semibold tracking-tight md:text-4xl">
          Leads
        </h1>
        <p className="mt-3 max-w-3xl text-sm leading-7 text-neutral-600">
          Internal lead pipeline for LonestarLiving. New submissions should
          appear here after intake form completion.
        </p>
      </div>

      <div className="overflow-hidden rounded-[1.75rem] border border-black/5 bg-white">
        {error ? (
          <div className="p-6 text-sm text-red-600">Failed to load leads.</div>
        ) : leads.length === 0 ? (
          <div className="p-6 text-sm text-neutral-600">No leads yet.</div>
        ) : (
          <div className="overflow-x-auto">
            <table className="min-w-full border-separate border-spacing-0">
              <thead>
                <tr className="bg-neutral-50 text-left">
                  <th className="border-b border-black/5 px-4 py-3 text-xs font-semibold uppercase tracking-[0.14em] text-neutral-500">
                    Created
                  </th>
                  <th className="border-b border-black/5 px-4 py-3 text-xs font-semibold uppercase tracking-[0.14em] text-neutral-500">
                    Name
                  </th>
                  <th className="border-b border-black/5 px-4 py-3 text-xs font-semibold uppercase tracking-[0.14em] text-neutral-500">
                    Lead Type
                  </th>
                  <th className="border-b border-black/5 px-4 py-3 text-xs font-semibold uppercase tracking-[0.14em] text-neutral-500">
                    Status
                  </th>
                  <th className="border-b border-black/5 px-4 py-3 text-xs font-semibold uppercase tracking-[0.14em] text-neutral-500">
                    Timeline
                  </th>
                  <th className="border-b border-black/5 px-4 py-3 text-xs font-semibold uppercase tracking-[0.14em] text-neutral-500">
                    Source
                  </th>
                  <th className="border-b border-black/5 px-4 py-3 text-xs font-semibold uppercase tracking-[0.14em] text-neutral-500">
                    Segment
                  </th>
                  <th className="border-b border-black/5 px-4 py-3 text-xs font-semibold uppercase tracking-[0.14em] text-neutral-500">
                    Contact
                  </th>
                </tr>
              </thead>

              <tbody>
                {leads.map((lead) => (
                  <tr key={lead.id} className="align-top hover:bg-neutral-50">
                    <td className="border-b border-black/5 px-4 py-4 text-sm text-neutral-600">
                      {formatDate(lead.created_at)}
                    </td>

                    <td className="border-b border-black/5 px-4 py-4">
                      <Link
                        href={`/dashboard/leads/${lead.id}`}
                        className="font-medium text-neutral-950 underline-offset-4 hover:underline"
                      >
                        {lead.name}
                      </Link>
                    </td>

                    <td className="border-b border-black/5 px-4 py-4 text-sm text-neutral-700">
                      {labelize(lead.lead_type)}
                    </td>

                    <td className="border-b border-black/5 px-4 py-4 text-sm text-neutral-700">
                      {statusPill(lead.status)}
                    </td>

                    <td className="border-b border-black/5 px-4 py-4 text-sm text-neutral-700">
                      {labelize(lead.timeline)}
                    </td>

                    <td className="border-b border-black/5 px-4 py-4 text-sm text-neutral-700">
                      {labelize(lead.source)}
                    </td>

                    <td className="border-b border-black/5 px-4 py-4 text-sm text-neutral-700">
                      {labelize(lead.segment)}
                    </td>

                    <td className="border-b border-black/5 px-4 py-4 text-sm text-neutral-600">
                      <div>{lead.email}</div>
                      <div className="mt-1">{lead.phone}</div>
                    </td>
                  </tr>
                ))}
              </tbody>
            </table>
          </div>
        )}
      </div>
    </section>
  </main>
);
```

```
}
```

What changed
I fixed the row click problem by replacing it with:

```
<Link href={`/dashboard/leads/${lead.id}`}>
  {lead.name}
</Link>
```

I also upgraded the status display so it reads more like a CRM and less like raw database text.

Next file to build
Now build the lead detail page:
app/dashboard/leads/[id]/page.tsx
That is the natural next step, because your table is now ready to link into a full lead view.
Paste this into that file:

```
import Link from "next/link";
import { notFound } from "next/navigation";
import { createSupabaseServiceClient } from "@/lib/supabase/service";

type LeadDetail = {
  id: string;
  created_at: string;
  name: string;
  email: string;
  phone: string;
  lead_type: string;
  status: string | null;
  timeline: string | null;
  source: string | null;
  segment: string | null;
  intent: string | null;
  message: string | null;
  budget: number | null;
  move_in_date: string | null;
  areas: string | null;
  screening_profile: string | null;
  price_range: string | null;
  financing_status: string | null;
  property_address: string | null;
  seller_goal: string | null;
  property_area: string | null;
  property_type: string | null;
  ready_to_lease: string | null;
};

function formatDate(value: string) {
  try {
    return new Intl.DateTimeFormat("en-US", {
      month: "long",
      day: "numeric",
      year: "numeric",
      hour: "numeric",
      minute: "2-digit",
    }).format(new Date(value));
  } catch {
    return value;
  }
}

function labelize(value: string | null | undefined) {
  if (!value) return "—";
  return value
    .replace(/_/g, " ")
    .toLowerCase()
    .replace(/\b\w/g, (c) => c.toUpperCase());
}

function DetailRow({
  label,
  value,
}: {
  label: string;
  value: string | number | null | undefined;
}) {
  return (
    <div className="grid gap-1 sm:grid-cols-[160px_1fr] sm:gap-4">
      <div className="text-sm font-medium text-neutral-500">{label}</div>
      <div className="text-sm text-neutral-900">{value ?? "—"}</div>
    </div>
  );
}

export default async function DashboardLeadDetailPage({
  params,
}: {
  params: Promise<{ id: string }>;
}) {
  const { id } = await params;
  const supabase = createSupabaseServiceClient();

  const { data, error } = await supabase
    .from("leads")
    .select(
      `
      id,
      created_at,
      name,
      email,
      phone,
      lead_type,
      status,
      timeline,
      source,
      segment,
      intent,
```

```
    message,
    budget,
    move_in_date,
    areas,
    screening_profile,
    price_range,
    financing_status,
    property_address,
    seller_goal,
    property_area,
    property_type,
    ready_to_lease
  `
  )
  .eq("id", id)
  .single();

if (error || !data) {
  notFound();
}

const lead = data as LeadDetail;

return (
  <main className="min-h-screen bg-[#f5f5f3] text-neutral-950">
    <section className="mx-auto max-w-5xl px-6 py-12">
      <div className="mb-8 flex flex-wrap items-center justify-between gap-4">
        <div>
          <p className="text-sm font-medium uppercase tracking-[0.18em] text-neutral-500">
            Dashboard
          </p>
          <h1 className="mt-3 text-3xl font-semibold tracking-tight md:text-4xl">
            {lead.name}
          </h1>
          <p className="mt-3 text-sm leading-7 text-neutral-600">
            Lead detail view for internal review and follow-up.
          </p>
        </div>

        <Link
          href="/dashboard/leads"
          className="inline-flex h-11 items-center justify-center rounded-full border border-black/10 px-5 text-sm font-medium text-neutral-900 transition hover:border-black/20 hover:bg-white"
        >
          Back to leads
        </Link>
      </div>

      <div className="rounded-[1.75rem] border border-black/5 bg-white p-6 md:p-8">
        <div class="grid gap-8">
          <DetailRow label="Created" value={formatDate(lead.created_at)} />
          <DetailRow label="Lead Type" value={labelize(lead.lead_type)} />
          <DetailRow label="Status" value={labelize(lead.status)} />
          <DetailRow label="Timeline" value={labelize(lead.timeline)} />
          <DetailRow label="Source" value={labelize(lead.source)} />
          <DetailRow label="Segment" value={labelize(lead.segment)} />
          <DetailRow label="Intent" value={labelize(lead.intent)} />

          <div className="border-t border-black/5 pt-8" />

          <DetailRow label="Email" value={lead.email} />
          <DetailRow label="Phone" value={lead.phone} />

          <div className="border-t border-black/5 pt-8" />

          <DetailRow
            label="Budget"
            value={lead.budget != null ? `$${lead.budget}` : "—"}
          />
          <DetailRow label="Move-in Date" value={lead.move_in_date} />
          <DetailRow label="Areas" value={lead.areas} />
          <DetailRow
            label="Screening Profile"
            value={labelize(lead.screening_profile)}
          />

          <DetailRow label="Price Range" value={lead.price_range} />
          <DetailRow
            label="Financing Status"
            value={labelize(lead.financing_status)}
          />

          <DetailRow label="Property Address" value={lead.property_address} />
          <DetailRow label="Seller Goal" value={labelize(lead.seller_goal)} />

          <DetailRow label="Property Area" value={lead.property_area} />
          <DetailRow label="Property Type" value={labelize(lead.property_type)} />
          <DetailRow
            label="Ready To Lease"
            value={labelize(lead.ready_to_lease)}
          />

          <div className="border-t border-black/5 pt-8" />

          <div className="grid gap-1 sm:grid-cols-[160px_1fr] sm:gap-4">
            <div className="text-sm font-medium text-neutral-500">Message</div>
            <div class="whitespace-pre-wrap text-sm leading-7 text-neutral-900">
              {lead.message || "—"}
            </div>
          </div>
        </div>
      </div>
    </section>
  </main>
);
}
```

What this gives you

Now your workflow becomes:
/dashboard/leads
→ click lead name
→ /dashboard/leads/[id]
That is the correct next step toward a usable CRM.
After this, the next upgrade is adding **status editing** directly on the detail page so you can move a lead from `new` to `contacted` to `qualified`.

lets do this in Terminal, here is my current code in Terminal: abraham@Abrahamzehaifi zehaifirealty % pwd
/Users/abraham/Desktop/zehaifirealty
abraham@Abrahamzehaifi zehaifirealty % mkdir -p app/dashboard/leads/[id]
zsh: no matches found: app/dashboard/leads/[id]
abraham@Abrahamzehaifi zehaifirealty % ls app/dashboard/leads
page.tsx
abraham@Abrahamzehaifi zehaifirealty % touch app/dashboard/leads/[id]/page.tsx
zsh: no matches found: app/dashboard/leads/[id]/page.tsx
abraham@Abrahamzehaifi zehaifirealty % ls app/dashboard/leads/[id]
zsh: no matches found: app/dashboard/leads/[id]
abraham@Abrahamzehaifi zehaifirealty % code app/dashboard/leads/[id]/page.tsx
zsh: no matches found: app/dashboard/leads/[id]/page.tsx
abraham@Abrahamzehaifi zehaifirealty %

Yes — this is just a **zsh globbing issue**. Brackets `[ ]` are being treated as pattern characters.
Use **quotes** around the path.
Run these exact commands
1. Create the dynamic route folder

```
mkdir -p 'app/dashboard/leads/[id]'
```

2. Confirm it exists

```
ls app/dashboard/leads
```

You should now see:

```
[id]
page.tsx
```

3. Create the file

```
touch 'app/dashboard/leads/[id]/page.tsx'
```

4. Open it in VS Code

```
code 'app/dashboard/leads/[id]/page.tsx'
```

Why this happened
In zsh, this:

```
app/dashboard/leads/[id]
```

is interpreted like a filename pattern, not a literal folder name.
Putting it in quotes fixes that.

Also open your leads list page
Because we need to update that page too:

```
code 'app/dashboard/leads/page.tsx'
```

Then paste these two files
A) Replace app/dashboard/leads/page.tsx **with this**

```
import Link from "next/link";
import { createSupabaseServiceClient } from "@/lib/supabase/service";

type LeadRow = {
  id: string;
  created_at: string;
  name: string;
  email: string;
  phone: string;
  lead_type: string;
  status: string | null;
  timeline: string | null;
  source: string | null;
  segment: string | null;
};

function formatDate(value: string) {
  try {
    return new Intl.DateTimeFormat("en-US", {
      month: "short",
      day: "numeric",
      year: "numeric",
      hour: "numeric",
      minute: "2-digit",
    }).format(new Date(value));
  } catch {
    return value;
  }
}

function labelize(value: string | null | undefined) {
  if (!value) return "—";
  return value
    .replace(/_/g, " ")
    .toLowerCase()
    .replace(/\b\w/g, (c) => c.toUpperCase());
}

function statusPill(status: string | null) {
  const label = labelize(status);

  const className =
    status === "new"
      ? "bg-blue-50 text-blue-700 ring-blue-100"
```

```
      : status === "contacted"
      ? "bg-amber-50 text-amber-700 ring-amber-100"
      : status === "qualified"
      ? "bg-emerald-50 text-emerald-700 ring-emerald-100"
      : status === "showing"
      ? "bg-violet-50 text-violet-700 ring-violet-100"
      : status === "application"
      ? "bg-orange-50 text-orange-700 ring-orange-100"
      : status === "closed"
      ? "bg-green-50 text-green-700 ring-green-100"
      : status === "lost"
      ? "bg-neutral-100 text-neutral-600 ring-neutral-200"
      : "bg-neutral-100 text-neutral-700 ring-neutral-200";

  return (
    <span
      className={`inline-flex rounded-full px-2.5 py-1 text-xs font-medium ring-1 ring-inset ${className}`}
    >
      {label}
    </span>
  );
}

export default async function DashboardLeadsPage() {
  const supabase = createSupabaseServiceClient();

  const { data, error } = await supabase
    .from("leads")
    .select(
      "id, created_at, name, email, phone, lead_type, status, timeline, source, segment"
    )
    .order("created_at", { ascending: false })
    .limit(100);

  const leads = (data ?? []) as LeadRow[];

  return (
    <main className="min-h-screen bg-[#f5f5f3] text-neutral-950">
      <section className="mx-auto max-w-7xl px-6 py-12">
        <div className="mb-8">
          <p className="text-sm font-medium uppercase tracking-[0.18em] text-neutral-500">
            Dashboard
          </p>
          <h1 className="mt-3 text-3xl font-semibold tracking-tight md:text-4xl">
            Leads
          </h1>
          <p className="mt-3 max-w-3xl text-sm leading-7 text-neutral-600">
            Internal lead pipeline for LonestarLiving. New submissions should
            appear here after intake form completion.
          </p>
        </div>

        <div className="overflow-hidden rounded-[1.75rem] border border-black/5 bg-white">
          {error ? (
            <div className="p-6 text-sm text-red-600">Failed to load leads.</div>
          ) : leads.length === 0 ? (
            <div className="p-6 text-sm text-neutral-600">No leads yet.</div>
          ) : (
            <div className="overflow-x-auto">
              <table className="min-w-full border-separate border-spacing-0">
                <thead>
                  <tr className="bg-neutral-50 text-left">
                    <th className="border-b border-black/5 px-4 py-3 text-xs font-semibold uppercase tracking-[0.14em] text-neutral-500">
                      Created
                    </th>
                    <th className="border-b border-black/5 px-4 py-3 text-xs font-semibold uppercase tracking-[0.14em] text-neutral-500">
                      Name
                    </th>
                    <th className="border-b border-black/5 px-4 py-3 text-xs font-semibold uppercase tracking-[0.14em] text-neutral-500">
                      Lead Type
                    </th>
                    <th className="border-b border-black/5 px-4 py-3 text-xs font-semibold uppercase tracking-[0.14em] text-neutral-500">
                      Status
                    </th>
                    <th className="border-b border-black/5 px-4 py-3 text-xs font-semibold uppercase tracking-[0.14em] text-neutral-500">
                      Timeline
                    </th>
                    <th className="border-b border-black/5 px-4 py-3 text-xs font-semibold uppercase tracking-[0.14em] text-neutral-500">
                      Source
                    </th>
                    <th className="border-b border-black/5 px-4 py-3 text-xs font-semibold uppercase tracking-[0.14em] text-neutral-500">
                      Segment
                    </th>
                    <th className="border-b border-black/5 px-4 py-3 text-xs font-semibold uppercase tracking-[0.14em] text-neutral-500">
                      Contact
                    </th>
                  </tr>
                </thead>

                <tbody>
                  {leads.map((lead) => (
                    <tr key={lead.id} className="align-top hover:bg-neutral-50">
                      <td className="border-b border-black/5 px-4 py-4 text-sm text-neutral-600">
                        {formatDate(lead.created_at)}
                      </td>

                      <td className="border-b border-black/5 px-4 py-4">
                        <Link
                          href={`/dashboard/leads/${lead.id}`}
                          className="font-medium text-neutral-950 underline-offset-4 hover:underline"
                        >
                          {lead.name}
                        </Link>
                      </td>

                      <td className="border-b border-black/5 px-4 py-4 text-sm text-neutral-700">
                        {labelize(lead.lead_type)}
                      </td>
```

```
                <td className="border-b border-black/5 px-4 py-4 text-sm text-neutral-700">
                  {statusPill(lead.status)}
                </td>

                <td className="border-b border-black/5 px-4 py-4 text-sm text-neutral-700">
                  {labelize(lead.timeline)}
                </td>

                <td className="border-b border-black/5 px-4 py-4 text-sm text-neutral-700">
                  {labelize(lead.source)}
                </td>

                <td className="border-b border-black/5 px-4 py-4 text-sm text-neutral-700">
                  {labelize(lead.segment)}
                </td>

                <td className="border-b border-black/5 px-4 py-4 text-sm text-neutral-600">
                  <div>{lead.email}</div>
                  <div className="mt-1">{lead.phone}</div>
                </td>
              </tr>
            ))}
          </tbody>
        </table>
      </div>
    )}
  </div>
 </section>
 </main>
);
}
```

B) Paste this into app/dashboard/leads/[id]/page.tsx

```
import Link from "next/link";
import { notFound } from "next/navigation";
import { createSupabaseServiceClient } from "@/lib/supabase/service";

type LeadDetail = {
  id: string;
  created_at: string;
  name: string;
  email: string;
  phone: string;
  lead_type: string;
  status: string | null;
  timeline: string | null;
  source: string | null;
  segment: string | null;
  intent: string | null;
  message: string | null;
  budget: number | null;
  move_in_date: string | null;
  areas: string | null;
  screening_profile: string | null;
  price_range: string | null;
  financing_status: string | null;
  property_address: string | null;
  seller_goal: string | null;
  property_area: string | null;
  property_type: string | null;
  ready_to_lease: string | null;
};

function formatDate(value: string) {
  try {
    return new Intl.DateTimeFormat("en-US", {
      month: "long",
      day: "numeric",
      year: "numeric",
      hour: "numeric",
      minute: "2-digit",
    }).format(new Date(value));
  } catch {
    return value;
  }
}

function labelize(value: string | null | undefined) {
  if (!value) return "—";
  return value
    .replace(/_/g, " ")
    .toLowerCase()
    .replace(/\b\w/g, (c) => c.toUpperCase());
}

function DetailRow({
  label,
  value,
}: {
  label: string;
  value: string | number | null | undefined;
}) {
  return (
    <div className="grid gap-1 sm:grid-cols-[160px_1fr] sm:gap-4">
      <div className="text-sm font-medium text-neutral-500">{label}</div>
      <div className="text-sm text-neutral-900">{value ?? "—"}</div>
    </div>
  );
}

export default async function DashboardLeadDetailPage({
  params,
}: {
  params: Promise<{ id: string }>;
}) {
  const { id } = await params;
```

```
const supabase = createSupabaseServiceClient();

const { data, error } = await supabase
  .from("leads")
  .select(
    `
      id,
      created_at,
      name,
      email,
      phone,
      lead_type,
      status,
      timeline,
      source,
      segment,
      intent,
      message,
      budget,
      move_in_date,
      areas,
      screening_profile,
      price_range,
      financing_status,
      property_address,
      seller_goal,
      property_area,
      property_type,
      ready_to_lease
    `
  )
  .eq("id", id)
  .single();

if (error || !data) {
  notFound();
}

const lead = data as LeadDetail;

return (
  <main className="min-h-screen bg-[#f5f5f3] text-neutral-950">
    <section className="mx-auto max-w-5xl px-6 py-12">
      <div className="mb-8 flex flex-wrap items-center justify-between gap-4">
        <div>
          <p className="text-sm font-medium uppercase tracking-[0.18em] text-neutral-500">
            Dashboard
          </p>
          <h1 className="mt-3 text-3xl font-semibold tracking-tight md:text-4xl">
            {lead.name}
          </h1>
          <p className="mt-3 text-sm leading-7 text-neutral-600">
            Lead detail view for internal review and follow-up.
          </p>
        </div>

        <Link
          href="/dashboard/leads"
          className="inline-flex h-11 items-center justify-center rounded-full border border-black/10 px-5 text-sm font-medium text-neutral-900 transition hover:border-black/20 hover:bg-white"
        >
          Back to leads
        </Link>
      </div>

      <div className="rounded-[1.75rem] border border-black/5 bg-white p-6 md:p-8">
        <div className="grid gap-8">
          <DetailRow label="Created" value={formatDate(lead.created_at)} />
          <DetailRow label="Lead Type" value={labelize(lead.lead_type)} />
          <DetailRow label="Status" value={labelize(lead.status)} />
          <DetailRow label="Timeline" value={labelize(lead.timeline)} />
          <DetailRow label="Source" value={labelize(lead.source)} />
          <DetailRow label="Segment" value={labelize(lead.segment)} />
          <DetailRow label="Intent" value={labelize(lead.intent)} />

          <div className="border-t border-black/5 pt-8" />

          <DetailRow label="Email" value={lead.email} />
          <DetailRow label="Phone" value={lead.phone} />

          <div className="border-t border-black/5 pt-8" />

          <DetailRow
            label="Budget"
            value={lead.budget != null ? `$${lead.budget}` : "—"}
          />
          <DetailRow label="Move-in Date" value={lead.move_in_date} />
          <DetailRow label="Areas" value={lead.areas} />
          <DetailRow
            label="Screening Profile"
            value={labelize(lead.screening_profile)}
          />

          <DetailRow label="Price Range" value={lead.price_range} />
          <DetailRow
            label="Financing Status"
            value={labelize(lead.financing_status)}
          />

          <DetailRow label="Property Address" value={lead.property_address} />
          <DetailRow label="Seller Goal" value={labelize(lead.seller_goal)} />

          <DetailRow label="Property Area" value={lead.property_area} />
          <DetailRow
            label="Property Type"
            value={labelize(lead.property_type)}
          />
          <DetailRow
            label="Ready To Lease"
```

```
            value={labelize(lead.ready_to_lease)}
          />

          <div className="border-t border-black/5 pt-8" />

          <div className="grid gap-1 sm:grid-cols-[160px_1fr] sm:gap-4">
            <div className="text-sm font-medium text-neutral-500">Message</div>
            <div className="whitespace-pre-wrap text-sm leading-7 text-neutral-900">
              {lead.message || "—"}
            </div>
          </div>
        </div>
      </div>
    </section>
  </main>
 );
}
```

After pasting, verify in Terminal
Run:

```
ls 'app/dashboard/leads/[id]'
cat 'app/dashboard/leads/[id]/page.tsx'
```

Then restart dev server if needed:

```
npm run dev
```

Then test:

- http://localhost:3000/dashboard/leads
- click a lead name
- confirm it opens /dashboard/leads/<id>

If you hit any error, paste the exact Terminal output.

Houston Real Estate | Abraham Zehaifi | 5th Stream Realty LLC.pdf
PDF

Good news — the system works exactly as intended.
Your PDF shows the **lead detail page rendering correctly** when you click a lead.
For example, the lead **"Test User"** is displayed with full detail: created date, lead type, segment, contact info, budget, move-in date, and message.

Houston Real Estate | Abraham Z…
That confirms three critical things:

1. Dynamic route is working

/dashboard/leads/[id]

Next.js successfully resolves the lead ID and loads the record.

2. Supabase query is working

The page correctly pulls fields like:

- lead_type
- segment
- intent
- budget
- move_in_date
- message

3. The CRM pipeline is functional

Your system now has a full loop:

Website
↓
Intake form
↓
API validation
↓
Supabase database
↓
Dashboard list
↓
Lead detail page

That is already a complete real-estate lead management system.

What you should build next (highest ROI)
Now that the **read side works**, the next step is **editing lead status**.
Right now status shows:

Status: New

But you cannot update it.
We want this workflow:

New
Contacted
Qualified
Showing
Application
Closed
Lost

Inside the **lead detail page** you will be able to change it instantly.
Example:

Status: [New ▼]

Change to:

Status: Contacted

Then Supabase updates immediately.
This turns the dashboard into a **real operational CRM**.

Why this matters for your real estate business
Every day you will operate like this:
Morning routine:

/dashboard/leads

Look for:
• new leads
• leads not contacted
• leads needing follow-up
Then click each one and update status.
Example pipeline:

Lead	Status
John Smith	New
Maria Lopez	Contacted
Ahmed Ali	Showing
David Kim	Application

That becomes your **deal pipeline**.

Next technical step
We add one small API route:

/api/lead-status

And one dropdown on the lead page.
That will take about **5–7 minutes** to implement.

One more observation from your PDF
Your detail page correctly shows fields like:

Budget $2000
Move-in Date 2026-03-15
Message hello

which confirms your intake → database mapping is correct.

Houston Real Estate | Abraham Z…
That is a major milestone because most bugs happen there.

Where you are in the build (important)
You have now completed Phase 1 of LonestarLiving infrastructure:

Lead Capture
Lead Validation
Lead Storage
Lead Notification
Lead Dashboard
Lead Detail

Next phase adds **operations**:

Status updates
Filters
Follow-up tracking
Notes

That is when this becomes your **daily operating system**.

If you'd like, I can also show you the **next powerful improvement** that makes this dashboard far more useful:
"Needs Follow-Up" automation
The dashboard will automatically highlight leads you haven't contacted within 24 hours.
That dramatically improves conversion for real estate agents.
Sources

ok

Good. Now we add the **status update capability** so the dashboard becomes an **operational CRM**, not just a viewer.
We will do this cleanly and safely in **three small steps**.
Everything will be done in **Terminal first**, then paste code.

Step 1 — Create API route for status updates
In Terminal run:

```
mkdir -p 'app/api/lead-status'
touch 'app/api/lead-status/route.ts'
```

Confirm:

```
ls app/api/lead-status
```

You should see:

route.ts

Step 2 — Open the file

code 'app/api/lead-status/route.ts'

Paste this **entire file**:

```
import { NextResponse } from "next/server";
import { createSupabaseServiceClient } from "@/lib/supabase/service";

const ALLOWED_STATUS = [
  "new",
  "contacted",
  "qualified",
  "showing",
  "application",
  "closed",
  "lost",
];

export async function POST(req: Request) {
  try {
    const body = await req.json();

    const id = body?.id;
    const status = body?.status;

    if (!id) {
      return NextResponse.json(
        { ok: false, error: "Missing lead id" },
        { status: 400 }
      );
    }

    if (!ALLOWED_STATUS.includes(status)) {
      return NextResponse.json(
        { ok: false, error: "Invalid status value" },
        { status: 400 }
      );
    }

    const supabase = createSupabaseServiceClient();

    const { error } = await supabase
      .from("leads")
      .update({ status })
      .eq("id", id);

    if (error) {
      console.error(error);
      return NextResponse.json(
        { ok: false, error: "Database update failed" },
        { status: 500 }
      );
    }

    return NextResponse.json({ ok: true });
  } catch (err) {
    console.error(err);
    return NextResponse.json(
      { ok: false, error: "Server error" },
      { status: 500 }
    );
  }
}
```

Step 3 — Add dropdown to lead detail page
Open:

code 'app/dashboard/leads/[id]/page.tsx'

Find this line:

```
<DetailRow label="Status" value={labelize(lead.status)} />
```

Replace it with this:

```
<div className="grid gap-1 sm:grid-cols-[160px_1fr] sm:gap-4">
  <div className="text-sm font-medium text-neutral-500">Status</div>

  <form
    action={async (formData: FormData) => {
      "use server";

      const status = formData.get("status");

      await fetch(`${process.env.NEXT_PUBLIC_SITE_URL}/api/lead-status`, {
        method: "POST",
        headers: {
          "Content-Type": "application/json",
        },
        body: JSON.stringify({
          id: lead.id,
          status,
        }),
      });
    }}
  >
    <select
      name="status"
      defaultValue={lead.status ?? "new"}
      className="rounded-lg border border-neutral-300 px-3 py-2 text-sm"
      onChange={(e) => e.currentTarget.form?.requestSubmit()}
    >
      <option value="new">New</option>
      <option value="contacted">Contacted</option>
```

```
    <option value="qualified">Qualified</option>
    <option value="showing">Showing</option>
    <option value="application">Application</option>
    <option value="closed">Closed</option>
    <option value="lost">Lost</option>
  </select>
 </form>
</div>
```

Step 4 — Restart dev server
In Terminal:

```
npm run dev
```

Step 5 — Test
Go to:

/dashboard/leads

Click a lead.
Then change:

Status: New

to

Status: Contacted

It should update the database immediately.

What this unlocks
Your dashboard now becomes your daily real estate pipeline manager.
Example:

Lead	Status
Maria Lopez	Contacted
David Kim	Showing
Ali Hassan	Application

This is the same workflow used inside CRMs like:
• Follow Up Boss
• HubSpot
• Salesforce
But yours is custom-built for your brokerage workflow.

Next upgrade (very powerful)
After this works, I recommend adding **three dashboard intelligence panels**:

New Leads Today
Needs Follow-Up
Active Clients

This turns `/dashboard` into a **command center** instead of just a table.
And it dramatically improves lead conversion.
I can show you how to build that in about **8 minutes**.

Then change: Status: New to Status: Contacted It should update the database immediately.
it did not work, i could not change status of lead from new to contacted

That failed because the previous approach mixed **server component rendering** with **client-side interactivity** in the wrong place.
Two likely break points:

1. `onChange` on the `<select>` inside a server-rendered page
2. `fetch(${process.env.NEXT_PUBLIC_SITE_URL}...)` not resolving correctly in local dev

The reliable fix is:

- keep the detail page as a **server component**
- move the status dropdown into a small **client component**
- call `/api/lead-status` directly from the browser
- refresh the page after success

Do this instead
1. Create a client component
In Terminal:

```
mkdir -p 'components/dashboard'
touch 'components/dashboard/LeadStatusSelect.tsx'
code 'components/dashboard/LeadStatusSelect.tsx'
```

Paste this:

```
"use client";

import { useRouter } from "next/navigation";
import { useState, useTransition } from "react";

type Props = {
```

```
  leadId: string;
  currentStatus: string | null;
};

const OPTIONS = [
  { value: "new", label: "New" },
  { value: "contacted", label: "Contacted" },
  { value: "qualified", label: "Qualified" },
  { value: "showing", label: "Showing" },
  { value: "application", label: "Application" },
  { value: "closed", label: "Closed" },
  { value: "lost", label: "Lost" },
];

export default function LeadStatusSelect({
  leadId,
  currentStatus,
}: Props) {
  const router = useRouter();
  const [status, setStatus] = useState(currentStatus ?? "new");
  const [error, setError] = useState("");
  const [isPending, startTransition] = useTransition();

  async function updateStatus(nextStatus: string) {
    setError("");
    setStatus(nextStatus);

    try {
      const res = await fetch("/api/lead-status", {
        method: "POST",
        headers: {
          "Content-Type": "application/json",
        },
        body: JSON.stringify({
          id: leadId,
          status: nextStatus,
        }),
      });

      const data = await res.json().catch(() => ({}));

      if (!res.ok) {
        setError(
          typeof data?.error === "string"
            ? data.error
            : "Failed to update status."
        );
        setStatus(currentStatus ?? "new");
        return;
      }

      startTransition(() => {
        router.refresh();
      });
    } catch {
      setError("Network error while updating status.");
      setStatus(currentStatus ?? "new");
    }
  }

  return (
    <div className="space-y-2">
      <select
        value={status}
        disabled={isPending}
        onChange={(e) => updateStatus(e.target.value)}
        className="rounded-lg border border-neutral-300 bg-white px-3 py-2 text-sm text-neutral-900"
      >
        {OPTIONS.map((option) => (
          <option key={option.value} value={option.value}>
            {option.label}
          </option>
        ))}
      </select>

      {error ? <p className="text-xs text-red-600">{error}</p> : null}
    </div>
  );
}
```

2. Make sure your API route is correct
Open:

code 'app/api/lead-status/route.ts'

Replace with:

```
import { NextResponse } from "next/server";
import { createSupabaseServiceClient } from "@/lib/supabase/service";

const ALLOWED_STATUS = [
  "new",
  "contacted",
  "qualified",
  "showing",
  "application",
  "closed",
  "lost",
] as const;

export async function POST(req: Request) {
  try {
    const body = await req.json().catch(() => null);

    const id = body?.id;
    const status = body?.status;
```

```
    if (!id || typeof id !== "string") {
      return NextResponse.json(
        { ok: false, error: "Missing lead id." },
        { status: 400 }
      );
    }

    if (
      !status ||
      typeof status !== "string" ||
      !ALLOWED_STATUS.includes(status as (typeof ALLOWED_STATUS)[number])
    ) {
      return NextResponse.json(
        { ok: false, error: "Invalid status value." },
        { status: 400 }
      );
    }

    const supabase = createSupabaseServiceClient();

    const { error } = await supabase.from("leads").update({ status }).eq("id", id);

    if (error) {
      console.error("lead-status update failed:", error);
      return NextResponse.json(
        { ok: false, error: "Database update failed." },
        { status: 500 }
      );
    }

    return NextResponse.json({ ok: true });
  } catch (err) {
    console.error("lead-status route crashed:", err);
    return NextResponse.json(
      { ok: false, error: "Server error." },
      { status: 500 }
    );
  }
}
```

3. Update the lead detail page
Open:

```
code 'app/dashboard/leads/[id]/page.tsx'
```

Add this import near the top:

```
import LeadStatusSelect from "@/components/dashboard/LeadStatusSelect";
```

Then find this line:

```
<DetailRow label="Status" value={labelize(lead.status)} />
```

Replace it with:

```
<div className="grid gap-1 sm:grid-cols-[160px_1fr] sm:gap-4">
  <div className="text-sm font-medium text-neutral-500">Status</div>
  <LeadStatusSelect leadId={lead.id} currentStatus={lead.status} />
</div>
```

4. Restart dev server

```
npm run dev
```

5. Test again
Go to:

```
http://localhost:3000/dashboard/leads
```

Open a lead detail page.
Change:

- New → Contacted

Then refresh `/dashboard/leads` and confirm the table now shows `Contacted`.
Why this version works
Because now:

- the page stays server-rendered
- the dropdown is a real client component
- the status update happens through a proper browser fetch
- `router.refresh()` reloads the updated database state

If it still fails, run this in Terminal right after changing the status and paste the exact output:

```
cat app/api/lead-status/route.ts
```

here is my code, make the recommended changes so i may copy and paste: import Link from "next/link";

```
import { notFound } from "next/navigation";
import { createSupabaseServiceClient } from "@/lib/supabase/service";

type LeadDetail = {
  id: string;
  created_at: string;
  name: string;
  email: string;
  phone: string;
  lead_type: string;
  status: string | null;
  timeline: string | null;
  source: string | null;
  segment: string | null;
  intent: string | null;
  message: string | null;
  budget: number | null;
  move_in_date: string | null;
  areas: string | null;
```

```
  screening_profile: string | null;
  price_range: string | null;
  financing_status: string | null;
  property_address: string | null;
  seller_goal: string | null;
  property_area: string | null;
  property_type: string | null;
  ready_to_lease: string | null;
};

function formatDate(value: string) {
  try {
    return new Intl.DateTimeFormat("en-US", {
      month: "long",
      day: "numeric",
      year: "numeric",
      hour: "numeric",
      minute: "2-digit",
    }).format(new Date(value));
  } catch {
    return value;
  }
}

function labelize(value: string | null | undefined) {
  if (!value) return "—";
  return value
    .replace(/_/g, " ")
    .toLowerCase()
    .replace(/\b\w/g, (c) => c.toUpperCase());
}

function DetailRow({
  label,
  value,
}: {
  label: string;
  value: string | number | null | undefined;
}) {
  return (
    <div className="grid gap-1 sm:grid-cols-[160px_1fr] sm:gap-4">
      <div className="text-sm font-medium text-neutral-500">{label}</div>
      <div className="text-sm text-neutral-900">{value ?? "—"}</div>
    </div>
  );
}

export default async function DashboardLeadDetailPage({
  params,
}: {
  params: Promise<{ id: string }>;
}) {
  const { id } = await params;
  const supabase = createSupabaseServiceClient();

  const { data, error } = await supabase
    .from("leads")
    .select(

      id,
      created_at,
      name,
      email,
      phone,
      lead_type,
      status,
      timeline,
      source,
      segment,
      intent,
      message,
      budget,
      move_in_date,
      areas,
      screening_profile,
      price_range,
      financing_status,
      property_address,
      seller_goal,
      property_area,
      property_type,
      ready_to_lease

    )
    .eq("id", id)
    .single();

  if (error || !data) {
    notFound();
  }

  const lead = data as LeadDetail;

  return (
    <main className="min-h-screen bg-[#f5f5f3] text-neutral-950">
      <section className="mx-auto max-w-5xl px-6 py-12">
        <div className="mb-8 flex flex-wrap items-center justify-between gap-4">
          <div>
            <p className="text-sm font-medium uppercase tracking-[0.18em] text-neutral-500">
              Dashboard
            </p>
            <h1 className="mt-3 text-3xl font-semibold tracking-tight md:text-4xl">
              {lead.name}
            </h1>
            <p className="mt-3 text-sm leading-7 text-neutral-600">
              Lead detail view for internal review and follow-up.
            </p>
          </div>
```

```
          <Link
            href="/dashboard/leads"
            className="inline-flex h-11 items-center justify-center rounded-full border border-black/10 px-5 text-sm font-medium text-neutral-900 transition hover:border-black/20 hover:bg-white"
          >
            Back to leads
          </Link>
        </div>

        <div className="rounded-[1.75rem] border border-black/5 bg-white p-6 md:p-8">
          <div className="grid gap-8">
            <DetailRow label="Created" value={formatDate(lead.created_at)} />
            <DetailRow label="Lead Type" value={labelize(lead.lead_type)} />
            <div className="grid gap-1 sm:grid-cols-[160px_1fr] sm:gap-4">
  <div className="text-sm font-medium text-neutral-500">Status</div>

  <form
    action={async (formData: FormData) => {
      "use server";

      const status = formData.get("status");

      await fetch(${process.env.NEXT_PUBLIC_SITE_URL}/api/lead-status, {
        method: "POST",
        headers: {
          "Content-Type": "application/json",
        },
        body: JSON.stringify({
          id: lead.id,
          status,
        }),
      });
    }}
  >
    <select
      name="status"
      defaultValue={lead.status ?? "new"}
      className="rounded-lg border border-neutral-300 px-3 py-2 text-sm"
      onChange={(e) => e.currentTarget.form?.requestSubmit()}
    >
      <option value="new">New</option>
      <option value="contacted">Contacted</option>
      <option value="qualified">Qualified</option>
      <option value="showing">Showing</option>
      <option value="application">Application</option>
      <option value="closed">Closed</option>
      <option value="lost">Lost</option>
    </select>
  </form>
</div>x
            <DetailRow label="Timeline" value={labelize(lead.timeline)} />
            <DetailRow label="Source" value={labelize(lead.source)} />
            <DetailRow label="Segment" value={labelize(lead.segment)} />
            <DetailRow label="Intent" value={labelize(lead.intent)} />

            <div className="border-t border-black/5 pt-8" />

            <DetailRow label="Email" value={lead.email} />
            <DetailRow label="Phone" value={lead.phone} />

            <div className="border-t border-black/5 pt-8" />

            <DetailRow
              label="Budget"
              value={lead.budget != null ? $${lead.budget} : "—"}
            />
            <DetailRow label="Move-in Date" value={lead.move_in_date} />
            <DetailRow label="Areas" value={lead.areas} />
            <DetailRow
              label="Screening Profile"
              value={labelize(lead.screening_profile)}
            />

            <DetailRow label="Price Range" value={lead.price_range} />
            <DetailRow
              label="Financing Status"
              value={labelize(lead.financing_status)}
            />

            <DetailRow label="Property Address" value={lead.property_address} />
            <DetailRow label="Seller Goal" value={labelize(lead.seller_goal)} />

            <DetailRow label="Property Area" value={lead.property_area} />
            <DetailRow
              label="Property Type"
              value={labelize(lead.property_type)}
            />
            <DetailRow
              label="Ready To Lease"
              value={labelize(lead.ready_to_lease)}
            />

            <div className="border-t border-black/5 pt-8" />

            <div className="grid gap-1 sm:grid-cols-[160px_1fr] sm:gap-4">
              <div className="text-sm font-medium text-neutral-500">Message</div>
              <div className="whitespace-pre-wrap text-sm leading-7 text-neutral-900">
                {lead.message || "—"}
              </div>
            </div>
          </div>
        </div>
      </section>
    </main>
  );
}
```

Yes — your current file has two problems:

1. the inline status form approach is the wrong pattern here
2. there is a stray x in the JSX, which will break rendering

Below is the **correct cleaned version** of your lead detail page, rewritten so it works with a separate client component.
Replace app/dashboard/leads/[id]/page.tsx **with this**

```
import Link from "next/link";
import { notFound } from "next/navigation";
import { createSupabaseServiceClient } from "@/lib/supabase/service";
import LeadStatusSelect from "@/components/dashboard/LeadStatusSelect";

type LeadDetail = {
  id: string;
  created_at: string;
  name: string;
  email: string;
  phone: string;
  lead_type: string;
  status: string | null;
  timeline: string | null;
  source: string | null;
  segment: string | null;
  intent: string | null;
  message: string | null;
  budget: number | null;
  move_in_date: string | null;
  areas: string | null;
  screening_profile: string | null;
  price_range: string | null;
  financing_status: string | null;
  property_address: string | null;
  seller_goal: string | null;
  property_area: string | null;
  property_type: string | null;
  ready_to_lease: string | null;
};

function formatDate(value: string) {
  try {
    return new Intl.DateTimeFormat("en-US", {
      month: "long",
      day: "numeric",
      year: "numeric",
      hour: "numeric",
      minute: "2-digit",
    }).format(new Date(value));
  } catch {
    return value;
  }
}

function labelize(value: string | null | undefined) {
  if (!value) return "—";
  return value
    .replace(/_/g, " ")
    .toLowerCase()
    .replace(/\b\w/g, (c) => c.toUpperCase());
}

function DetailRow({
  label,
  value,
}: {
  label: string;
  value: string | number | null | undefined;
}) {
  return (
    <div className="grid gap-1 sm:grid-cols-[160px_1fr] sm:gap-4">
      <div className="text-sm font-medium text-neutral-500">{label}</div>
      <div className="text-sm text-neutral-900">{value ?? "—"}</div>
    </div>
  );
}

export default async function DashboardLeadDetailPage({
  params,
}: {
  params: Promise<{ id: string }>;
}) {
  const { id } = await params;
  const supabase = createSupabaseServiceClient();

  const { data, error } = await supabase
    .from("leads")
    .select(
      `
      id,
      created_at,
      name,
      email,
      phone,
      lead_type,
      status,
      timeline,
      source,
      segment,
      intent,
      message,
      budget,
      move_in_date,
      areas,
      screening_profile,
      price_range,
      financing_status,
      property_address,
      seller_goal,
      property_area,
      property_type,
```

```
    ready_to_lease
  `
  )
  .eq("id", id)
  .single();

if (error || !data) {
  notFound();
}

const lead = data as LeadDetail;

return (
  <main className="min-h-screen bg-[#f5f5f3] text-neutral-950">
    <section className="mx-auto max-w-5xl px-6 py-12">
      <div className="mb-8 flex flex-wrap items-center justify-between gap-4">
        <div>
          <p className="text-sm font-medium uppercase tracking-[0.18em] text-neutral-500">
            Dashboard
          </p>
          <h1 className="mt-3 text-3xl font-semibold tracking-tight md:text-4xl">
            {lead.name}
          </h1>
          <p className="mt-3 text-sm leading-7 text-neutral-600">
            Lead detail view for internal review and follow-up.
          </p>
        </div>

        <Link
          href="/dashboard/leads"
          className="inline-flex h-11 items-center justify-center rounded-full border border-black/10 px-5 text-sm font-medium text-neutral-900 transition hover:border-black/20 hover:bg-white"
        >
          Back to leads
        </Link>
      </div>

      <div className="rounded-[1.75rem] border border-black/5 bg-white p-6 md:p-8">
        <div className="grid gap-8">
          <DetailRow label="Created" value={formatDate(lead.created_at)} />
          <DetailRow label="Lead Type" value={labelize(lead.lead_type)} />

          <div className="grid gap-1 sm:grid-cols-[160px_1fr] sm:gap-4">
            <div className="text-sm font-medium text-neutral-500">Status</div>
            <LeadStatusSelect
              leadId={lead.id}
              currentStatus={lead.status}
            />
          </div>

          <DetailRow label="Timeline" value={labelize(lead.timeline)} />
          <DetailRow label="Source" value={labelize(lead.source)} />
          <DetailRow label="Segment" value={labelize(lead.segment)} />
          <DetailRow label="Intent" value={labelize(lead.intent)} />

          <div className="border-t border-black/5 pt-8" />

          <DetailRow label="Email" value={lead.email} />
          <DetailRow label="Phone" value={lead.phone} />

          <div className="border-t border-black/5 pt-8" />

          <DetailRow
            label="Budget"
            value={lead.budget != null ? `$${lead.budget}` : "—"}
          />
          <DetailRow label="Move-in Date" value={lead.move_in_date} />
          <DetailRow label="Areas" value={lead.areas} />
          <DetailRow
            label="Screening Profile"
            value={labelize(lead.screening_profile)}
          />

          <DetailRow label="Price Range" value={lead.price_range} />
          <DetailRow
            label="Financing Status"
            value={labelize(lead.financing_status)}
          />

          <DetailRow label="Property Address" value={lead.property_address} />
          <DetailRow label="Seller Goal" value={labelize(lead.seller_goal)} />

          <DetailRow label="Property Area" value={lead.property_area} />
          <DetailRow
            label="Property Type"
            value={labelize(lead.property_type)}
          />
          <DetailRow
            label="Ready To Lease"
            value={labelize(lead.ready_to_lease)}
          />

          <div className="border-t border-black/5 pt-8" />

          <div className="grid gap-1 sm:grid-cols-[160px_1fr] sm:gap-4">
            <div className="text-sm font-medium text-neutral-500">
              Message
            </div>
            <div className="whitespace-pre-wrap text-sm leading-7 text-neutral-900">
              {lead.message || "—"}
            </div>
          </div>
        </div>
      </div>
    </section>
  </main>
);
}
```

Create components/dashboard/LeadStatusSelect.tsx
If you do not already have it, create this file and paste this code:

```
"use client";

import { useRouter } from "next/navigation";
import { useState, useTransition } from "react";

type Props = {
  leadId: string;
  currentStatus: string | null;
};

const OPTIONS = [
  { value: "new", label: "New" },
  { value: "contacted", label: "Contacted" },
  { value: "qualified", label: "Qualified" },
  { value: "showing", label: "Showing" },
  { value: "application", label: "Application" },
  { value: "closed", label: "Closed" },
  { value: "lost", label: "Lost" },
];

export default function LeadStatusSelect({
  leadId,
  currentStatus,
}: Props) {
  const router = useRouter();
  const [status, setStatus] = useState(currentStatus ?? "new");
  const [error, setError] = useState("");
  const [isPending, startTransition] = useTransition();

  async function updateStatus(nextStatus: string) {
    setError("");
    setStatus(nextStatus);

    try {
      const res = await fetch("/api/lead-status", {
        method: "POST",
        headers: {
          "Content-Type": "application/json",
        },
        body: JSON.stringify({
          id: leadId,
          status: nextStatus,
        }),
      });

      const data = await res.json().catch(() => ({}));

      if (!res.ok) {
        setError(
          typeof data?.error === "string"
            ? data.error
            : "Failed to update status."
        );
        setStatus(currentStatus ?? "new");
        return;
      }

      startTransition(() => {
        router.refresh();
      });
    } catch {
      setError("Network error while updating status.");
      setStatus(currentStatus ?? "new");
    }
  }

  return (
    <div className="space-y-2">
      <select
        value={status}
        disabled={isPending}
        onChange={(e) => updateStatus(e.target.value)}
        className="rounded-lg border border-neutral-300 bg-white px-3 py-2 text-sm text-neutral-900"
      >
        {OPTIONS.map((option) => (
          <option key={option.value} value={option.value}>
            {option.label}
          </option>
        ))}
      </select>

      {error ? <p className="text-xs text-red-600">{error}</p> : null}
    </div>
  );
}
```

Make sure app/api/lead-status/route.ts **is this**

```
import { NextResponse } from "next/server";
import { createSupabaseServiceClient } from "@/lib/supabase/service";

const ALLOWED_STATUS = [
  "new",
  "contacted",
  "qualified",
  "showing",
  "application",
  "closed",
  "lost",
] as const;

export async function POST(req: Request) {
  try {
    const body = await req.json().catch(() => null);
```

```
    const id = body?.id;
    const status = body?.status;

    if (!id || typeof id !== "string") {
      return NextResponse.json(
        { ok: false, error: "Missing lead id." },
        { status: 400 }
      );
    }

    if (
      !status ||
      typeof status !== "string" ||
      !ALLOWED_STATUS.includes(status as (typeof ALLOWED_STATUS)[number])
    ) {
      return NextResponse.json(
        { ok: false, error: "Invalid status value." },
        { status: 400 }
      );
    }

    const supabase = createSupabaseServiceClient();

    const { error } = await supabase
      .from("leads")
      .update({ status })
      .eq("id", id);

    if (error) {
      console.error("lead-status update failed:", error);
      return NextResponse.json(
        { ok: false, error: "Database update failed." },
        { status: 500 }
      );
    }

    return NextResponse.json({ ok: true });
  } catch (err) {
    console.error("lead-status route crashed:", err);
    return NextResponse.json(
      { ok: false, error: "Server error." },
      { status: 500 }
    );
  }
}
```

Then do this
Save all three files, restart dev:

```
npm run dev
```

Then test:

1. open /dashboard/leads
2. click a lead
3. change status from New to Contacted
4. go back to /dashboard/leads
5. confirm the badge changed

If it still fails, paste the exact Terminal error and I'll isolate the broken point.

Create `components/dashboard/LeadStatusSelect.tsx`
lets do this in Terminal

Run these exact Terminal commands.
1. Confirm you are in the project

```
pwd
```

You should be at:

/Users/abraham/Desktop/zehaifirealty

2. Create the component folder

```
mkdir -p 'components/dashboard'
```

3. Create the status select file

```
touch 'components/dashboard/LeadStatusSelect.tsx'
```

4. Confirm it exists

```
ls 'components/dashboard'
```

You should see:

LeadStatusSelect.tsx

5. Open it in VS Code

```
code 'components/dashboard/LeadStatusSelect.tsx'
```

6. Paste this into components/dashboard/LeadStatusSelect.tsx

```
"use client";

import { useRouter } from "next/navigation";
import { useState, useTransition } from "react";

type Props = {
  leadId: string;
  currentStatus: string | null;
};

const OPTIONS = [
  { value: "new", label: "New" },
  { value: "contacted", label: "Contacted" },
```

```
  { value: "qualified", label: "Qualified" },
  { value: "showing", label: "Showing" },
  { value: "application", label: "Application" },
  { value: "closed", label: "Closed" },
  { value: "lost", label: "Lost" },
];

export default function LeadStatusSelect({
  leadId,
  currentStatus,
}: Props) {
  const router = useRouter();
  const [status, setStatus] = useState(currentStatus ?? "new");
  const [error, setError] = useState("");
  const [isPending, startTransition] = useTransition();

  async function updateStatus(nextStatus: string) {
    setError("");
    setStatus(nextStatus);

    try {
      const res = await fetch("/api/lead-status", {
        method: "POST",
        headers: {
          "Content-Type": "application/json",
        },
        body: JSON.stringify({
          id: leadId,
          status: nextStatus,
        }),
      });

      const data = await res.json().catch(() => ({}));

      if (!res.ok) {
        setError(
          typeof data?.error === "string"
            ? data.error
            : "Failed to update status."
        );
        setStatus(currentStatus ?? "new");
        return;
      }

      startTransition(() => {
        router.refresh();
      });
    } catch {
      setError("Network error while updating status.");
      setStatus(currentStatus ?? "new");
    }
  }

  return (
    <div className="space-y-2">
      <select
        value={status}
        disabled={isPending}
        onChange={(e) => updateStatus(e.target.value)}
        className="rounded-lg border border-neutral-300 bg-white px-3 py-2 text-sm text-neutral-900"
      >
        {OPTIONS.map((option) => (
          <option key={option.value} value={option.value}>
            {option.label}
          </option>
        ))}
      </select>

      {error ? <p className="text-xs text-red-600">{error}</p> : null}
    </div>
  );
}
```

7. Verify from Terminal

```
cat 'components/dashboard/LeadStatusSelect.tsx'
```

8. Then open the lead detail page

```
code 'app/dashboard/leads/[id]/page.tsx'
```

9. Replace that file with the corrected version
Paste this entire file:

```
import Link from "next/link";
import { notFound } from "next/navigation";
import { createSupabaseServiceClient } from "@/lib/supabase/service";
import LeadStatusSelect from "@/components/dashboard/LeadStatusSelect";

type LeadDetail = {
  id: string;
  created_at: string;
  name: string;
  email: string;
  phone: string;
  lead_type: string;
  status: string | null;
  timeline: string | null;
  source: string | null;
  segment: string | null;
  intent: string | null;
  message: string | null;
  budget: number | null;
  move_in_date: string | null;
  areas: string | null;
  screening_profile: string | null;
  price_range: string | null;
  financing_status: string | null;
```

```
  property_address: string | null;
  seller_goal: string | null;
  property_area: string | null;
  property_type: string | null;
  ready_to_lease: string | null;
};

function formatDate(value: string) {
  try {
    return new Intl.DateTimeFormat("en-US", {
      month: "long",
      day: "numeric",
      year: "numeric",
      hour: "numeric",
      minute: "2-digit",
    }).format(new Date(value));
  } catch {
    return value;
  }
}

function labelize(value: string | null | undefined) {
  if (!value) return "—";
  return value
    .replace(/_/g, " ")
    .toLowerCase()
    .replace(/\b\w/g, (c) => c.toUpperCase());
}

function DetailRow({
  label,
  value,
}: {
  label: string;
  value: string | number | null | undefined;
}) {
  return (
    <div className="grid gap-1 sm:grid-cols-[160px_1fr] sm:gap-4">
      <div className="text-sm font-medium text-neutral-500">{label}</div>
      <div className="text-sm text-neutral-900">{value ?? "—"}</div>
    </div>
  );
}

export default async function DashboardLeadDetailPage({
  params,
}: {
  params: Promise<{ id: string }>;
}) {
  const { id } = await params;
  const supabase = createSupabaseServiceClient();

  const { data, error } = await supabase
    .from("leads")
    .select(
      `
      id,
      created_at,
      name,
      email,
      phone,
      lead_type,
      status,
      timeline,
      source,
      segment,
      intent,
      message,
      budget,
      move_in_date,
      areas,
      screening_profile,
      price_range,
      financing_status,
      property_address,
      seller_goal,
      property_area,
      property_type,
      ready_to_lease
    `
    )
    .eq("id", id)
    .single();

  if (error || !data) {
    notFound();
  }

  const lead = data as LeadDetail;

  return (
    <main className="min-h-screen bg-[#f5f5f3] text-neutral-950">
      <section className="mx-auto max-w-5xl px-6 py-12">
        <div className="mb-8 flex flex-wrap items-center justify-between gap-4">
          <div>
            <p className="text-sm font-medium uppercase tracking-[0.18em] text-neutral-500">
              Dashboard
            </p>
            <h1 className="mt-3 text-3xl font-semibold tracking-tight md:text-4xl">
              {lead.name}
            </h1>
            <p className="mt-3 text-sm leading-7 text-neutral-600">
              Lead detail view for internal review and follow-up.
            </p>
          </div>

          <Link
            href="/dashboard/leads"
```

```
          className="inline-flex h-11 items-center justify-center rounded-full border border-black/10 px-5 text-sm font-medium text-neutral-900 transition hover:border-black/20 hover:bg-white"
        >
          Back to leads
        </Link>
      </div>

      <div className="rounded-[1.75rem] border border-black/5 bg-white p-6 md:p-8">
        <div className="grid gap-8">
          <DetailRow label="Created" value={formatDate(lead.created_at)} />
          <DetailRow label="Lead Type" value={labelize(lead.lead_type)} />

          <div className="grid gap-1 sm:grid-cols-[160px_1fr] sm:gap-4">
            <div className="text-sm font-medium text-neutral-500">Status</div>
            <LeadStatusSelect
              leadId={lead.id}
              currentStatus={lead.status}
            />
          </div>

          <DetailRow label="Timeline" value={labelize(lead.timeline)} />
          <DetailRow label="Source" value={labelize(lead.source)} />
          <DetailRow label="Segment" value={labelize(lead.segment)} />
          <DetailRow label="Intent" value={labelize(lead.intent)} />

          <div className="border-t border-black/5 pt-8" />

          <DetailRow label="Email" value={lead.email} />
          <DetailRow label="Phone" value={lead.phone} />

          <div className="border-t border-black/5 pt-8" />

          <DetailRow
            label="Budget"
            value={lead.budget != null ? `$${lead.budget}` : "—"}
          />
          <DetailRow label="Move-in Date" value={lead.move_in_date} />
          <DetailRow label="Areas" value={lead.areas} />
          <DetailRow
            label="Screening Profile"
            value={labelize(lead.screening_profile)}
          />

          <DetailRow label="Price Range" value={lead.price_range} />
          <DetailRow
            label="Financing Status"
            value={labelize(lead.financing_status)}
          />

          <DetailRow label="Property Address" value={lead.property_address} />
          <DetailRow label="Seller Goal" value={labelize(lead.seller_goal)} />

          <DetailRow label="Property Area" value={lead.property_area} />
          <DetailRow
            label="Property Type"
            value={labelize(lead.property_type)}
          />
          <DetailRow
            label="Ready To Lease"
            value={labelize(lead.ready_to_lease)}
          />

          <div className="border-t border-black/5 pt-8" />

          <div className="grid gap-1 sm:grid-cols-[160px_1fr] sm:gap-4">
            <div className="text-sm font-medium text-neutral-500">
              Message
            </div>
            <div className="whitespace-pre-wrap text-sm leading-7 text-neutral-900">
              {lead.message || "—"}
            </div>
          </div>
        </div>
      </div>
    </section>
  </main>
);
}
```

10. Make sure the API route exists
Run:

```
ls 'app/api/lead-status'
```

If it does not exist, run:

```
mkdir -p 'app/api/lead-status'
touch 'app/api/lead-status/route.ts'
code 'app/api/lead-status/route.ts'
```

Paste this:

```
import { NextResponse } from "next/server";
import { createSupabaseServiceClient } from "@/lib/supabase/service";

const ALLOWED_STATUS = [
  "new",
  "contacted",
  "qualified",
  "showing",
  "application",
  "closed",
  "lost",
] as const;

export async function POST(req: Request) {
  try {
    const body = await req.json().catch(() => null);
```

```
    const id = body?.id;
    const status = body?.status;

    if (!id || typeof id !== "string") {
      return NextResponse.json(
        { ok: false, error: "Missing lead id." },
        { status: 400 }
      );
    }

    if (
      !status ||
      typeof status !== "string" ||
      !ALLOWED_STATUS.includes(status as (typeof ALLOWED_STATUS)[number])
    ) {
      return NextResponse.json(
        { ok: false, error: "Invalid status value." },
        { status: 400 }
      );
    }

    const supabase = createSupabaseServiceClient();

    const { error } = await supabase
      .from("leads")
      .update({ status })
      .eq("id", id);

    if (error) {
      console.error("lead-status update failed:", error);
      return NextResponse.json(
        { ok: false, error: "Database update failed." },
        { status: 500 }
      );
    }

    return NextResponse.json({ ok: true });
  } catch (err) {
    console.error("lead-status route crashed:", err);
    return NextResponse.json(
      { ok: false, error: "Server error." },
      { status: 500 }
    );
  }
}
```

11. Restart dev

```
npm run dev
```

12. Test

- open /dashboard/leads
- click a lead
- change status
- go back to /dashboard/leads
- confirm the badge changed

If it fails, paste:

- cat 'components/dashboard/LeadStatusSelect.tsx'
- cat 'app/api/lead-status/route.ts'
- the Terminal error after you try changing the status